AWS FINAL EXAM CONTENT

AWS HANDS-ON PRACTICE & NETWORK SECURITY

DESIGN & DEPLOY HIGHLY AVAILABLE WEB ARCHITECTURE ON AWS

Created by **Hussain Ali**

Hussain Ali

NETWORKING

AWS Final Exam Content

This document provides an overview of the material related to your final exam. Use it as a reference to support your study.

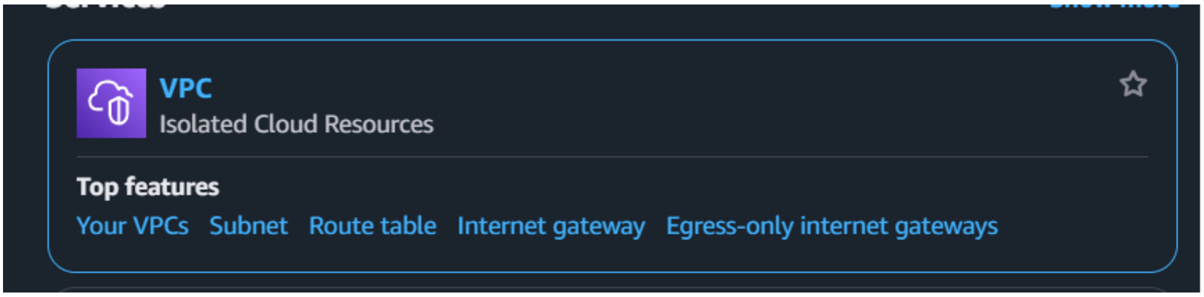
Note: This document may have some errors. Please refer to the **official PPT** as your primary study resource.

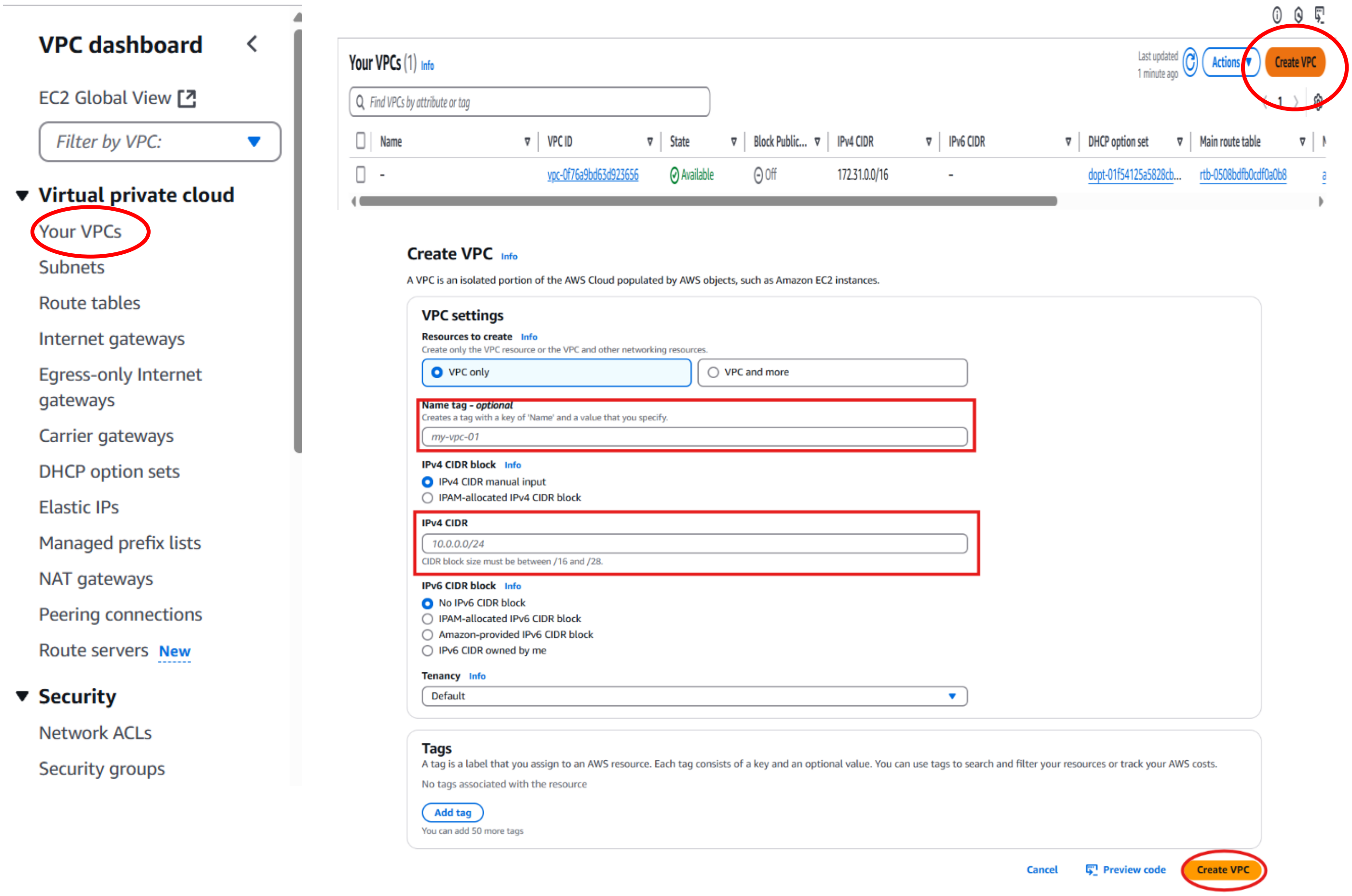
This document is open source, created by Hussain Ali

Table of Contents

How to Create a VPC

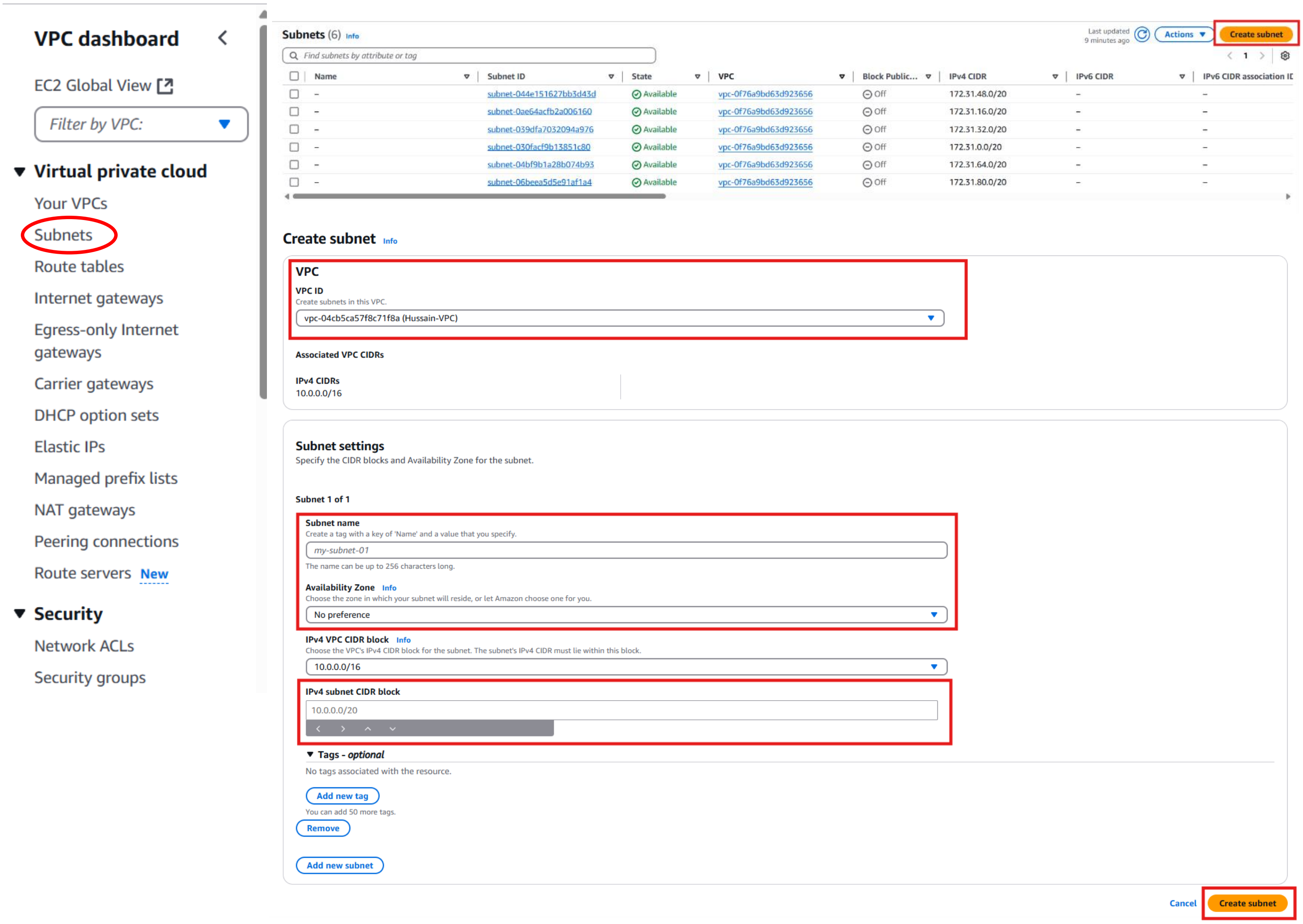
Create VPC





Write the name of the VPC and the IPv4 CIDR (10.0.0.0/16), then choose **Create VPC**.

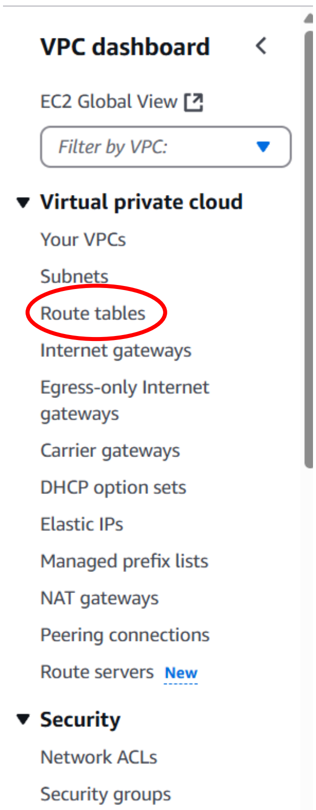
Create subnets

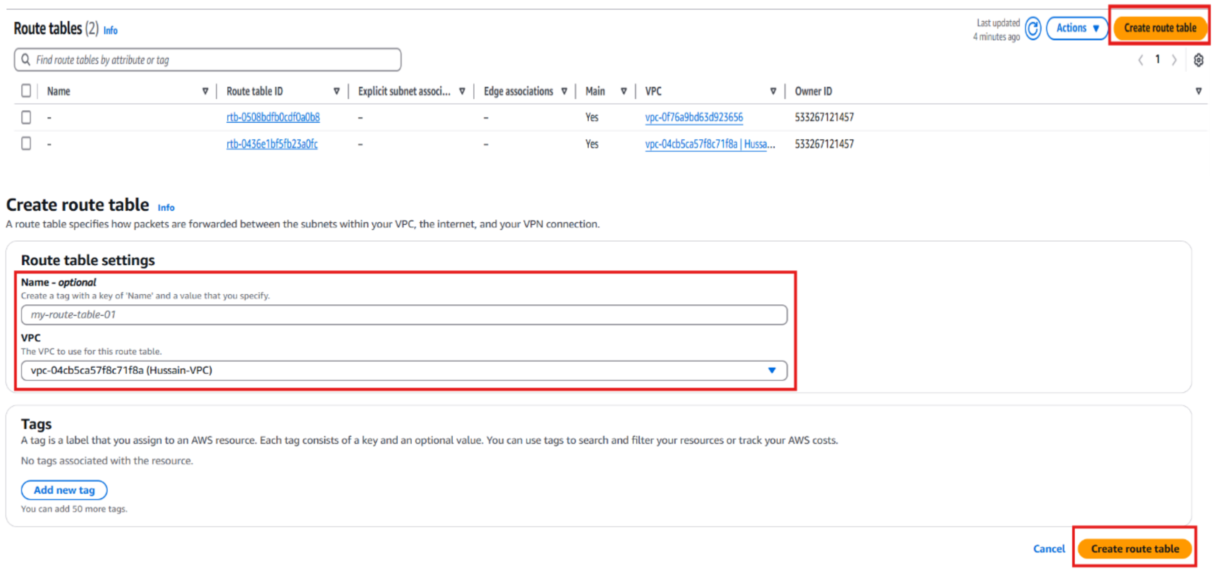


First, choose the VPC that you created before. Secondly, enter a name for your subnet. Thirdly, choose the Availability Zone. Lastly, enter the IPv4 subnet and click on create subnet.

Note: In subnets, you must create two separate subnets, one for the public and another for the private, and each subnet must have two different Availability Zones (public, private).

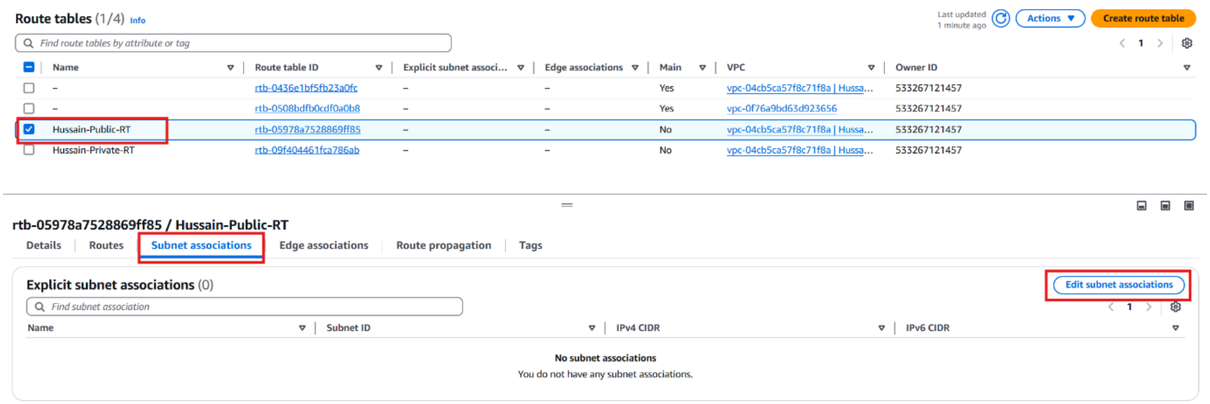
Create Route Table



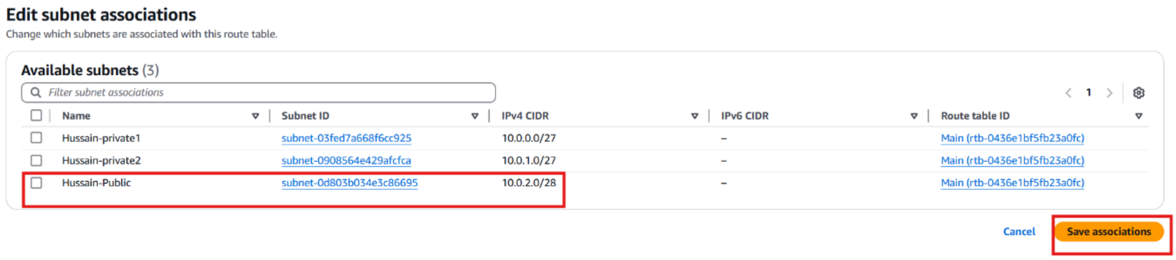


For the route table, first, choose a name. Second, select your VPC that you created before. Click on **Create route table**.

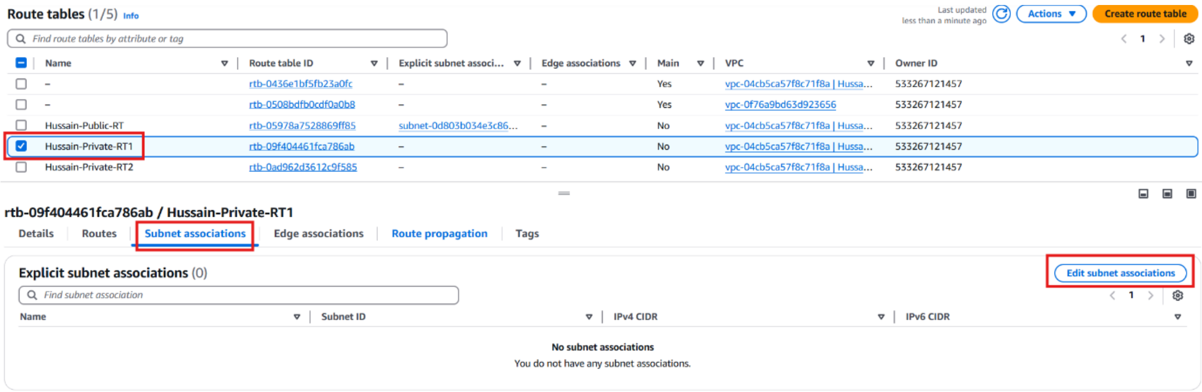
Note: You must create two route tables, one for the public and one for the private subnets.

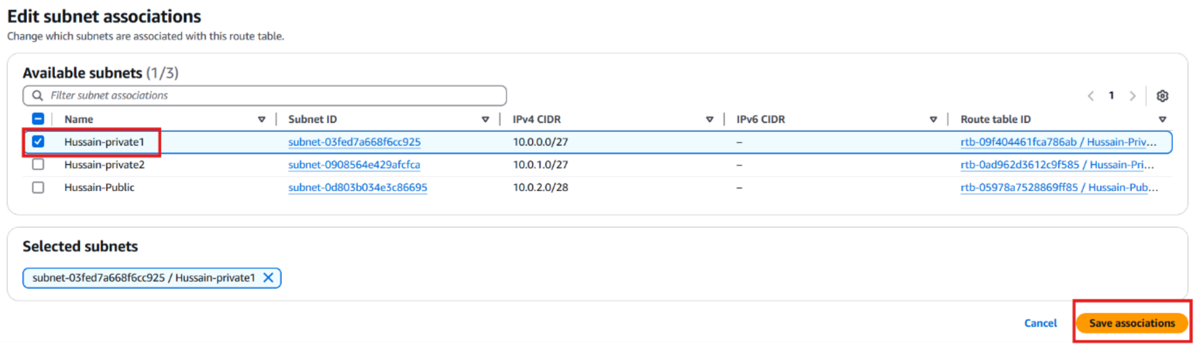


After creating the route table, you must add the subnets. First, choose the public route table and from the drop-down menu select **Subnet associations**, then choose **Edit subnet associations.**



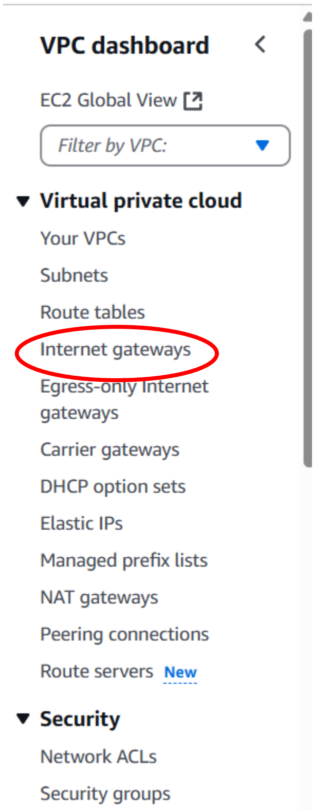
Choose the public subnet, then click save associations.

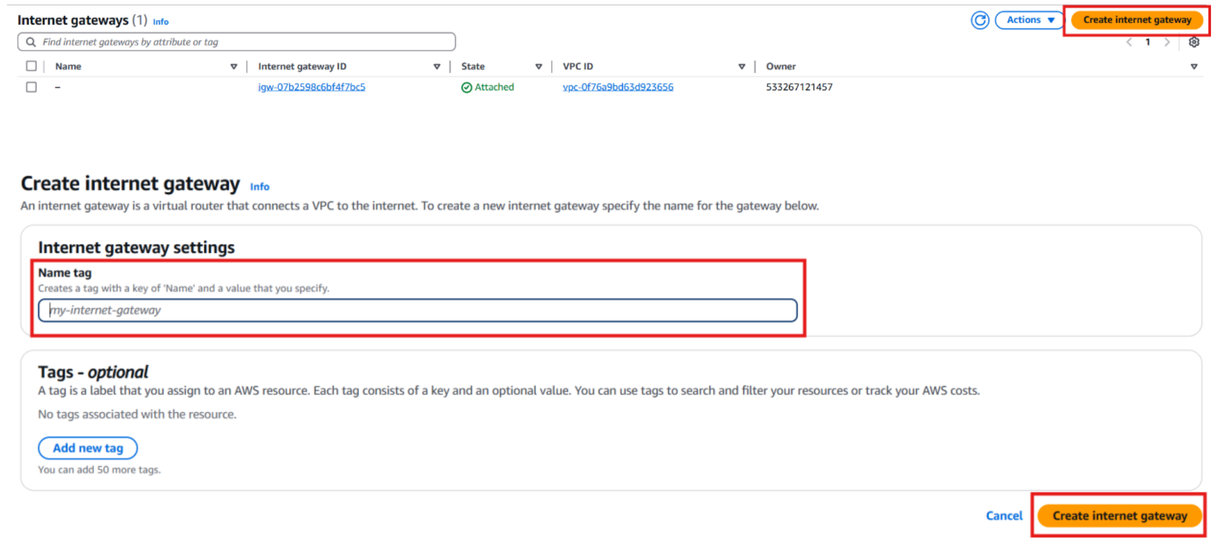




We do the same for the private subnet.

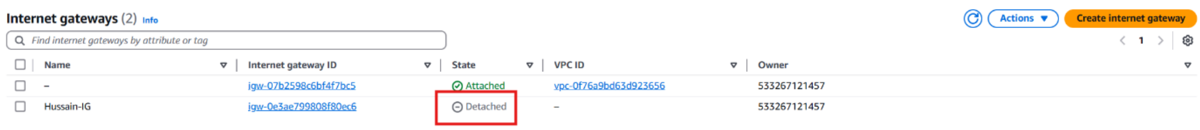
Create Internet Gateways



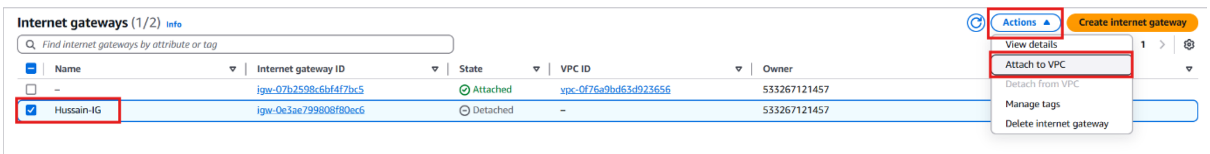


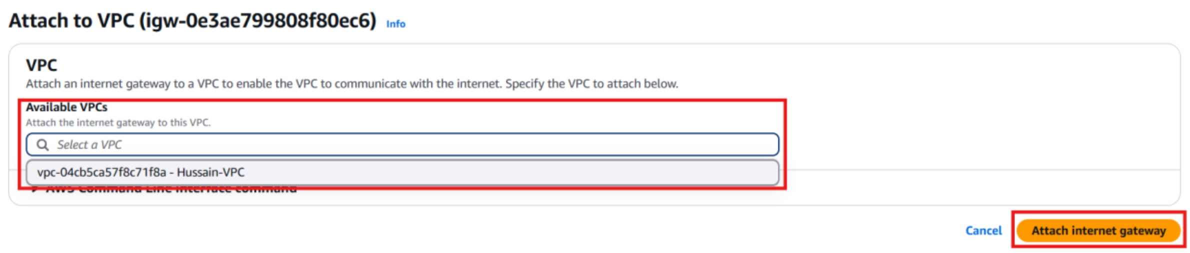
Enter a name for the internet gateway and click Create Internet Gateway.

Note: remember that the internet gateway is the one that will connect you with the internet; without it, you can't access the internet.



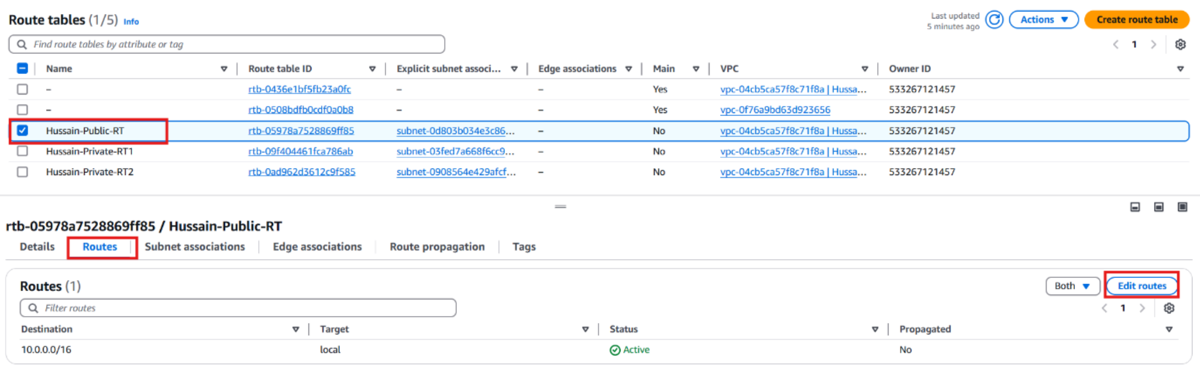
After you create the internet gateway, you will notice that in the state it says that it is Detached; you have to attach it to your VPC.



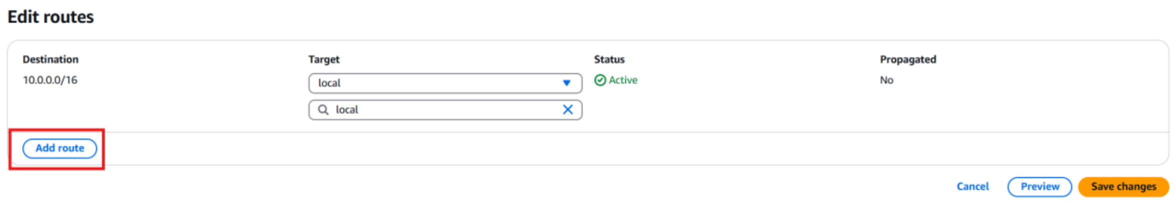


First, **select the internet gateway that you created**. Second, click on Actions, then **Attach to VPC**. Lastly, select your VPC and click on **Attach internet gateway**.

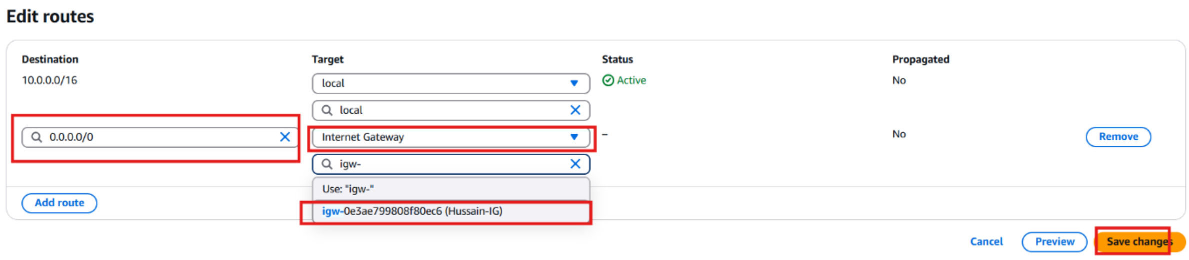
To connect the internet gateway with the public route table to have internet access:



First, choose the public route, then from the drop-down menu choose routes, and then **Edit routes.**



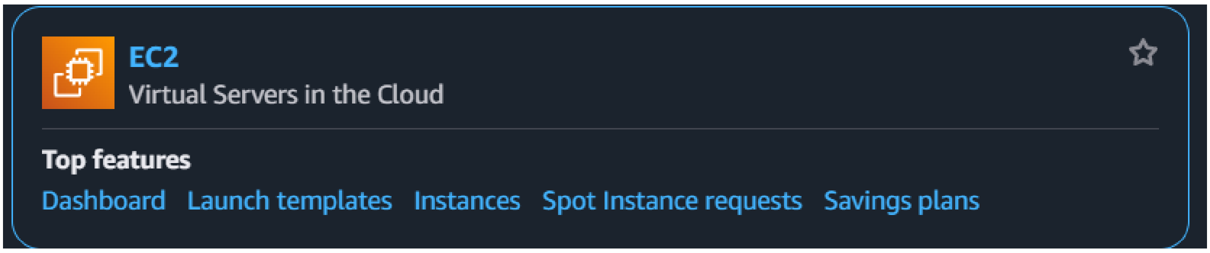
Click on **Add route**.

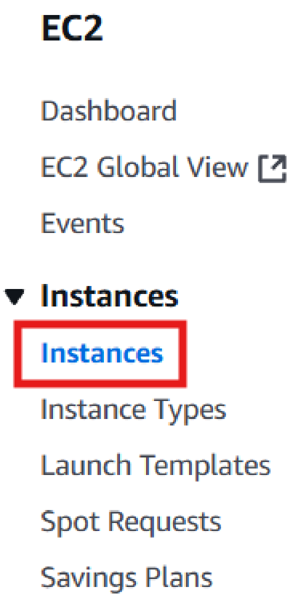


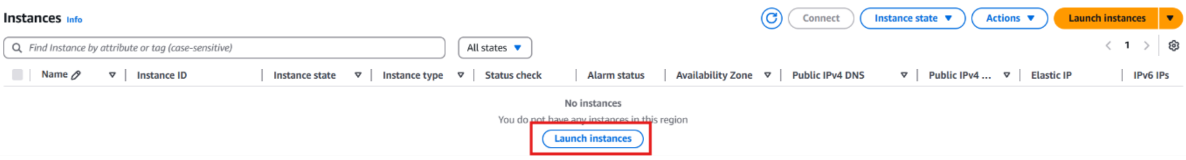
First, choose 0.0.0.0/0, then for the target, choose internet gateway, and **select** the **internet gateway** that you created before. Finally, **click save changes**.

How to Create an EC2 instance

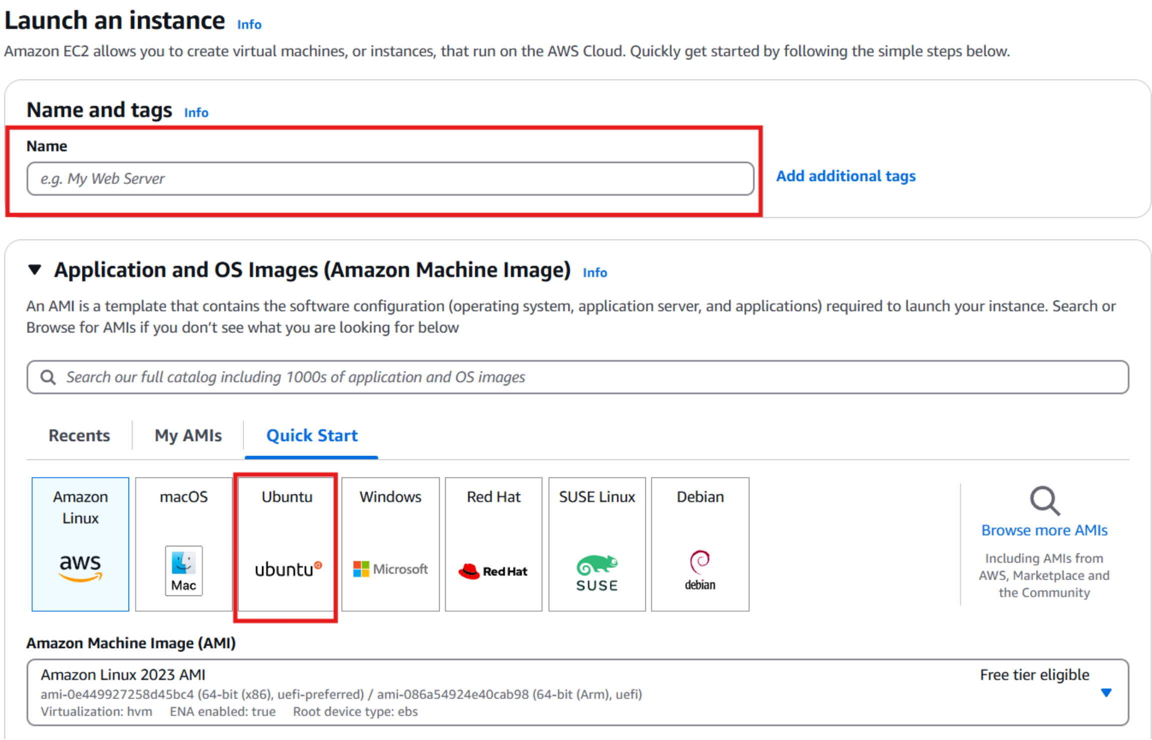
Create Ubuntu EC2



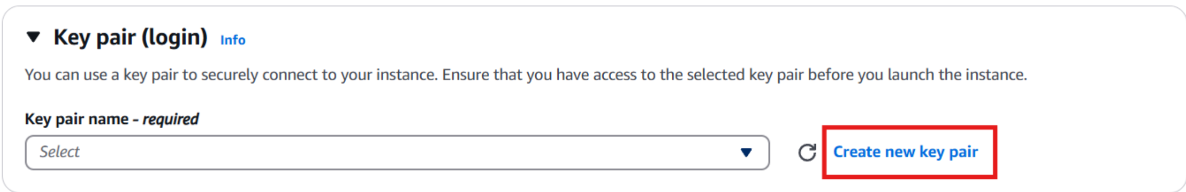




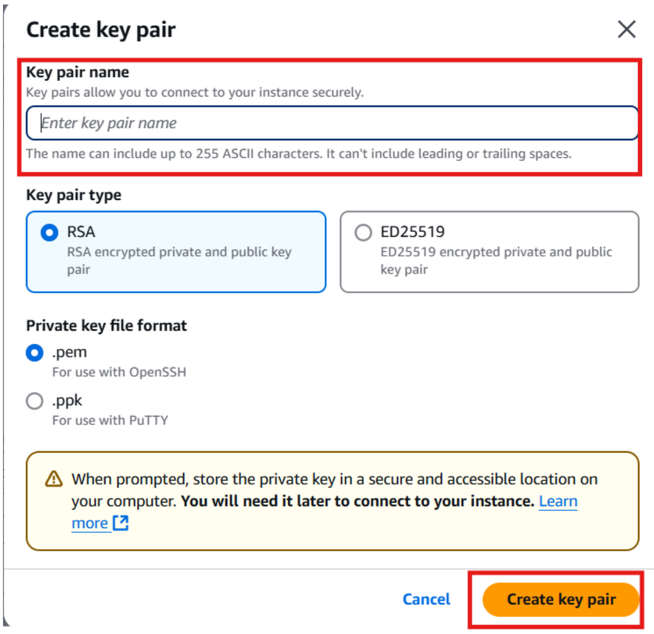
We will create a web server that runs Ubuntu.



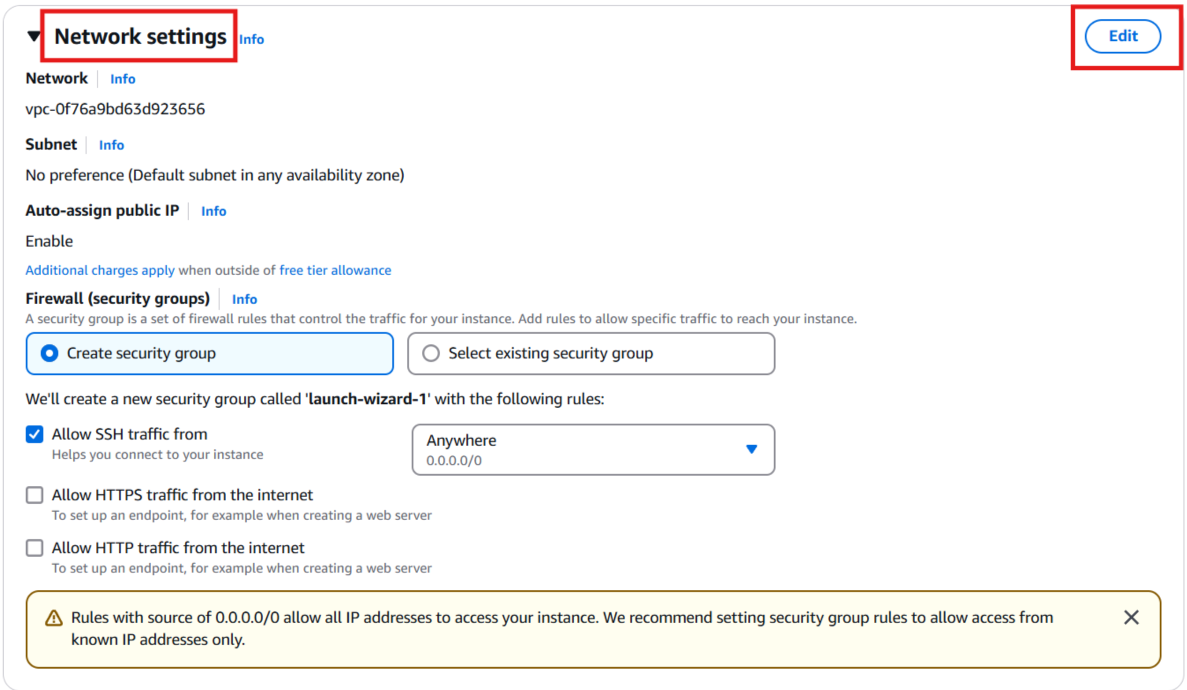
First, enter a name for the web server and choose Ubuntu.



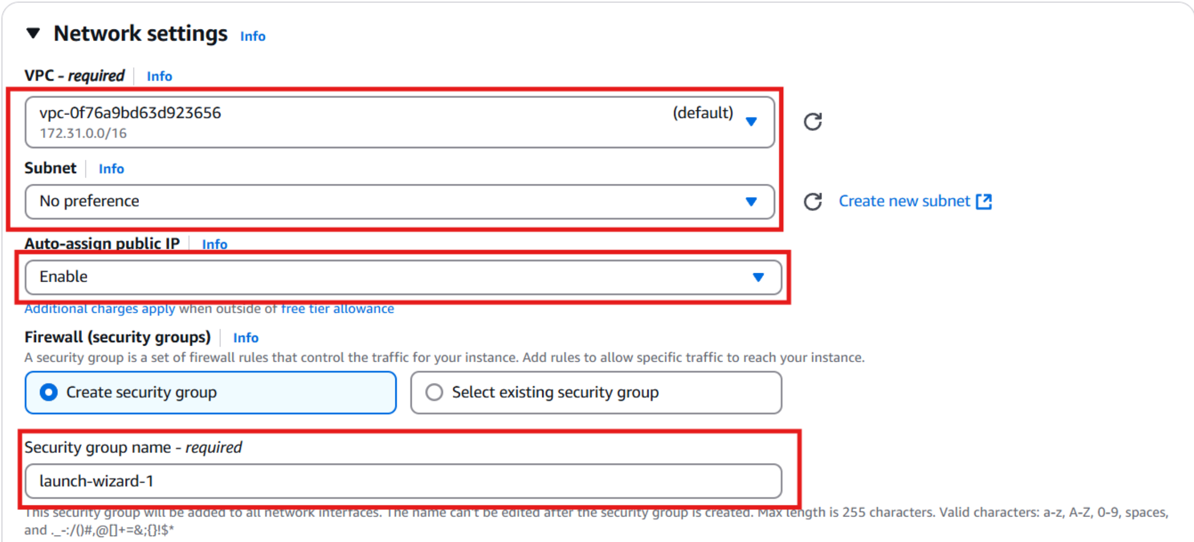
Scroll down to Key pair, choose **Create new Key pair**.



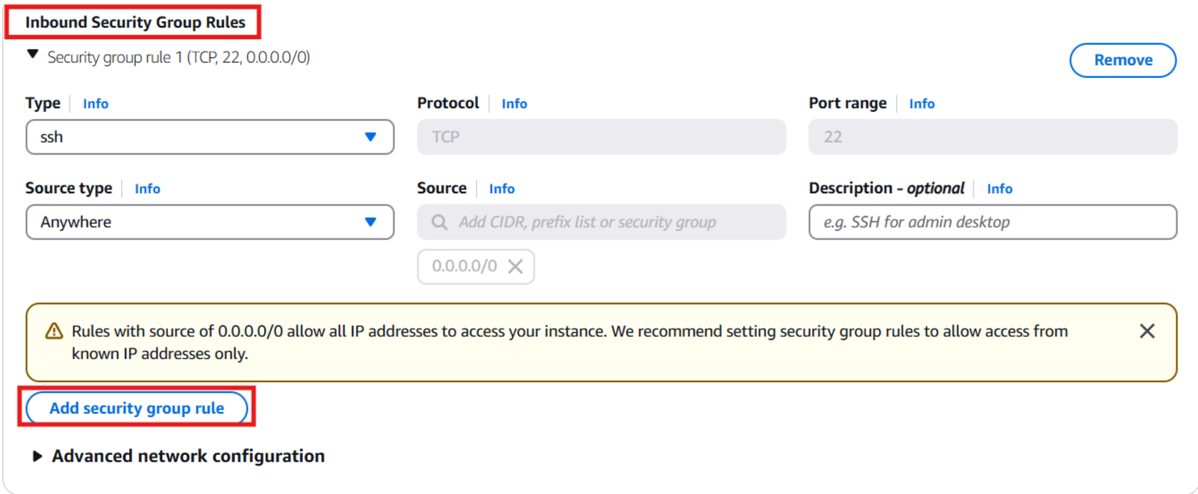
Choose a name for the key pair and click on **Create Key Pair**.



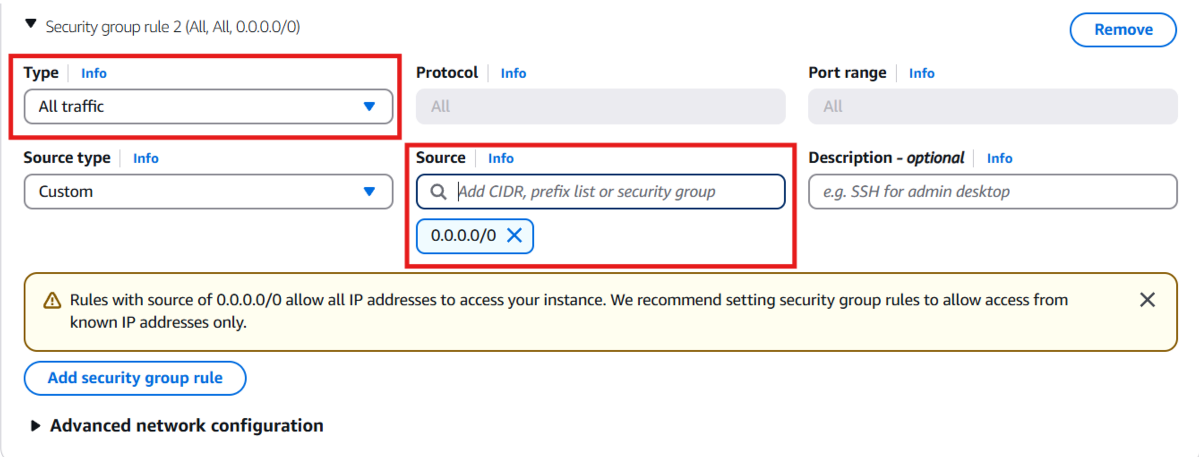
In network settings, click on **edit.**



First, choose your VPC. Second, select the public subnet and make sure that the Auto-assign public IP is enabled. Last, **rename the Security group.**

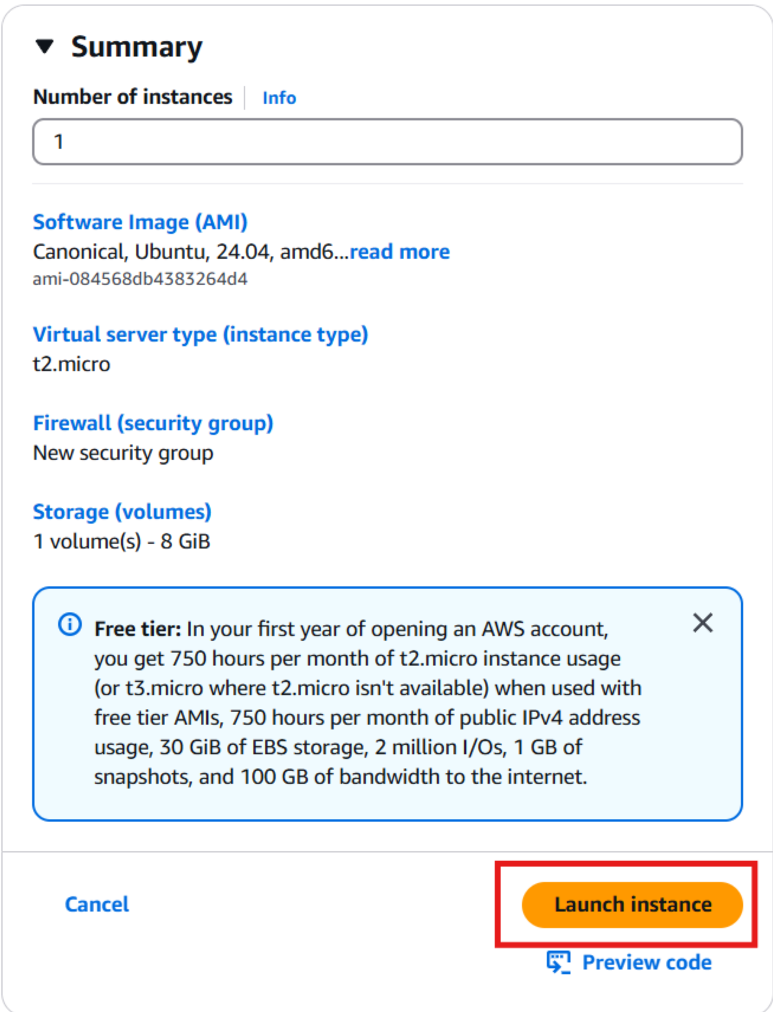


Scroll down in the **Inbound Security Group Rules,** click on **Add security group rule.**

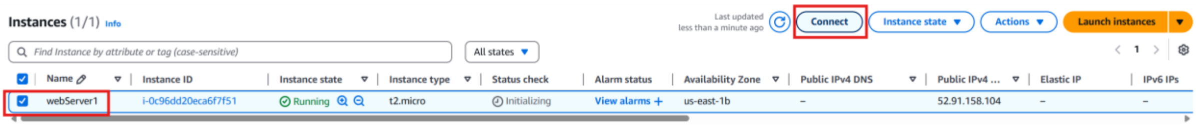


For the type, **choose All traffic,** and for the source, choose 0.0.0.0/0

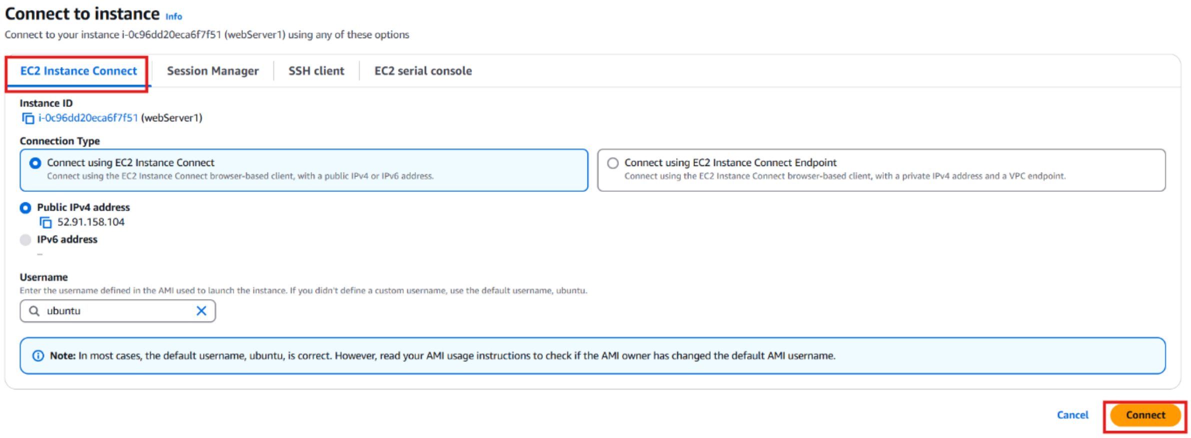
Note: In the exam, you must choose the protocol that is given to you, and you can’t choose all traffic.



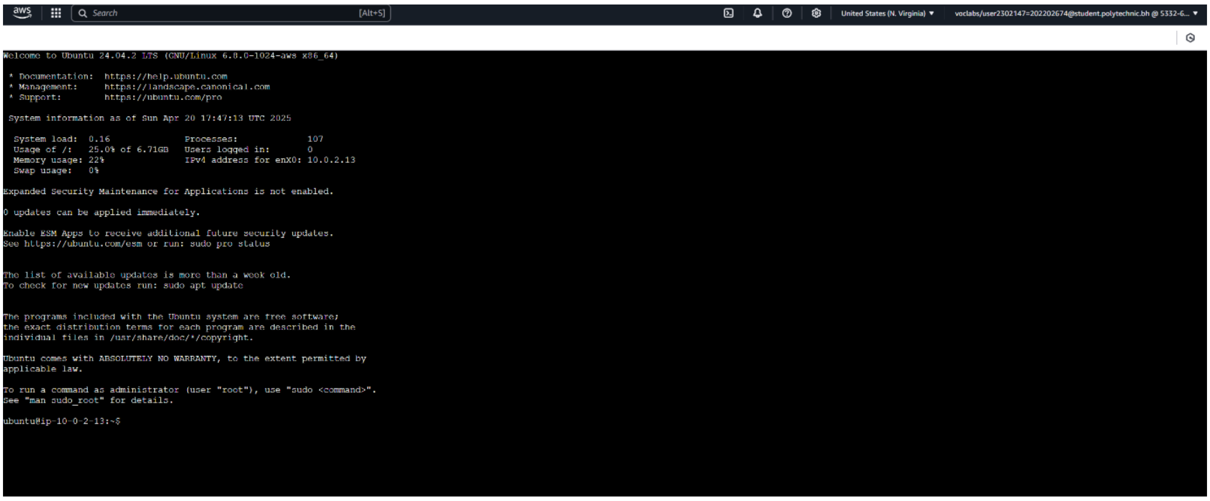
Finally, click on **Launch instance.**



There are two ways to connect to the web server: choosing your instance and clicking on Connect; the [second way](#) will be explained in the Red Hat EC2 instance.

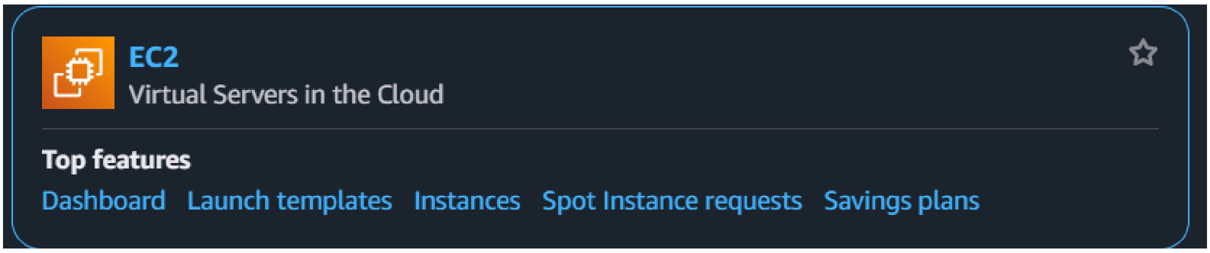


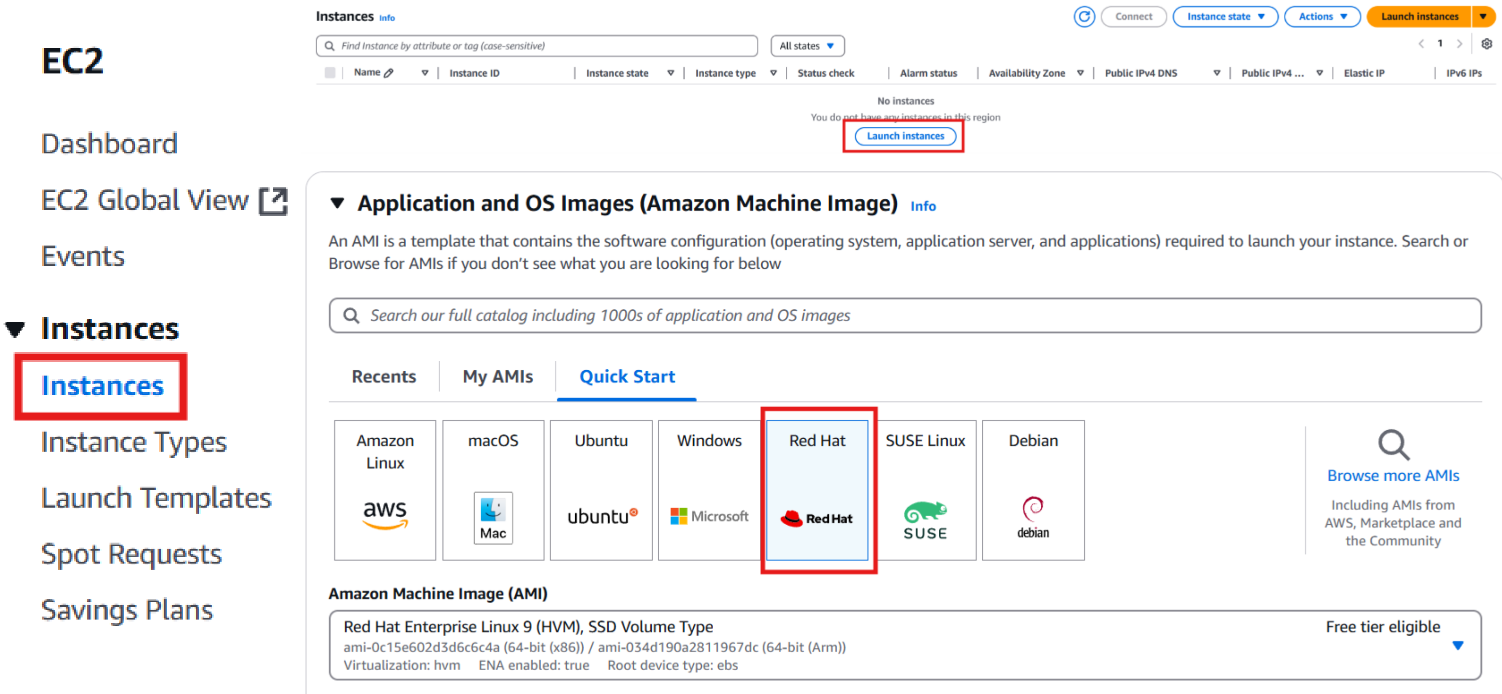
Choose **EC2 Instance Connect and click on Connect.**



If you have done everything correctly, it should open the terminal for you.

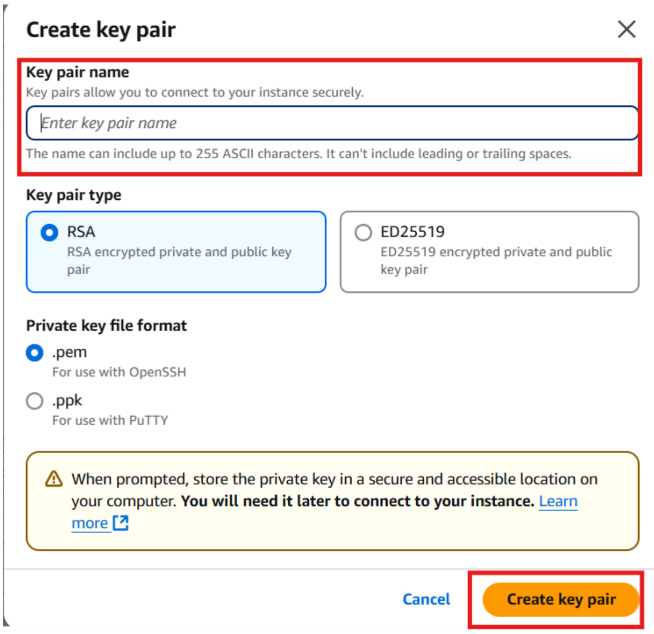
Creating Red Hat EC2:



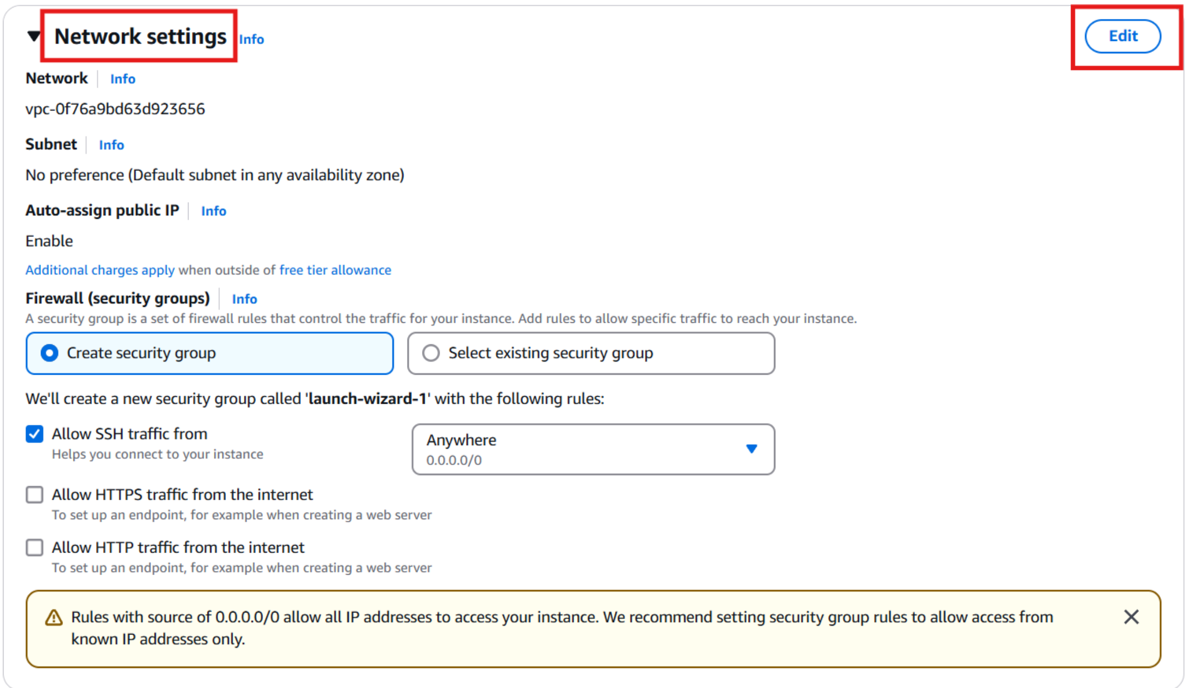


Choose **Red Hat from the Quick Start** after entering the name of the EC2.

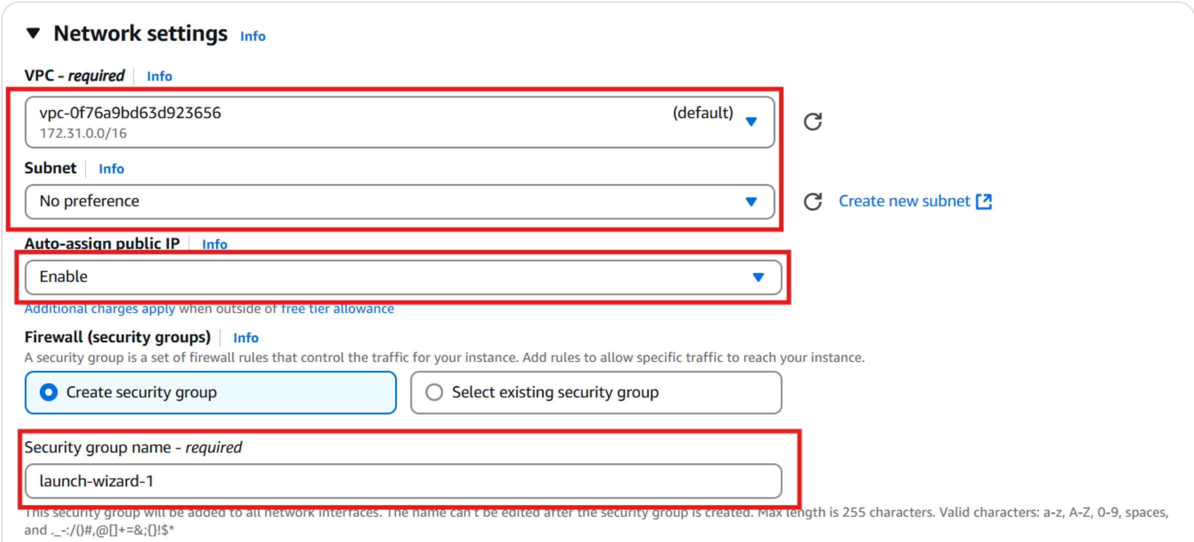
Scroll down to Key pair, choose **Create new Key pair**.



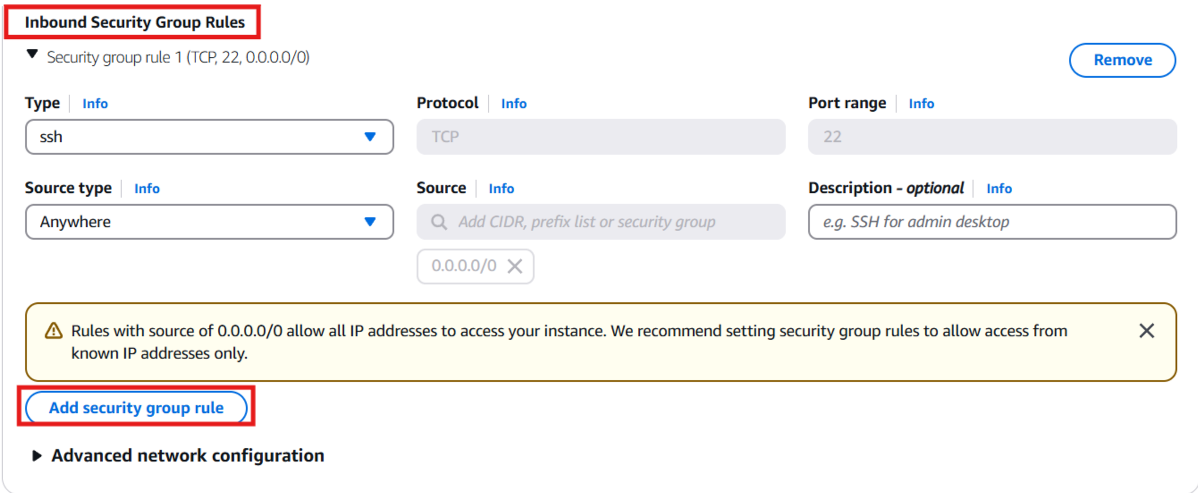
Choose a name for the key pair and click on **Create Key Pair**.



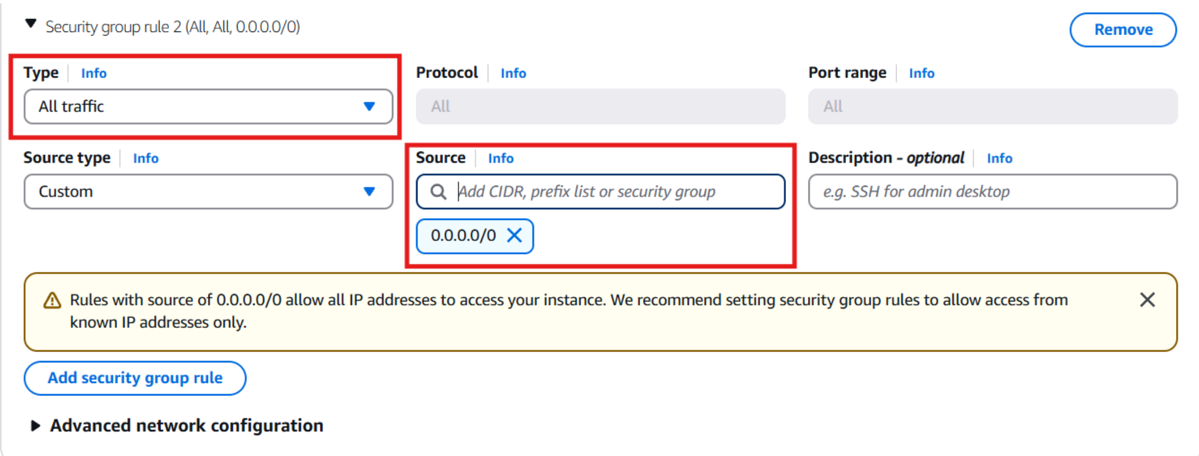
In network settings, click on **edit.**



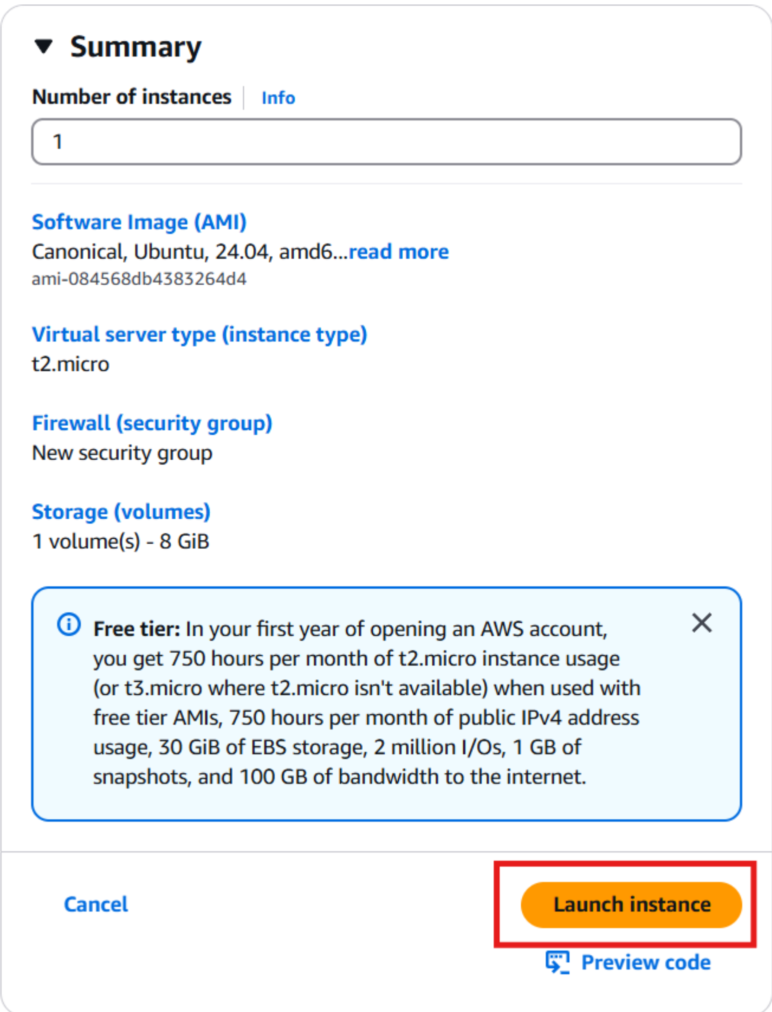
First, choose your VPC. Second, select the public subnet and **make sure** that the **Auto-assign public IP is enabled (Enable).** Last, **rename the Security group.**



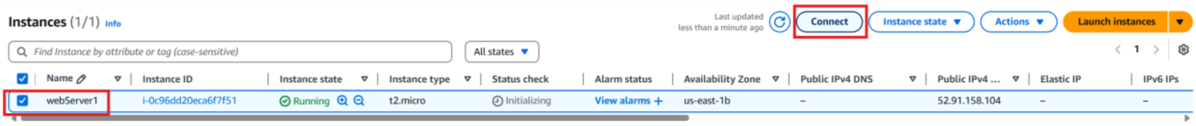
Scroll down in the **Inbound Security Group Rules,** and click on Add **security group rule.**



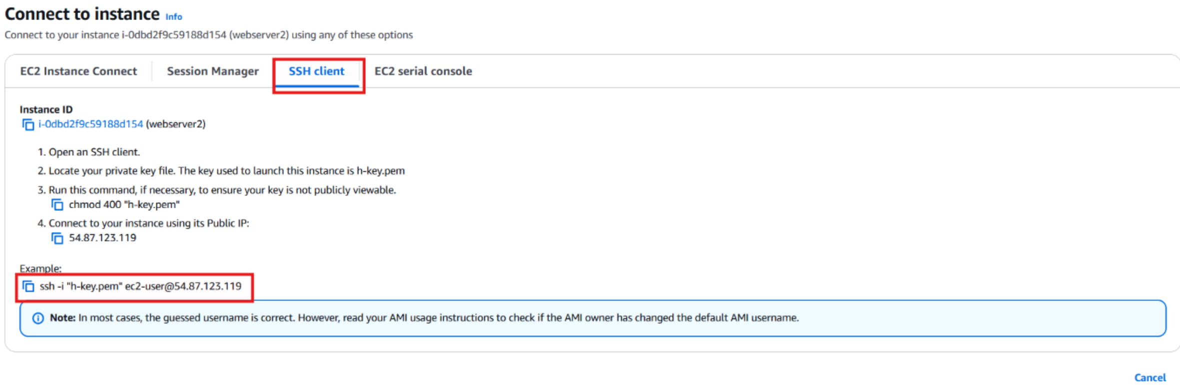
For the type, choose **All traffic,** and for the source, choose **0.0.0.0/0**



Finally, click on **Launch instance.**



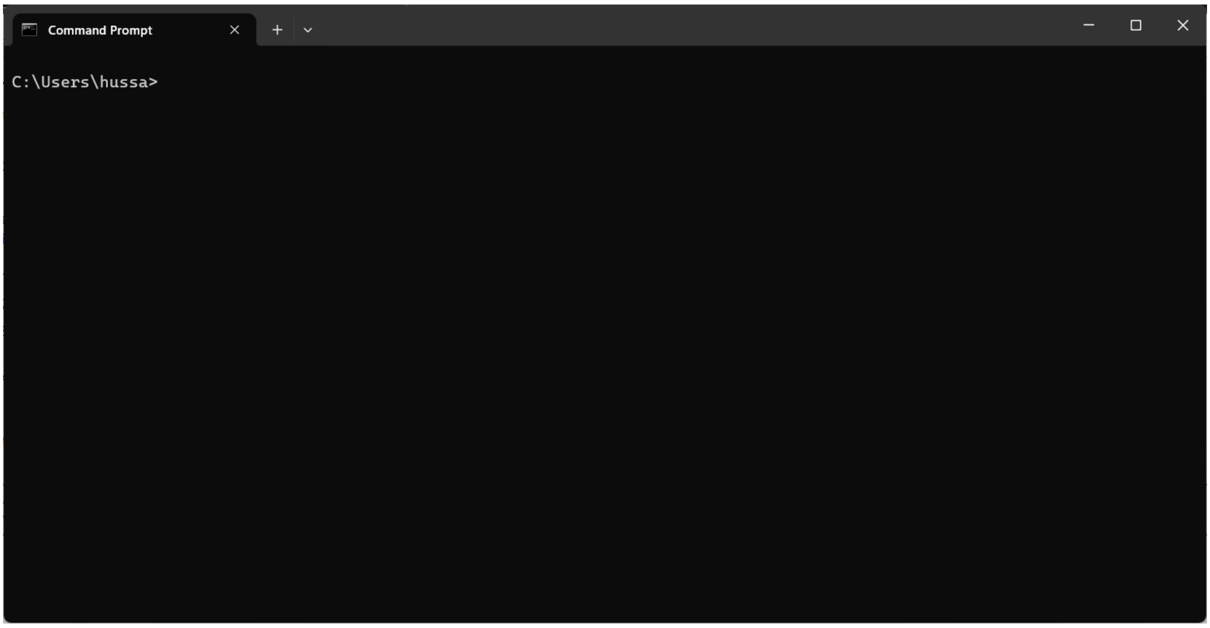
To connect to the web server, **choose your instance** and click on **Connect**.



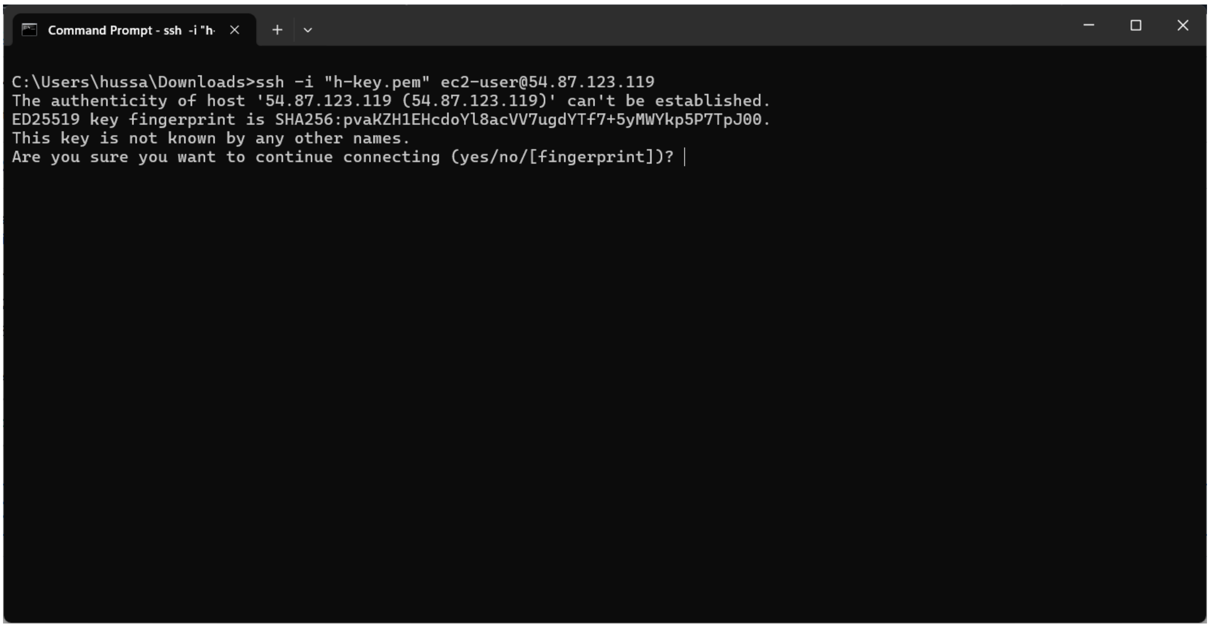
Choose **SSH client and copy the Example.**

Note: Red Hat Linux, you can't connect to it directly like Ubuntu; you have to connect to it from the CMD.

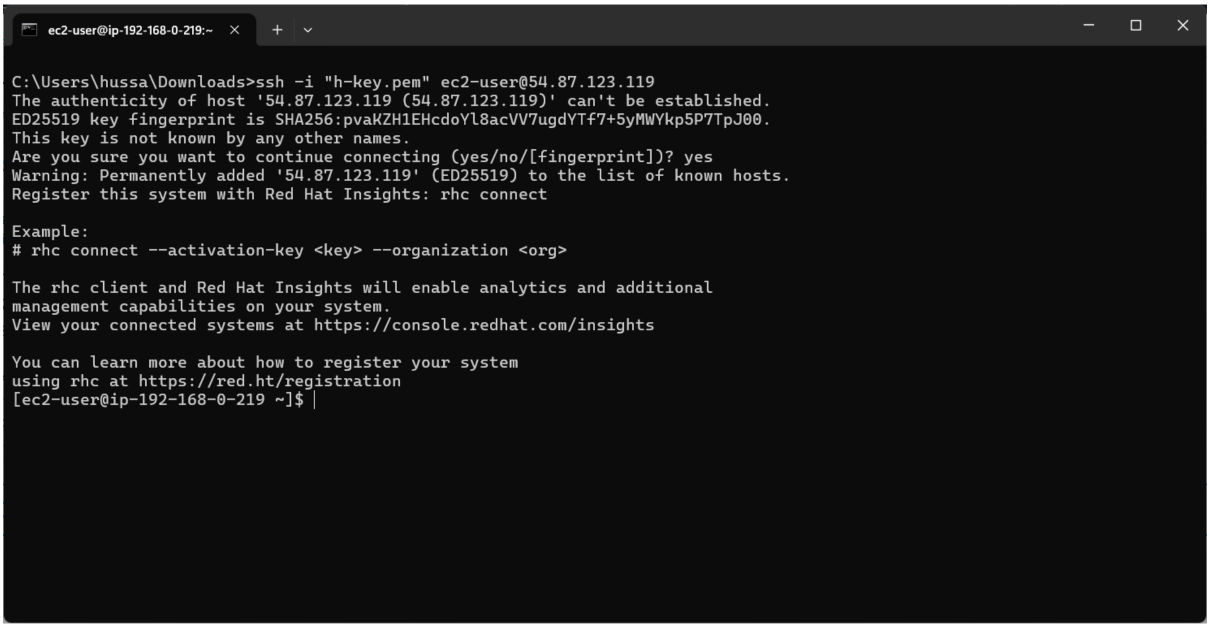
Connecting an EC2 instance from the CMD



Open the CMD and go to where the key you created was downloaded, and enter the command that you copied.

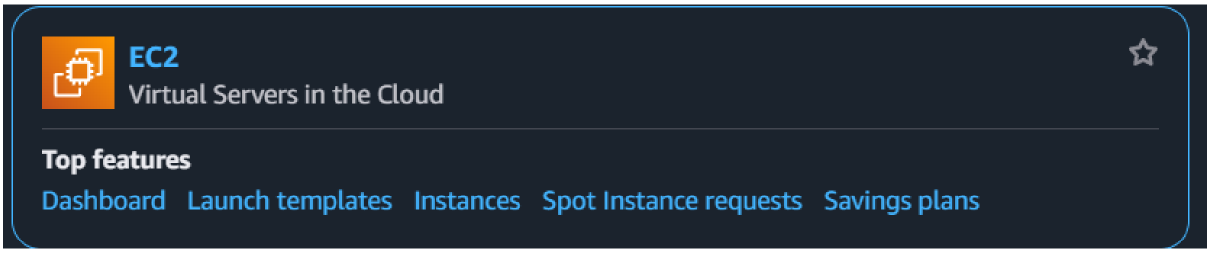


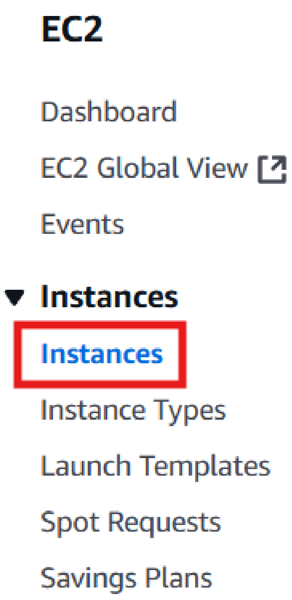
Enter **yes.**

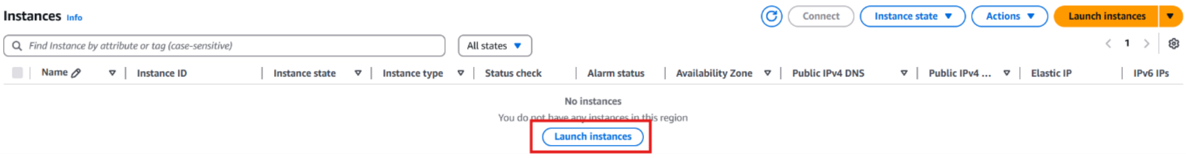


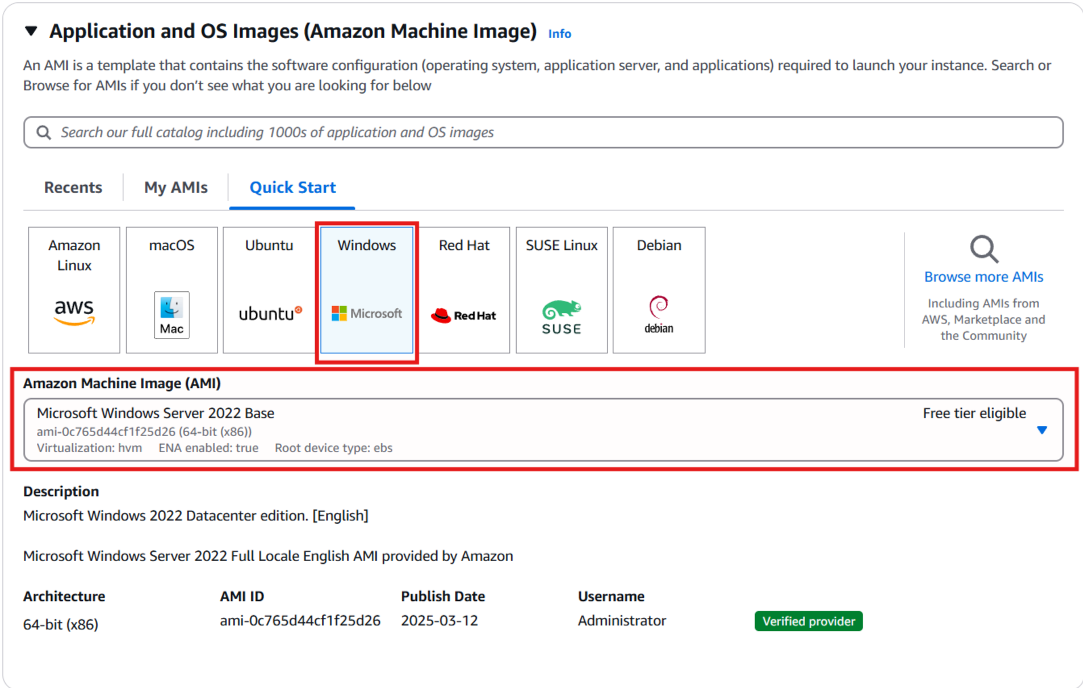
Now you will see that it is connected. Try running the command “**sudo yum update**” to update Red Hat.

Creating Windows EC2



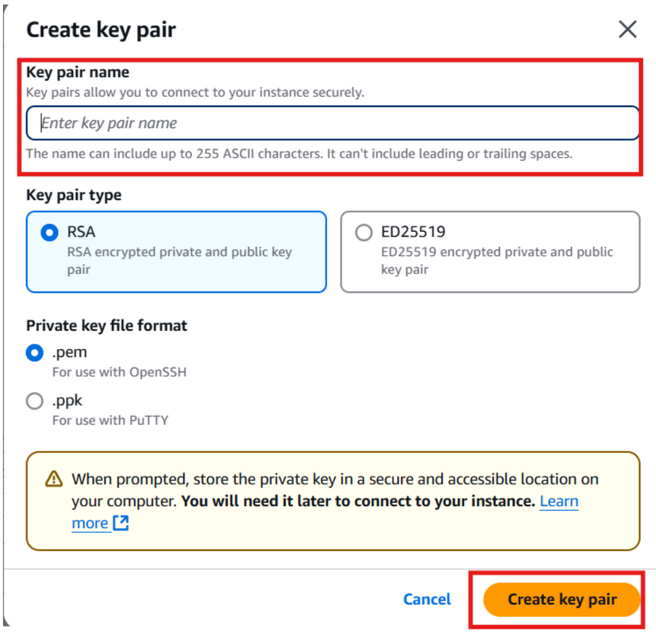




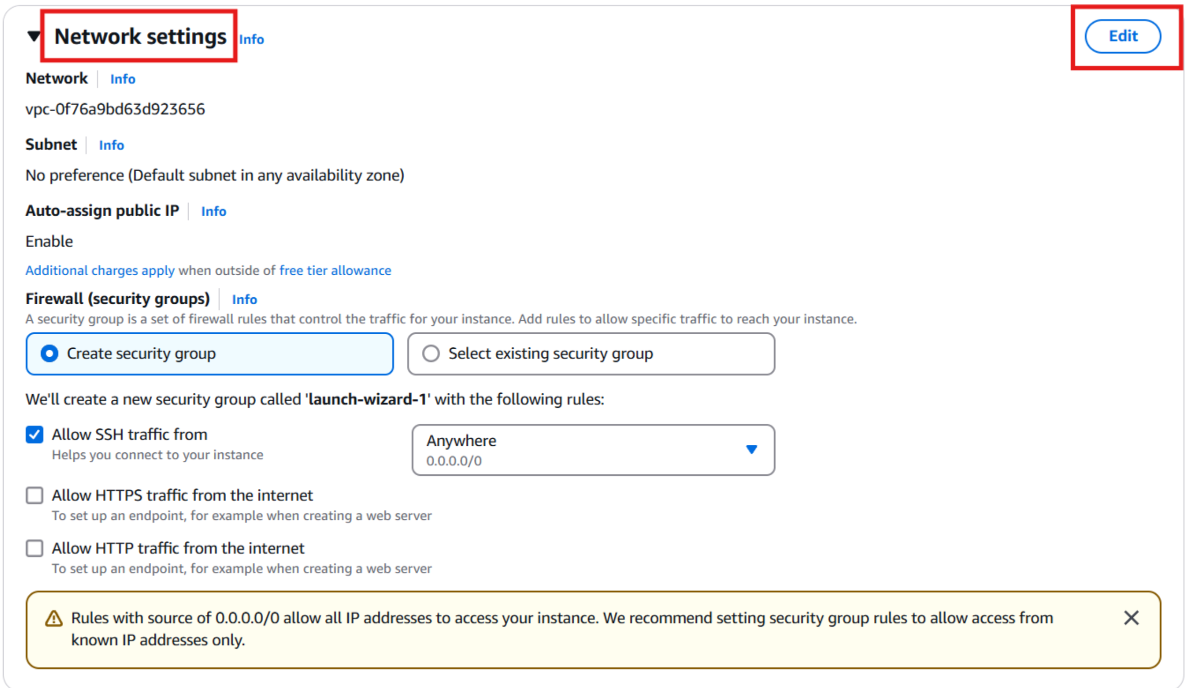


Choose Windows from the Quick start after entering the name for the EC2, and make sure that you choose **Microsoft Windows Server 2022 Base** from **Amazon Machine Image (AMI)**

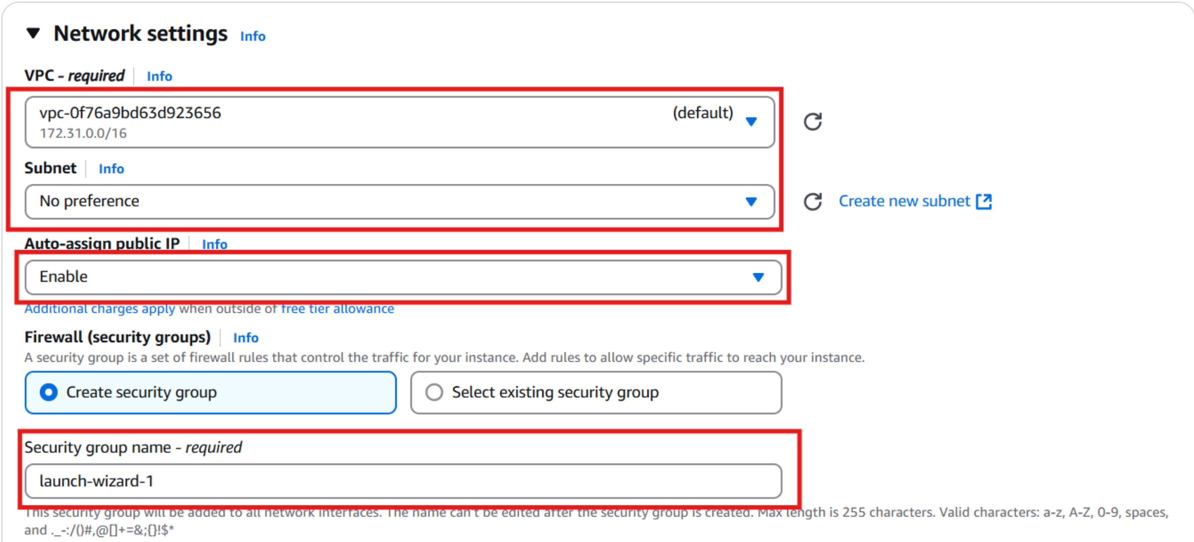
Scroll down to Key pair, choose **Create new Key pair**.



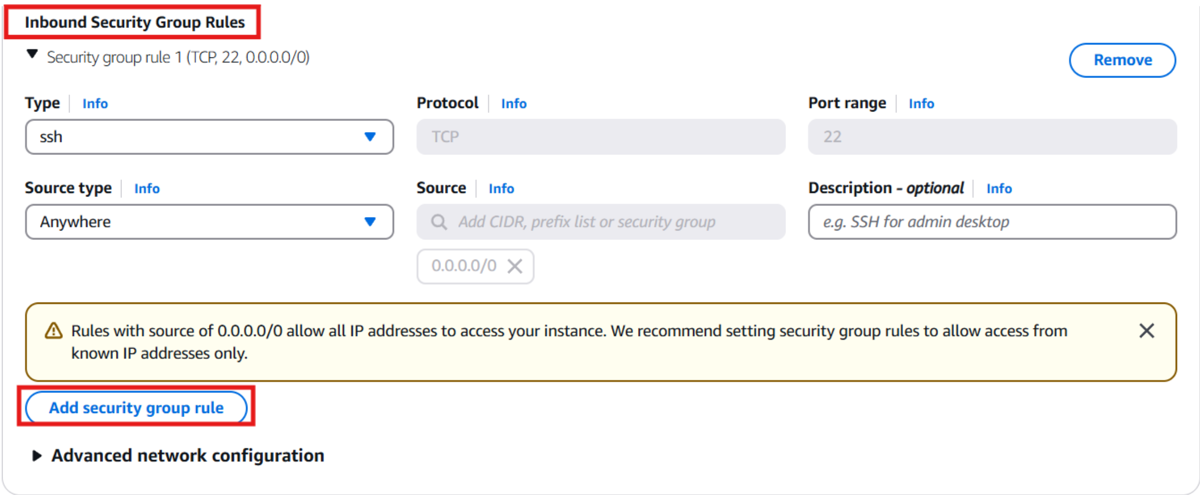
Choose a name for the key pair and click on **Create Key Pair**.



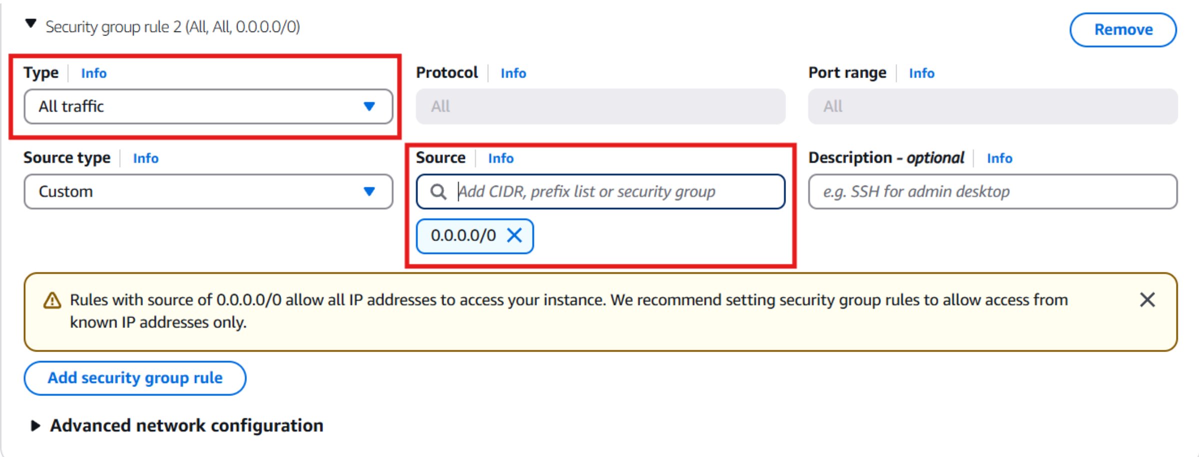
In network settings, click on **edit.**



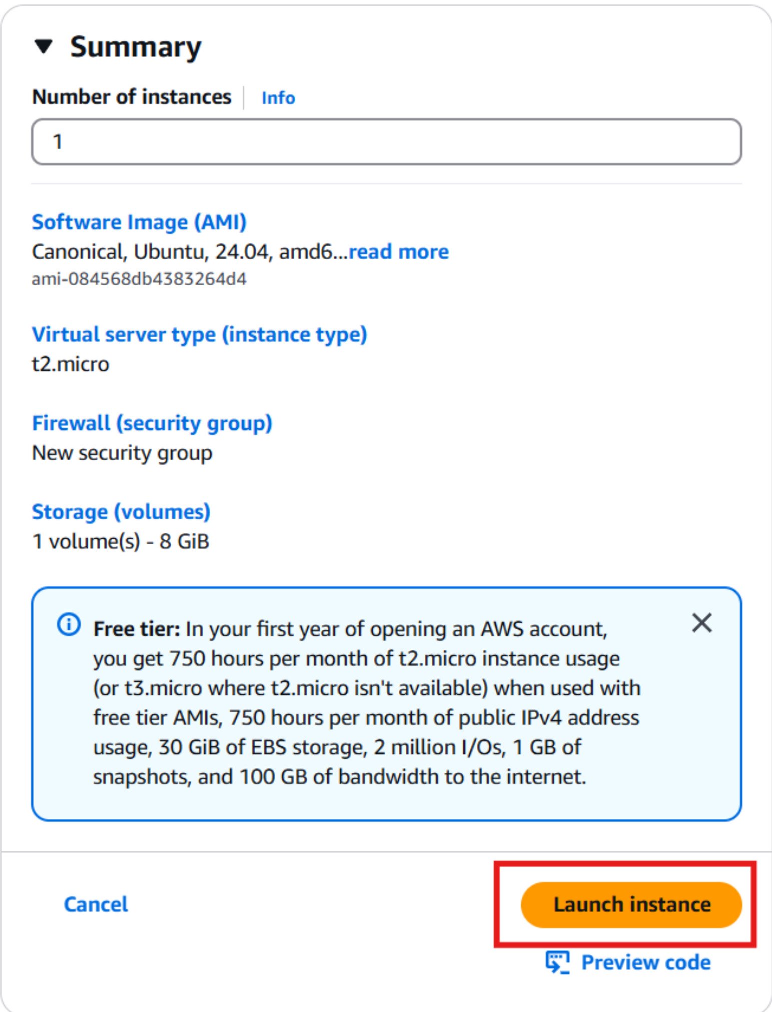
First, choose your VPC. Second, select the public subnet and make sure that the Auto-assign public IP is enabled. Last, **rename the Security group.**



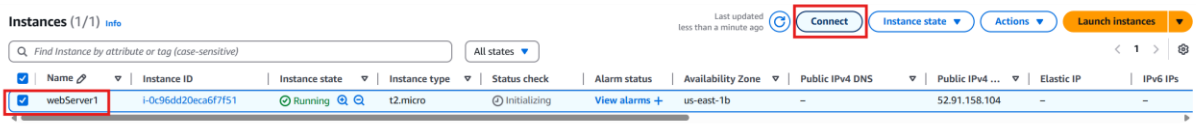
Scroll down in the **Inbound Security Group Rules,** click on Add **security group rule.**



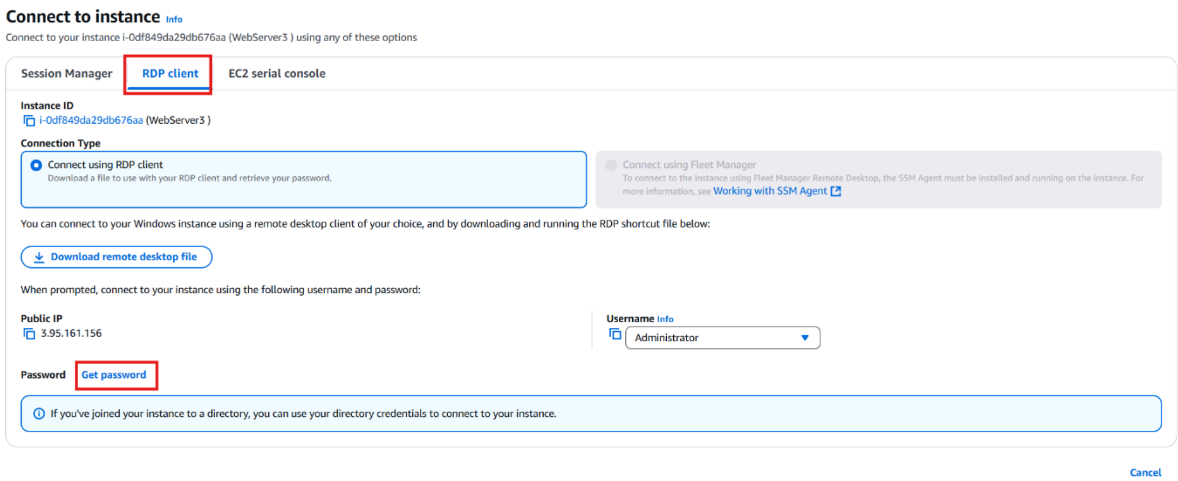
For the type, choose **All traffic,** and for the source, choose **0.0.0.0/0**



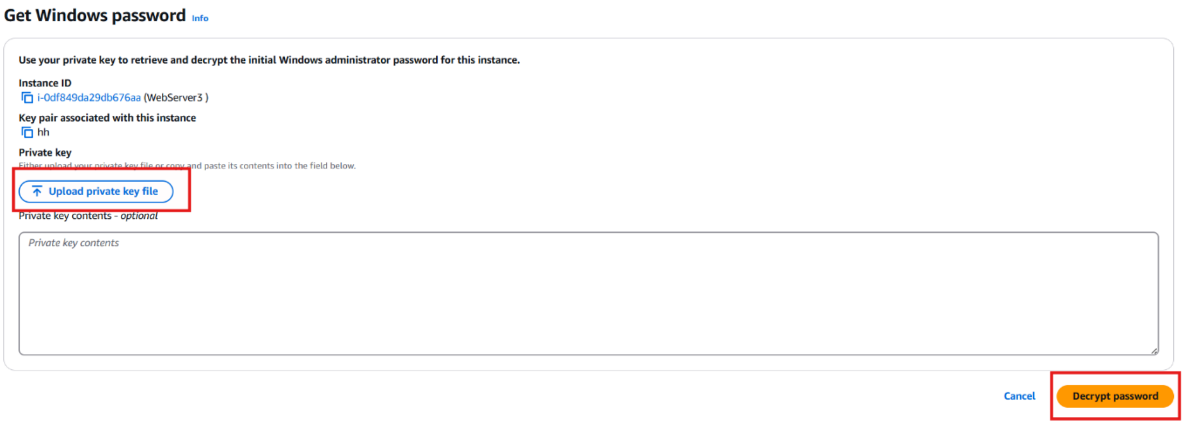
Finally, click on Launch instance.



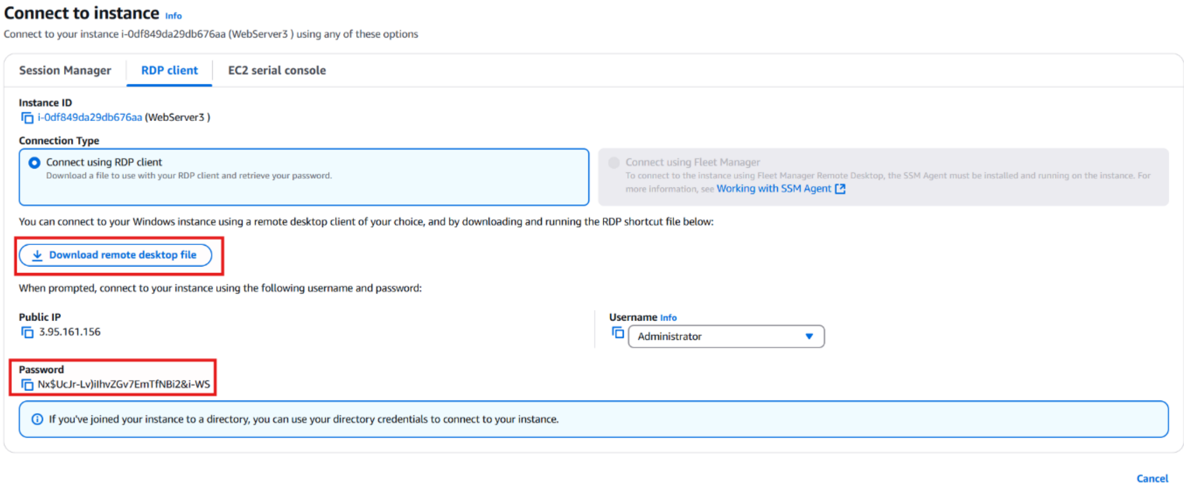
To connect to the web server, **choose your instance** and click on **Connect**.



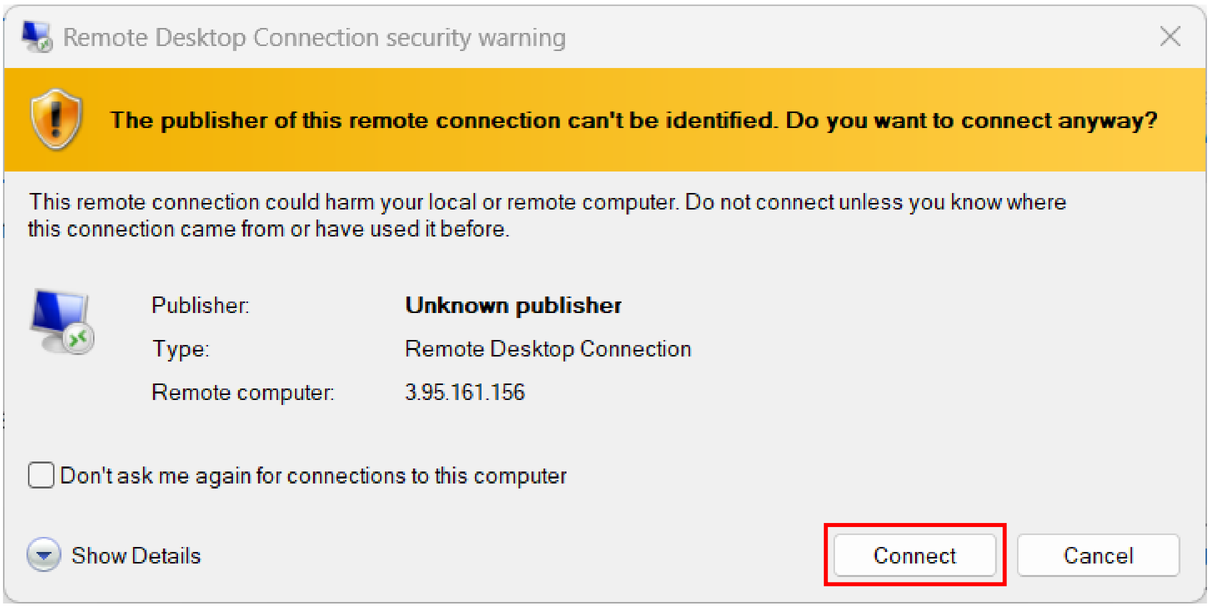
First, go to the **RDP client** and click on **Get password.**



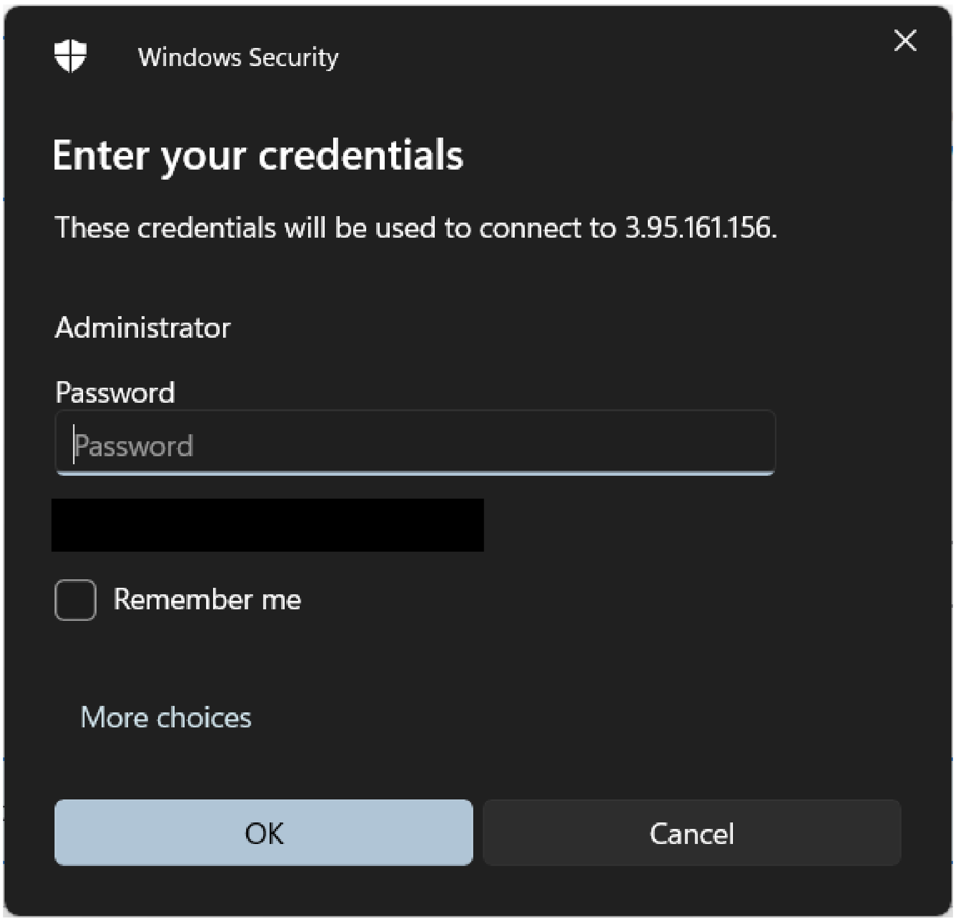
Click on **Upload private key file** and upload the key you have created for the Windows EC2, then click on **Decrypt password**.



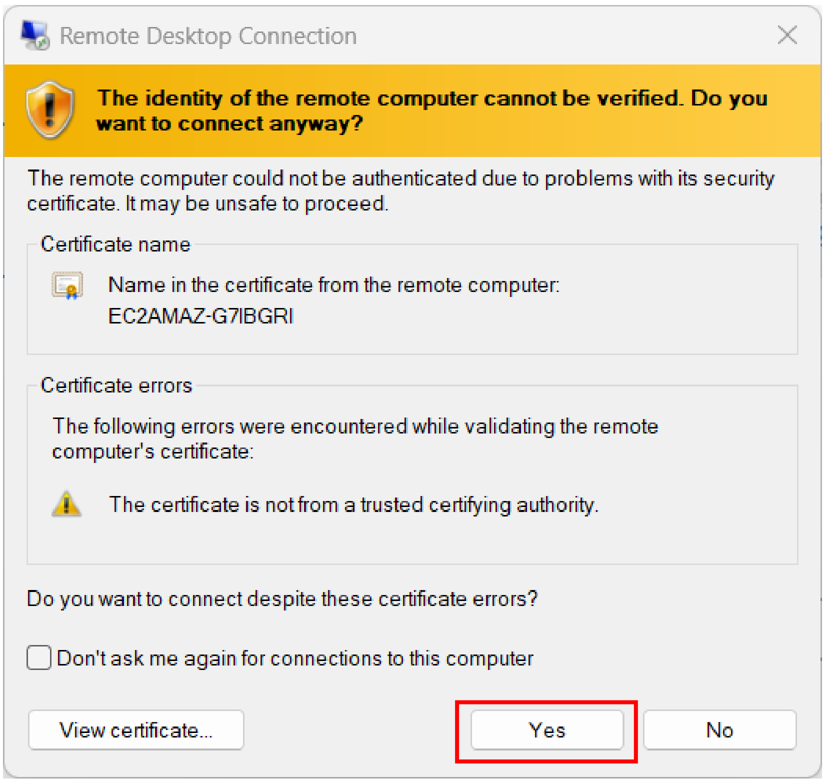
After decrypting the key, it will give you the password. Copy it. Then download the **Remote Desktop file and open it.**



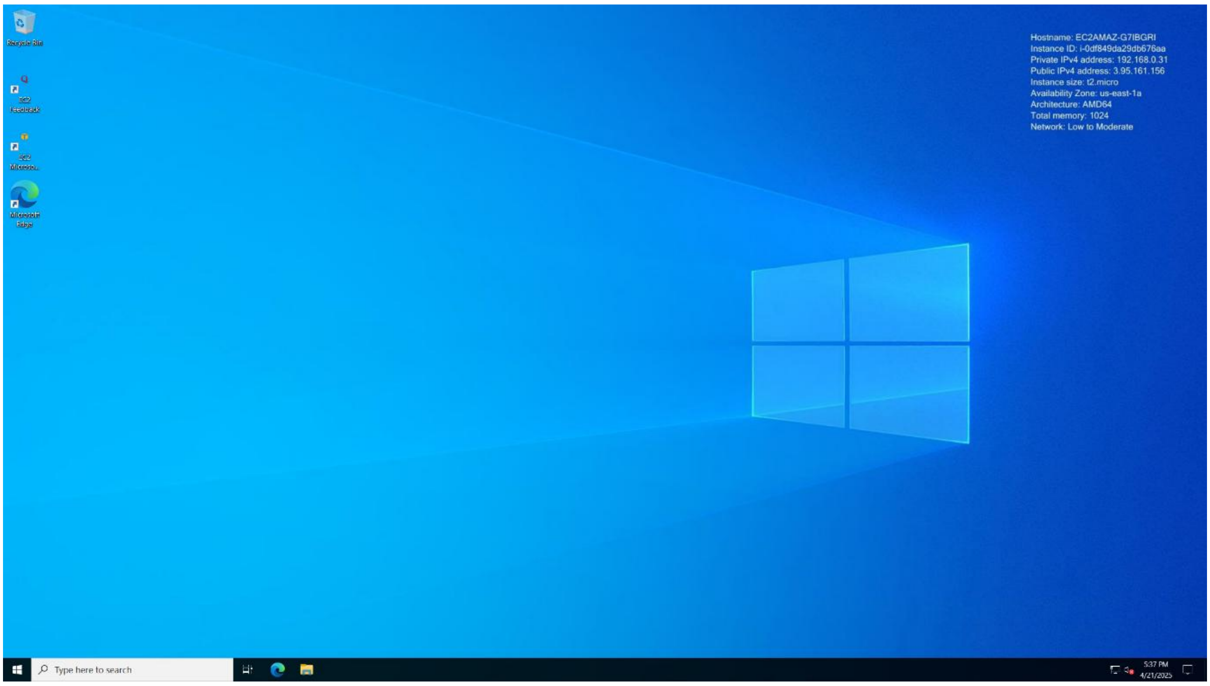
Click **connect.**



Paste the password that you copied before and click ok.



Choose **Yes,** and the window will start.

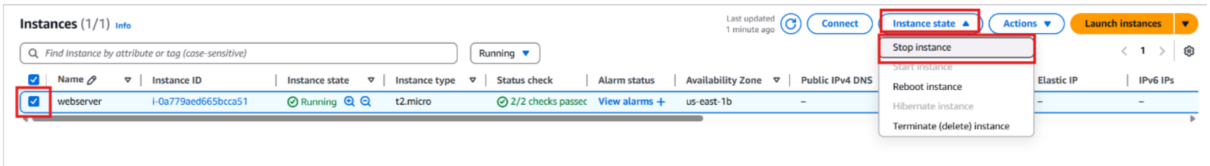


If you've done everything correctly, it will open for you like this.

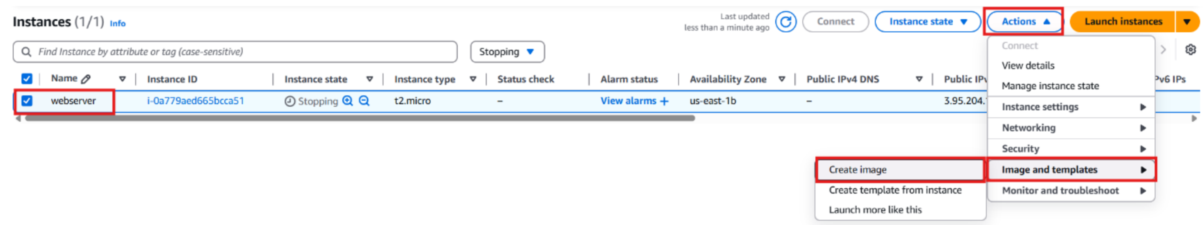
How to Create Load Balancers

Requirements: [VPC]() with **2 subnets** (public and private **in different Availability Zones**) and [an EC2 instance that runs Ubuntu]() **with Apache2 service on it, and** [an internet gateway]()**.**

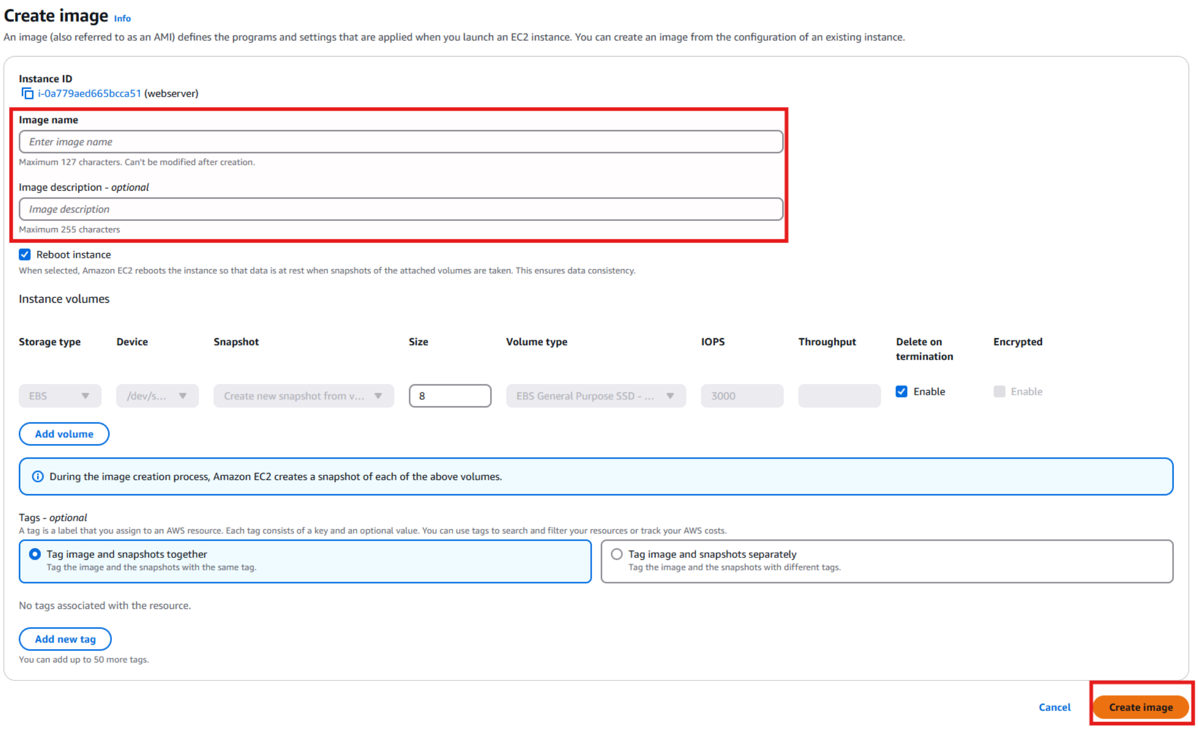
Create an image (AMI) from EC2 instance



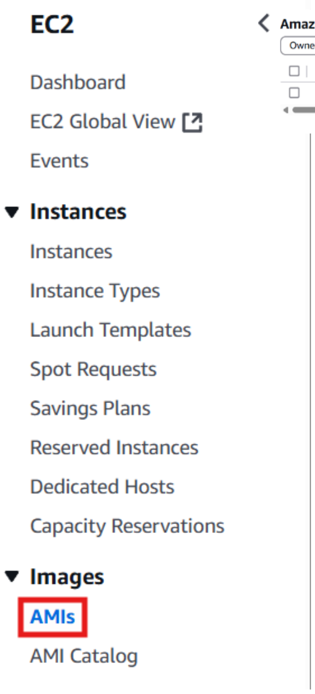
First, go to EC2. Second, select the instance and stop the web server that you created before from the instance state. It will show you in the Instance state **Stopped.**

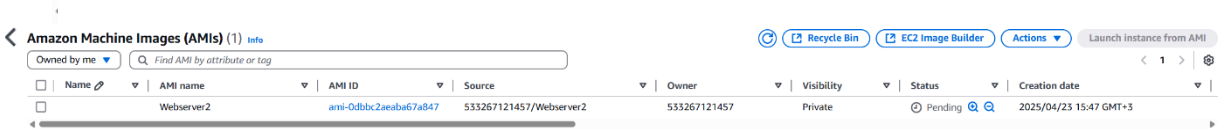


Now you will **create an image** from the web server that you stopped. First, select your instance and click on **action** from the drop-down menu, select **image and templates,** then click on **create image,** which will create a copy of your original instance.



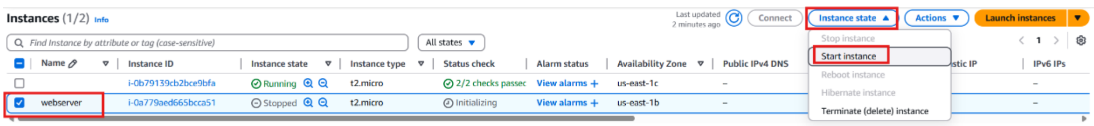
Give your image a name and a meaningful description, and click **Create Image**.



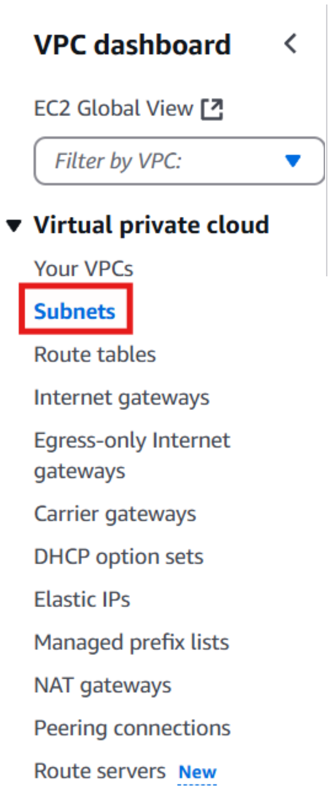


From the left-side menu, choose **AMIs,** and you will find the image you created there.

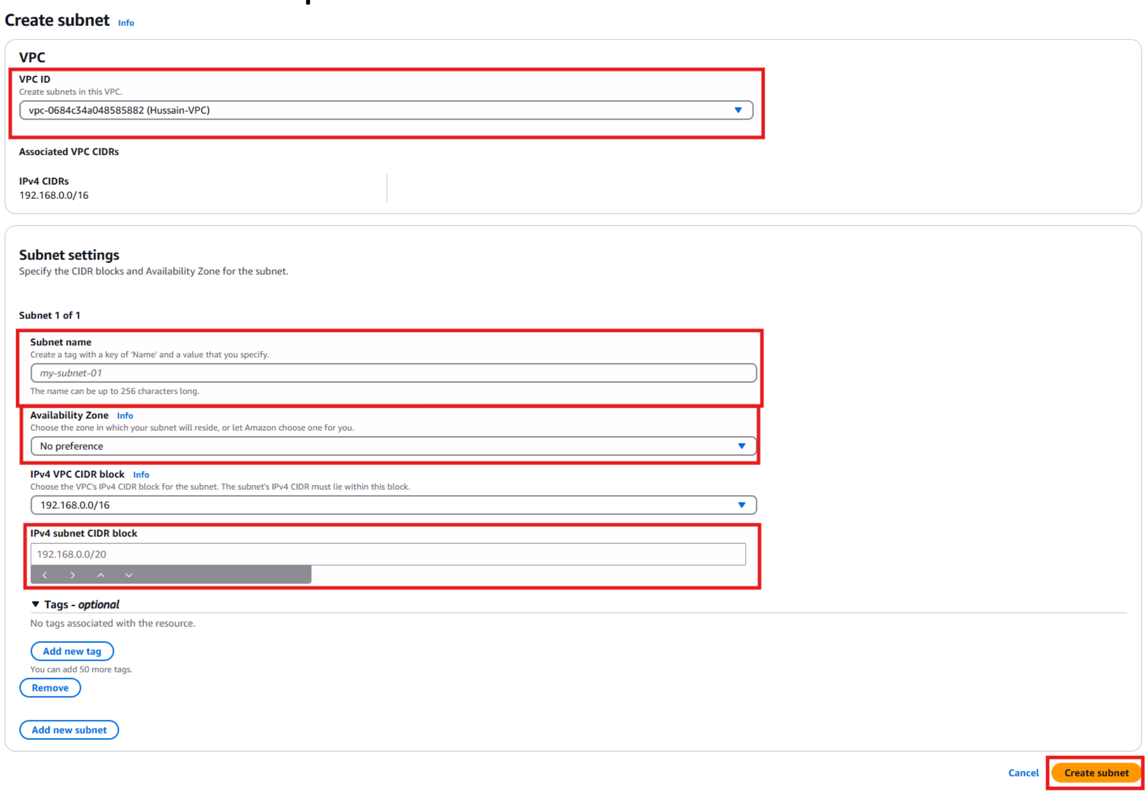
Note: if you have created the image, start the EC2 instance again, and stop it by taking the same steps for stopping the instance.



Create a new public subnet



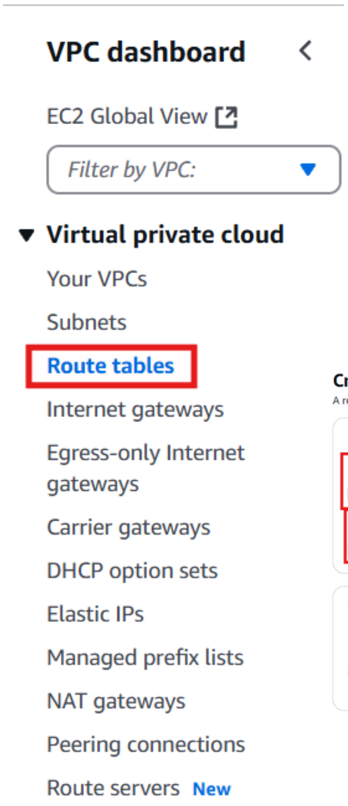
You must create a new public subnet.



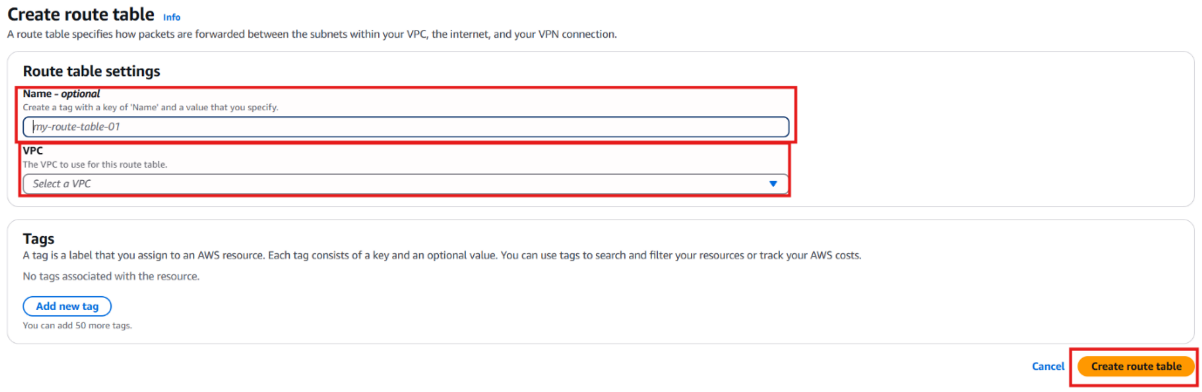
now go to VPC under subnets. Click on **Create subnet**.

First, choose the VPC that you created before. Second, enter a name for your new public subnet. Third, choose a different available Zone from the other 2 subnets (public, private) that you created before. Finally, choose an IPV4 CIDR and click **Create Subnet.**

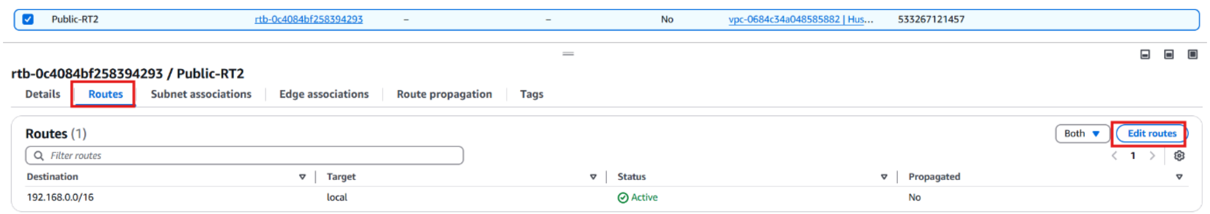
Create a Route table for the new subnet



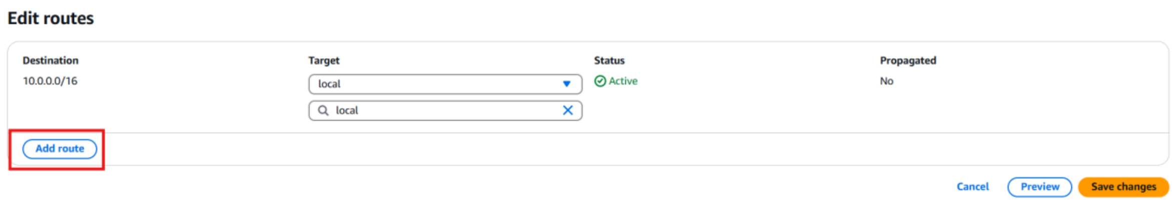
From the left-side menu, select **Route tables** and click on **Create route** table.



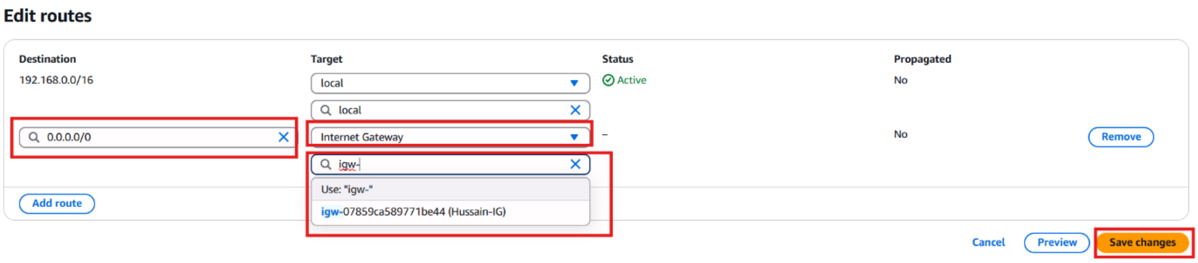
Give it a name, choose your VPC, and click **create route table.**



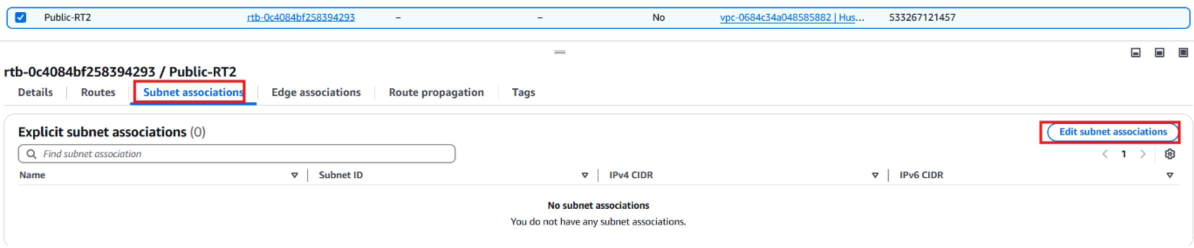
Select your route table. From the drop-down menu, select Route, and click **Edit routes**.



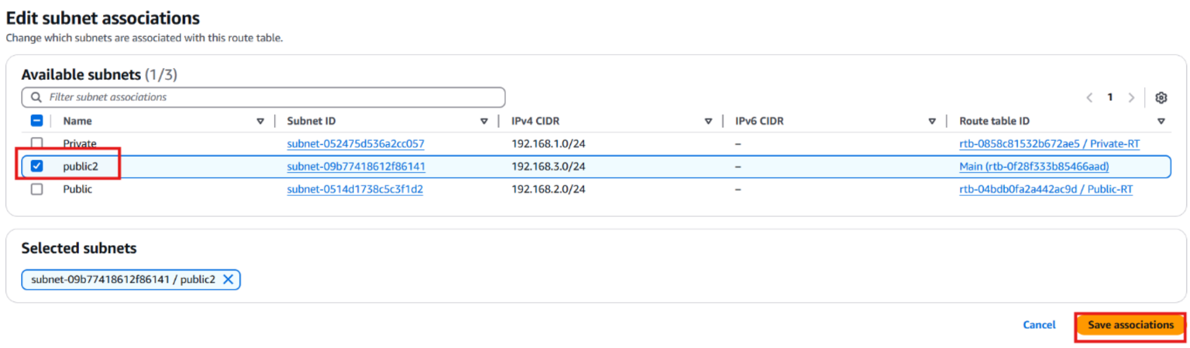
Click on **Add route**.



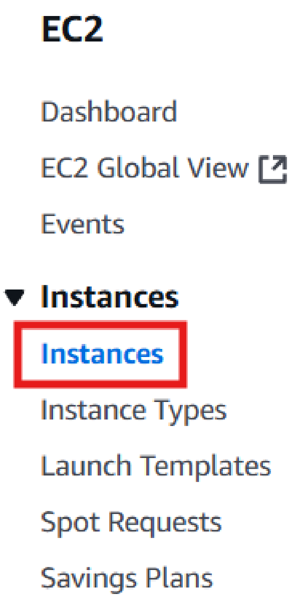
First, **choose 0.0.0.0/0,** then for the **target, choose internet gateway,** and then **select** the **internet gateway** that you created before. Finally, **click save changes**.

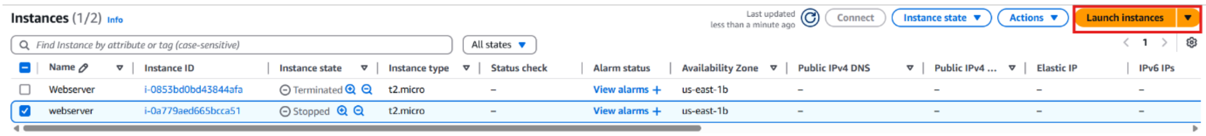


After you create the route table, you must add the subnets. First, choose the public route table and from the drop-down menu select Subnet associations, then choose **Edit subnet associations.**

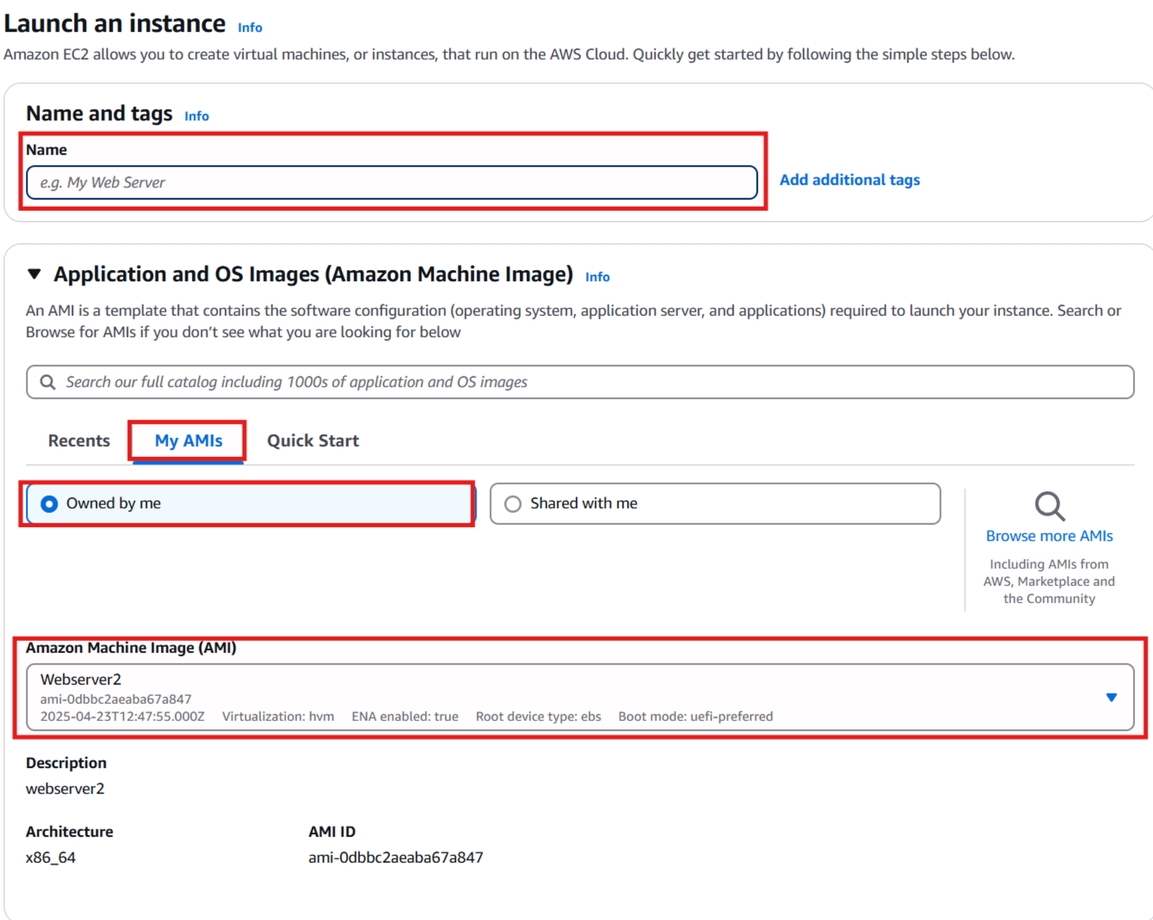


Choose the **public2** subnet, then click **save associations**.

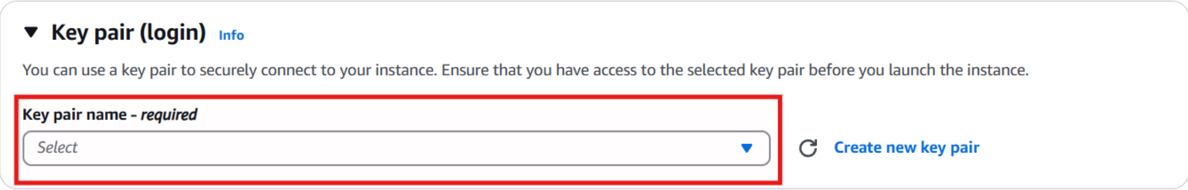




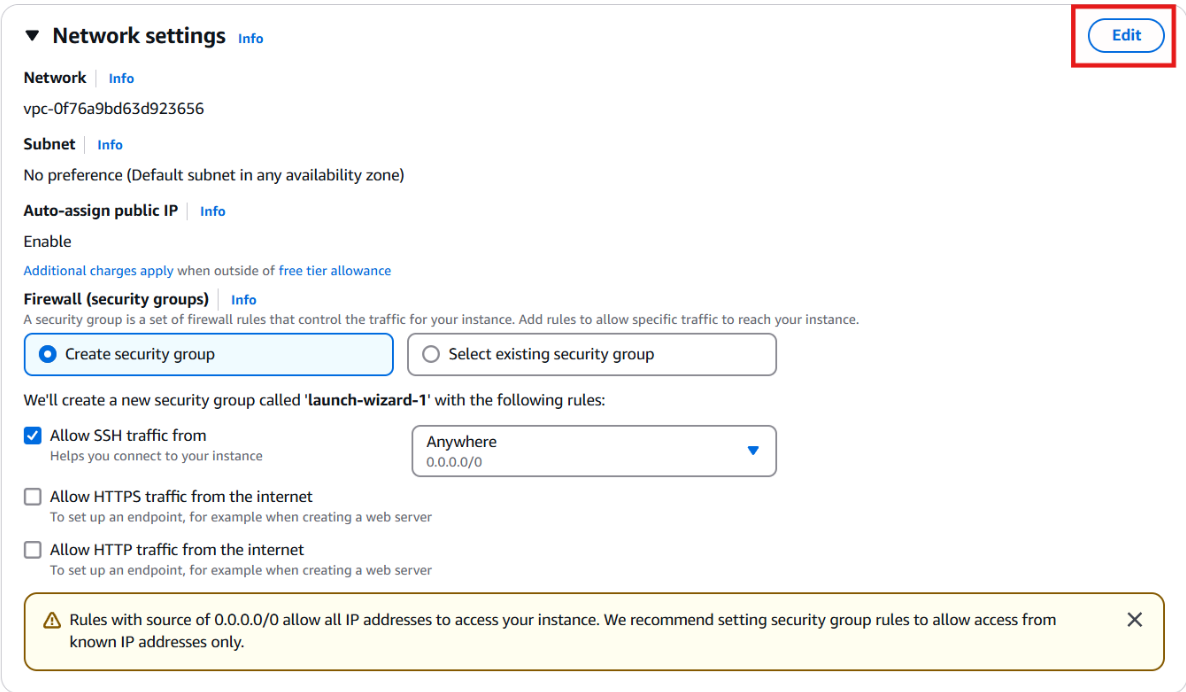
Go to EC2 from the side menu, choose **Launch instance**.



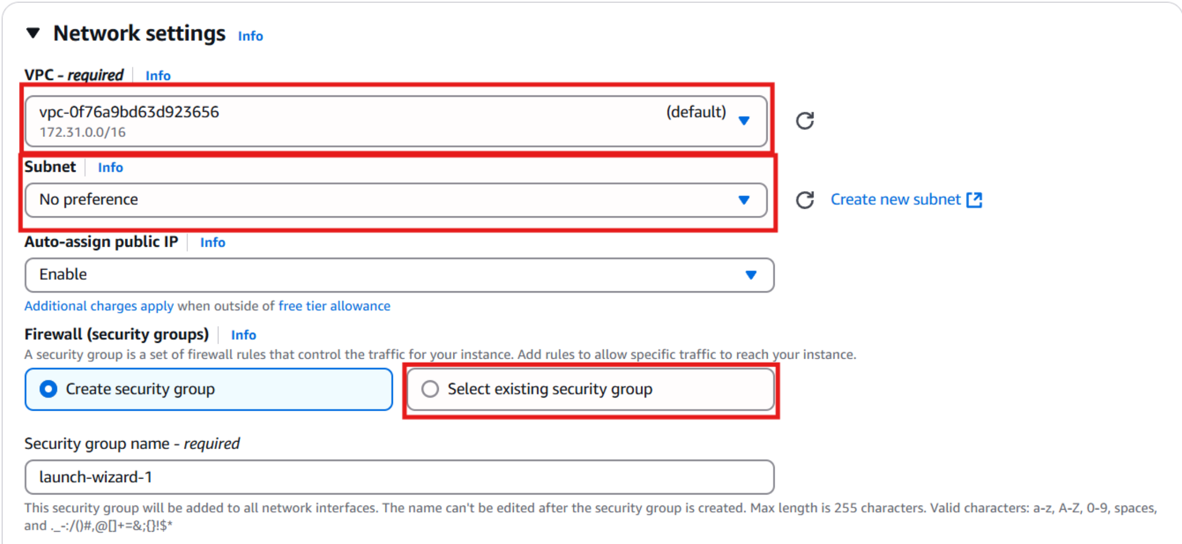
First, give your EC2 a name. Second, in **Application and OS Images (Amazon Machine Image),** select **My AMIs.** Third, make sure **you select Owned by me.** Lastly, **choose the image you created before**.



In the Key pair, select the **earlier key you created**.

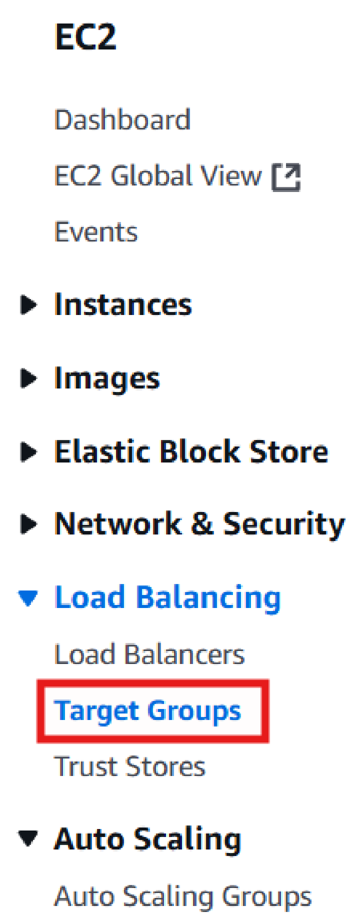


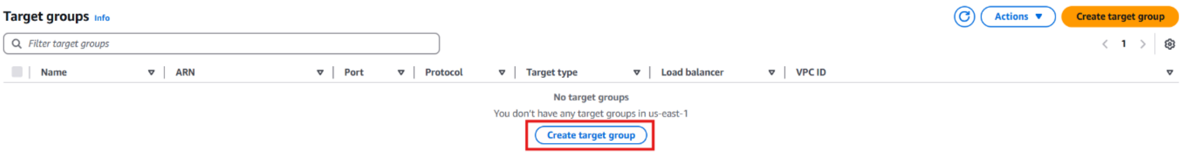
In the network settings, click on **Edit**.



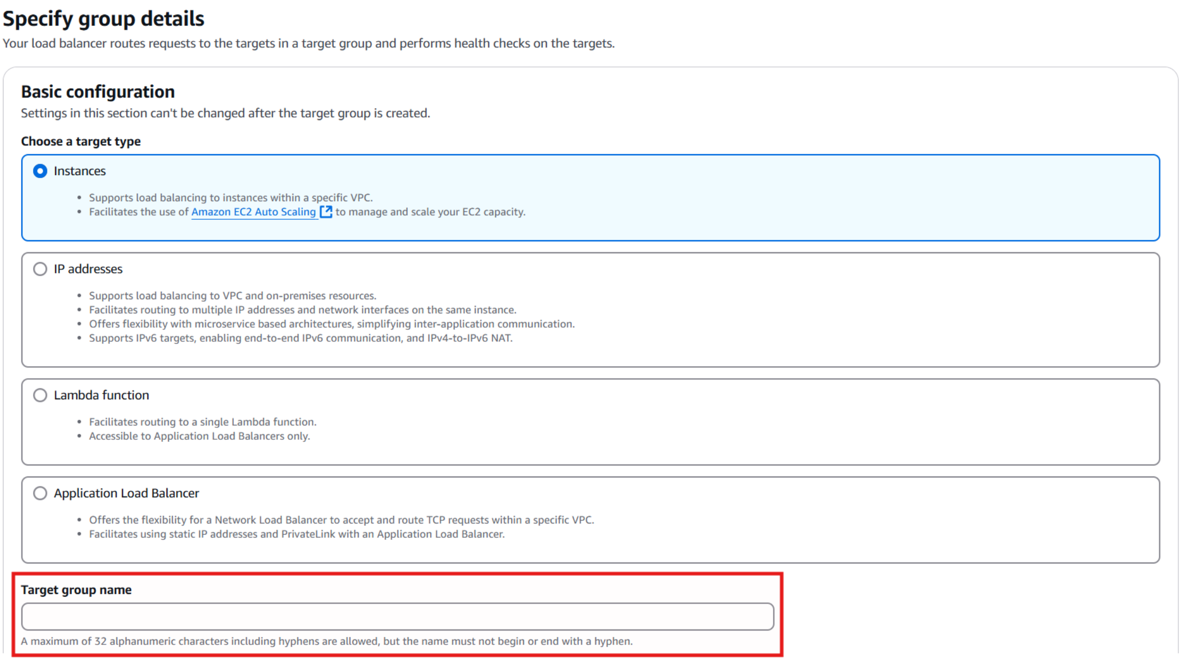
First, choose your VPC. Second, choose your second private subnet you created before and **make sure** that the **Auto-assign public IP is Enable. Lastly,** click on Select **existing security group** and select your security group, then click on **Launch instance.**

Create Target Group

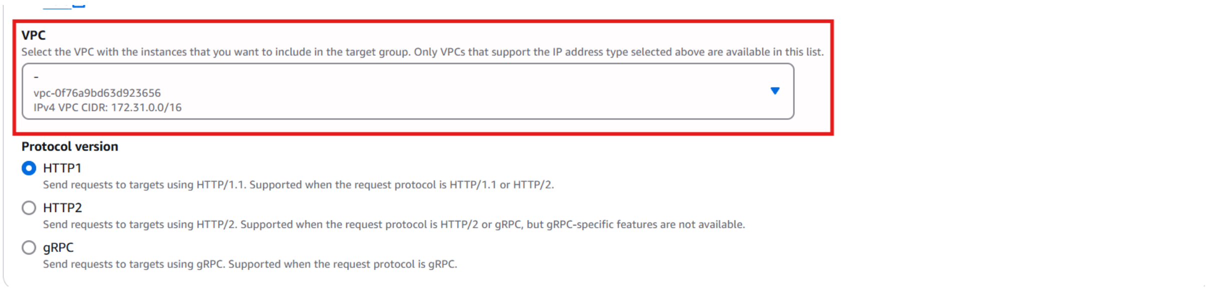




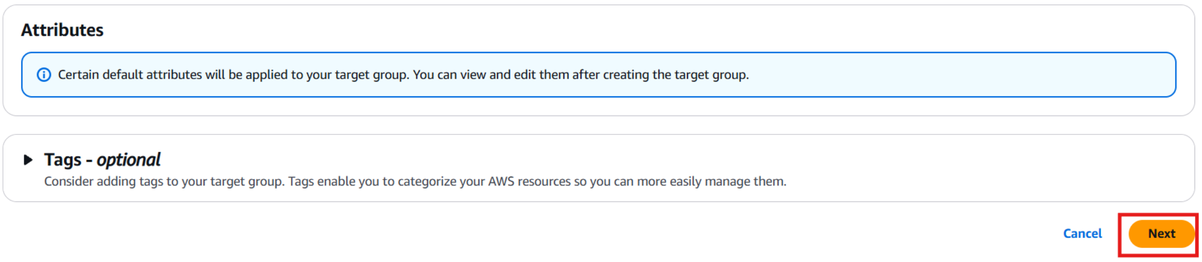
Go to EC2 from the side menu, choose Target group. Click on **Create Target group.**



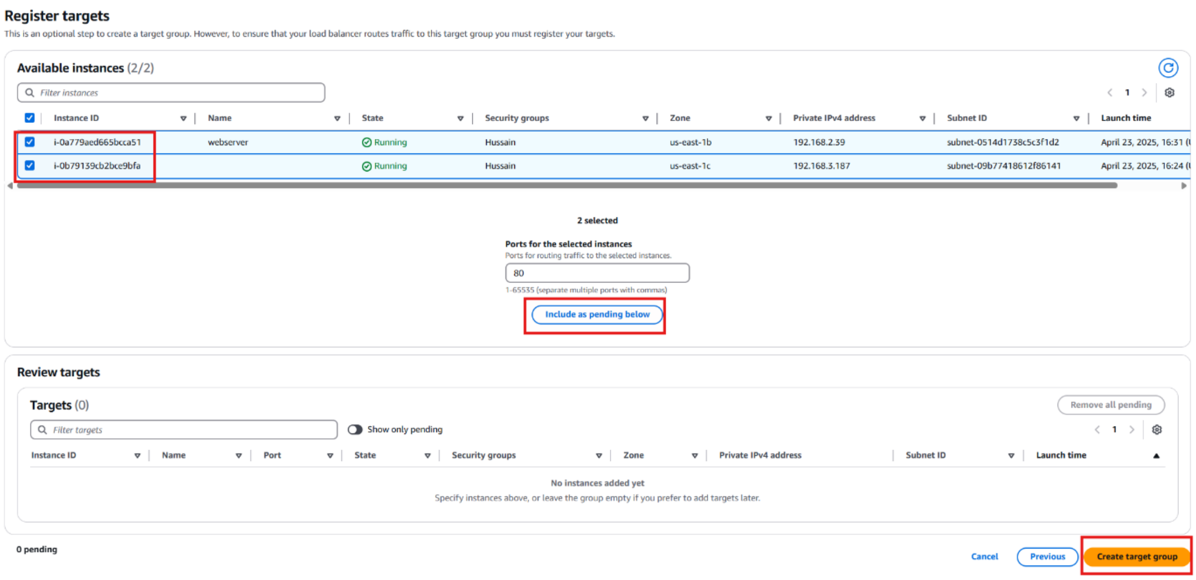
Scroll down and give the target group a name.



Choose your **VPC** that you created before.

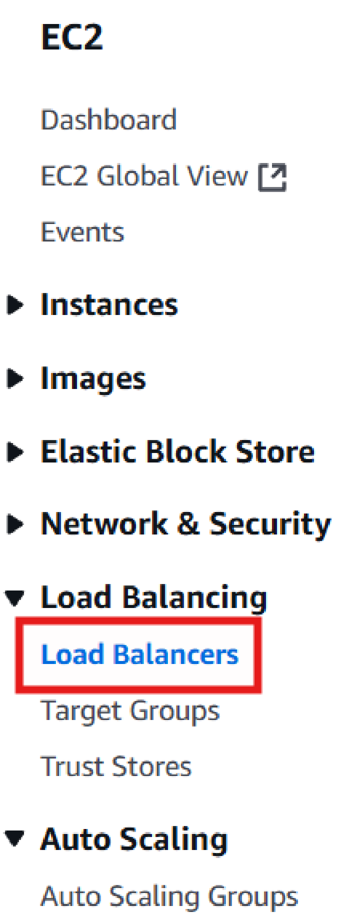


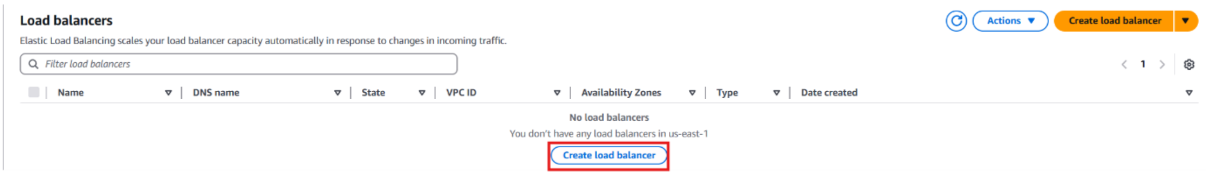
Leave everything as the default option and click Next.



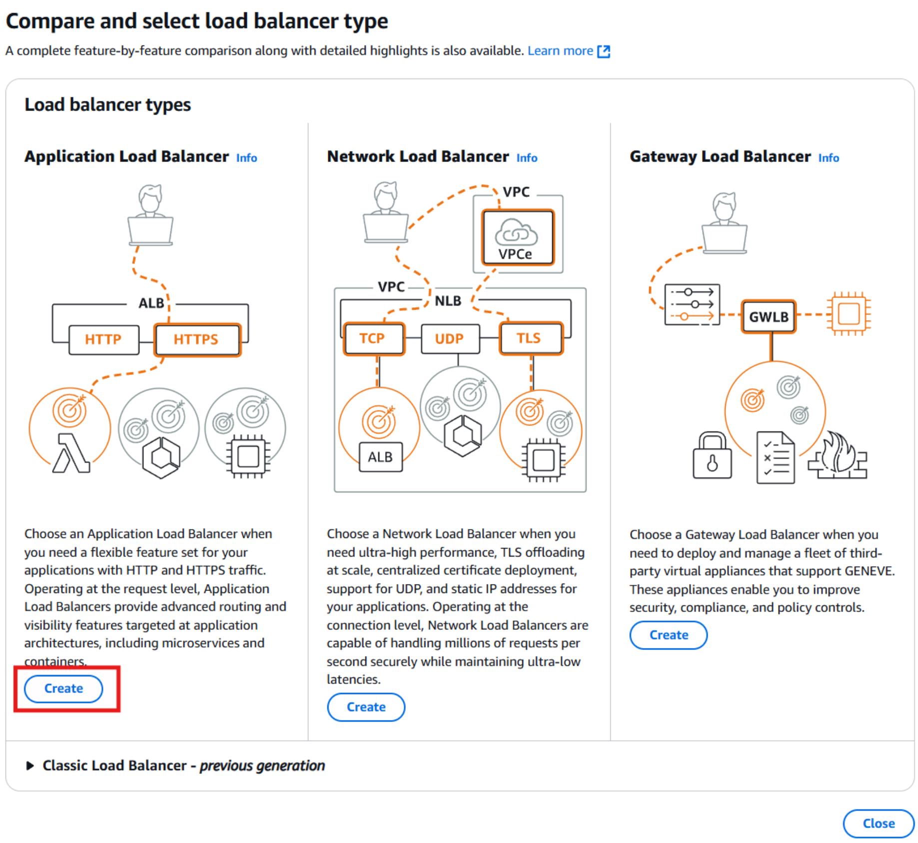
Select your two EC2 instances and click on **Include as pending below.** Last, click on **Create Target group.**

Create Load Balancer

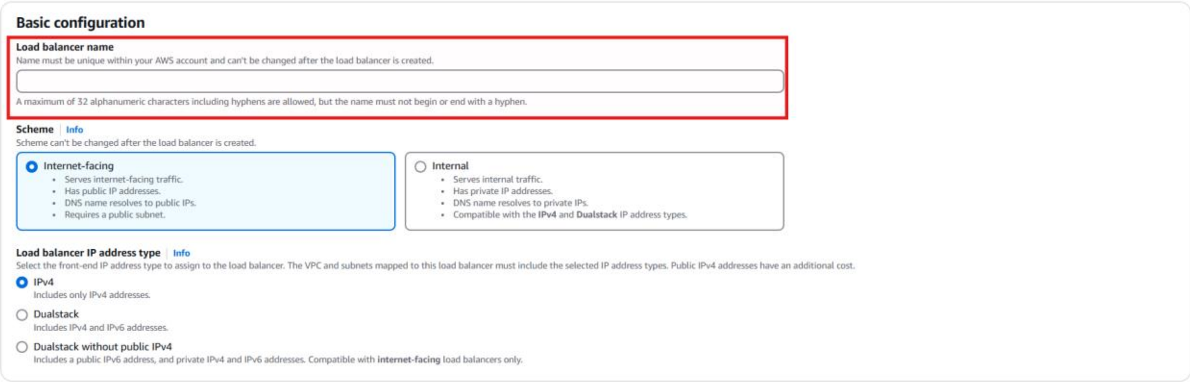




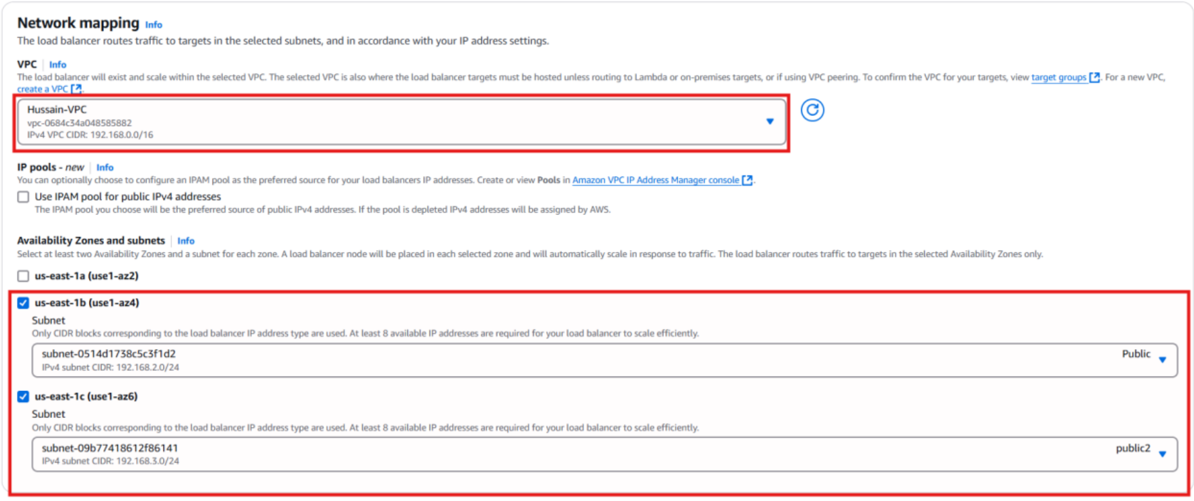
Go to EC2 from the side menu and choose **Load Balancer**. Click on **Create load balancer.**



Select **Application Load Balancer and click on Create.**

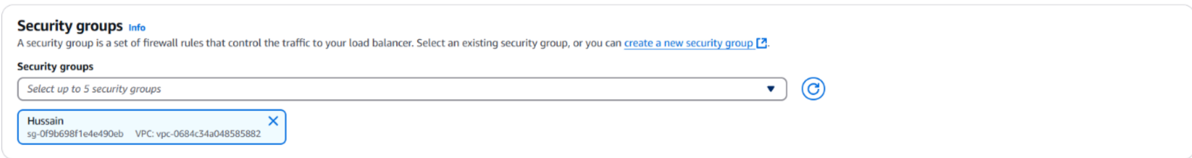


Enter a name for the **Load Balancer.**

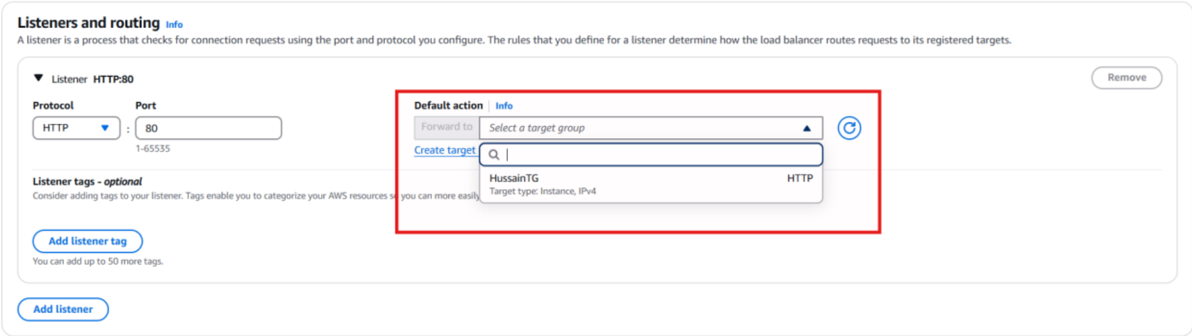


First, choose **your VPC** that you created before. Second, under **availability zones and subnets,** select the zones that are associated with public subnets.

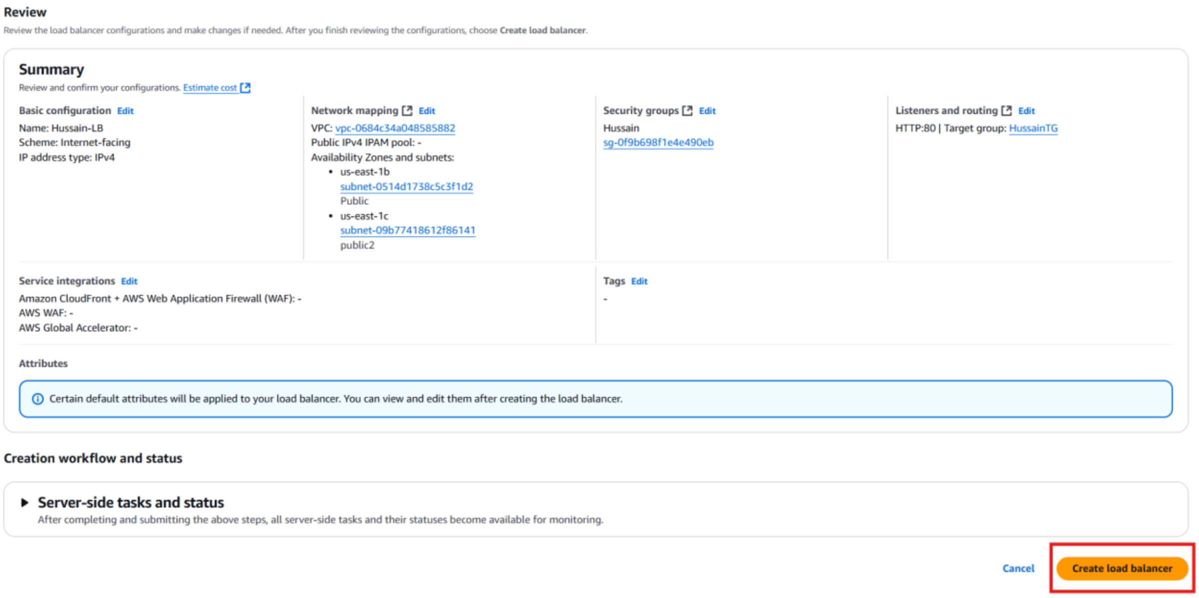
Note: For the load balancer to work, it needs two subnets in different availability zones.



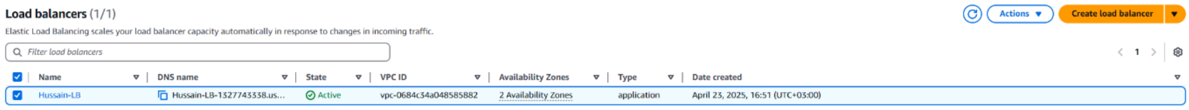
choose your **security group** and **remove the default option**.

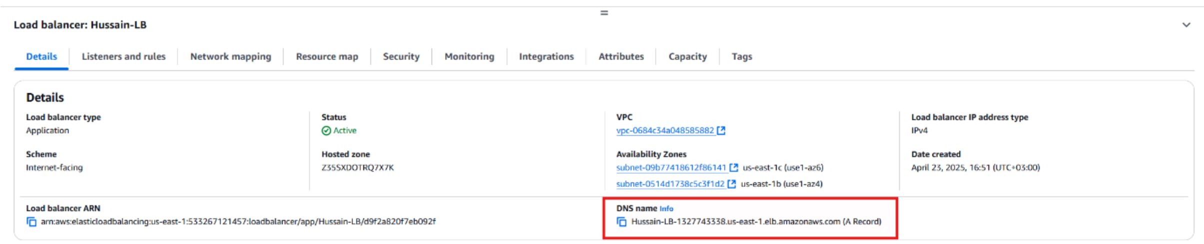


In **Listeners and routing** under the **Default action,** select your **target group** that you created before.



Leave the rest at the default option and scroll down and choose **create Load Balancer.**





After the load balancer has been set up, **copy the DNS name** and paste it into your browser, and you will notice that it is shifting between the first and the second server.

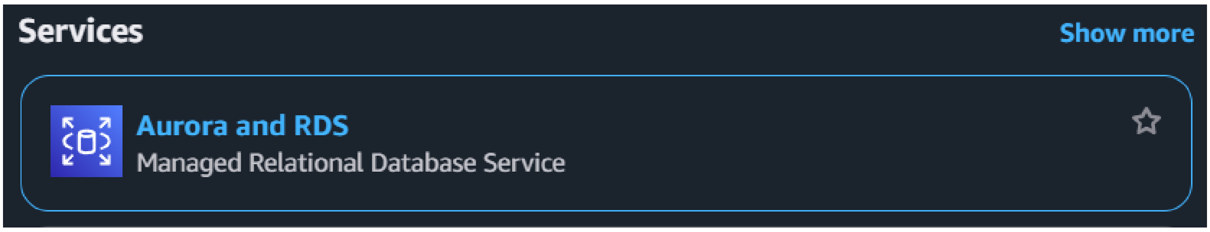
Note: for you to notice the shifting, you must make changes to one of the EC2 instances.

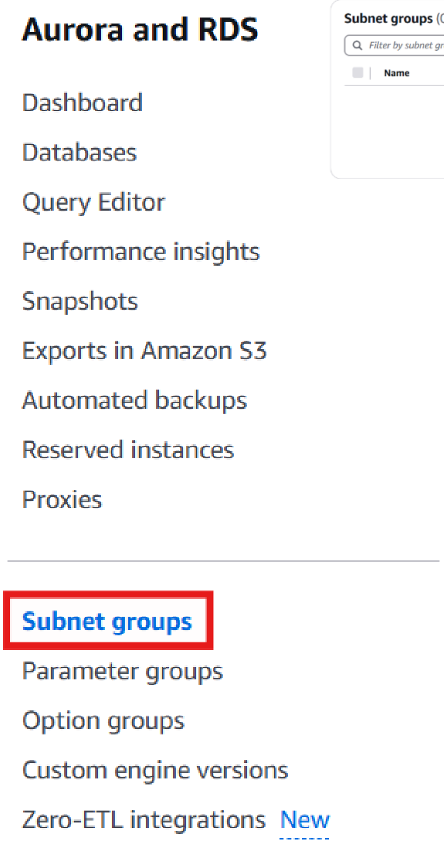
How to Create an RDS Database

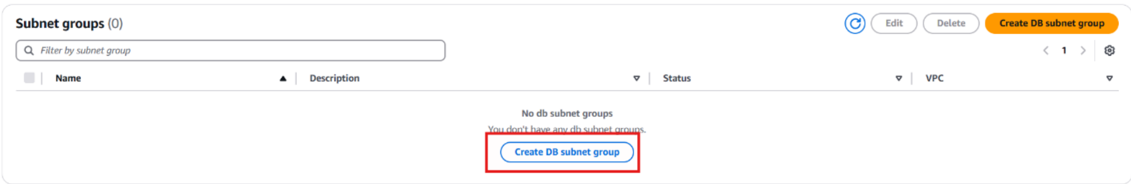
Note: You must create a VPC and EC2 with Ubuntu on it, then you can follow the steps.

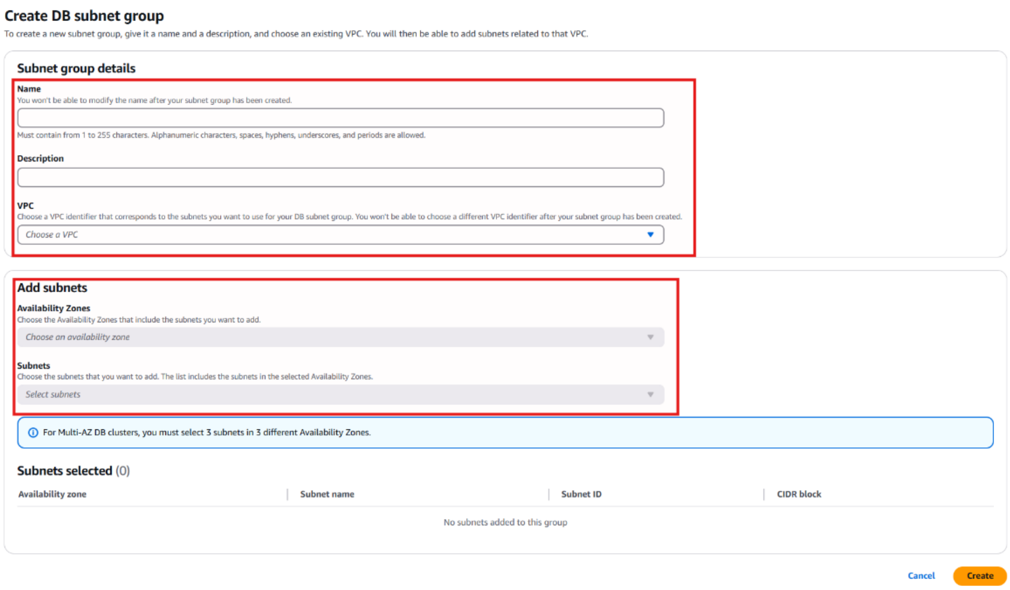
Requirements: that you must have at least **2 Subnets (Public, Private)** and each one must be in a different **Availability Zone**

Create Database Subnet Group

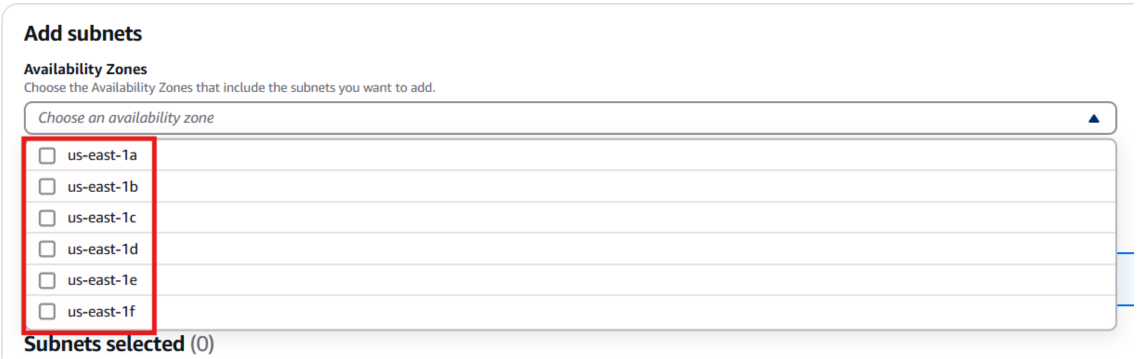




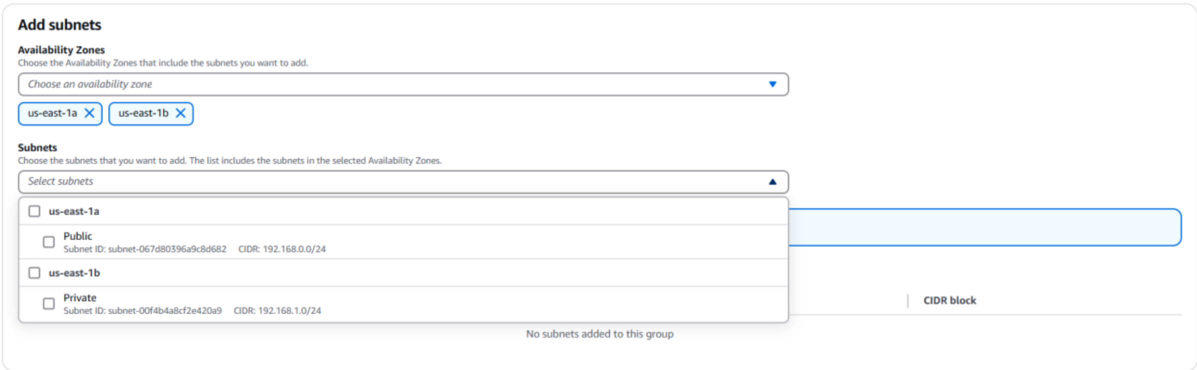




First, select the **Subnet group**. Second, click on **Create DB subnet group**. Third, **choose a name** for the subnet group and **write a meaningful Description**. Last, **choose your VPC** that you created before (and make sure it has at least 2 VPCs and each one in a different Availability Zone).

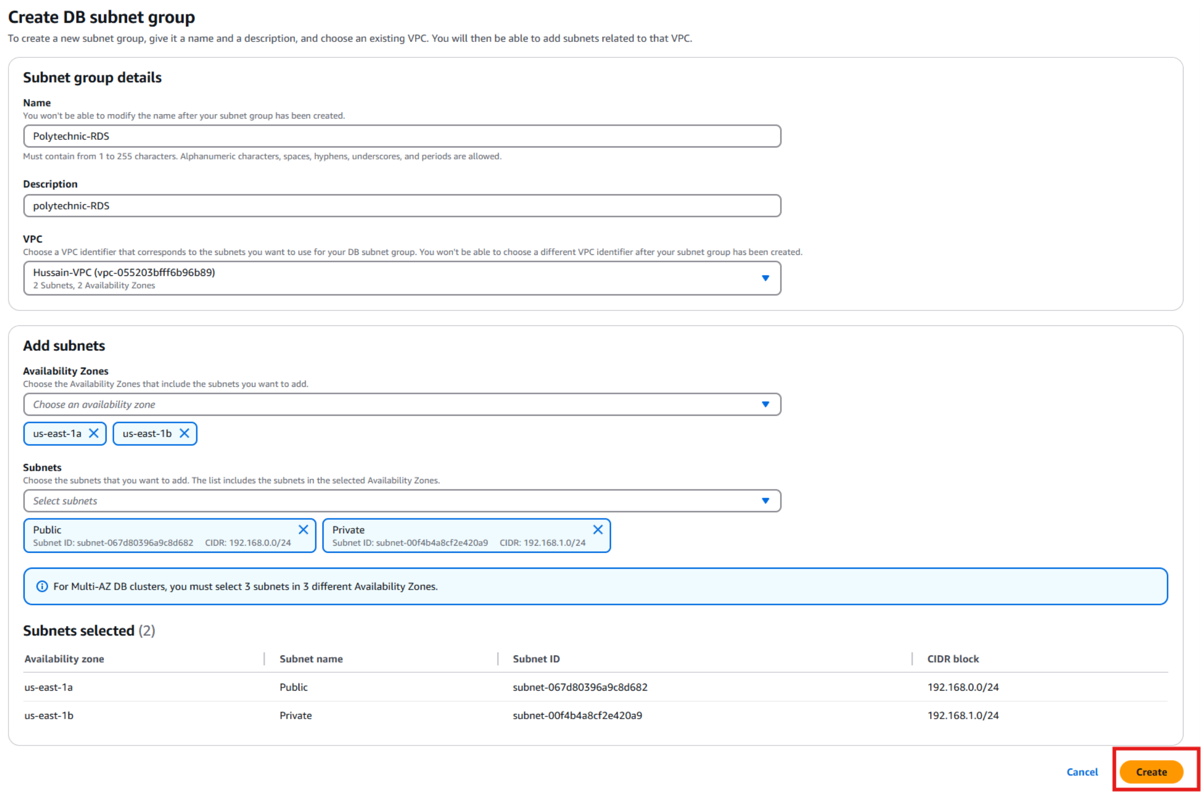


Select your **Availability Zones in which your subnets are in.**



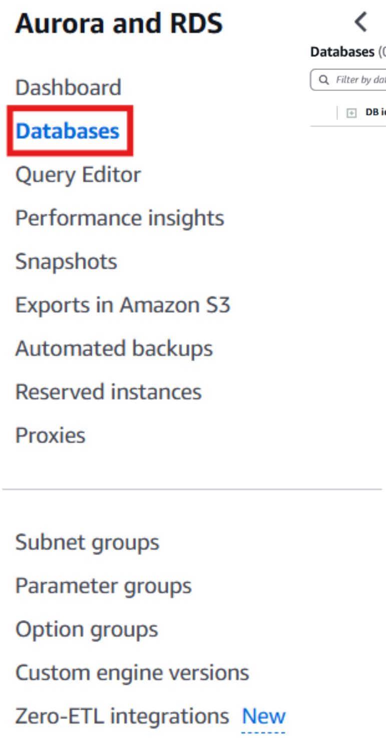
Afterwards, select your Availability Zones that your subnets are in. You will see all the subnets you have in every Availability Zone you choose. Choose the subnets you want them to be in the subnet group.

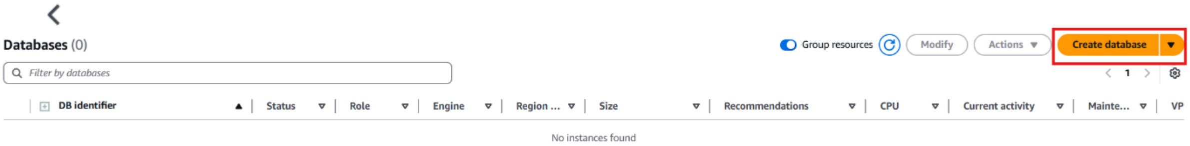
Note: No need for your subnets to be one public and one private; there can be two privates because you use it for a database, and it does not require public access.



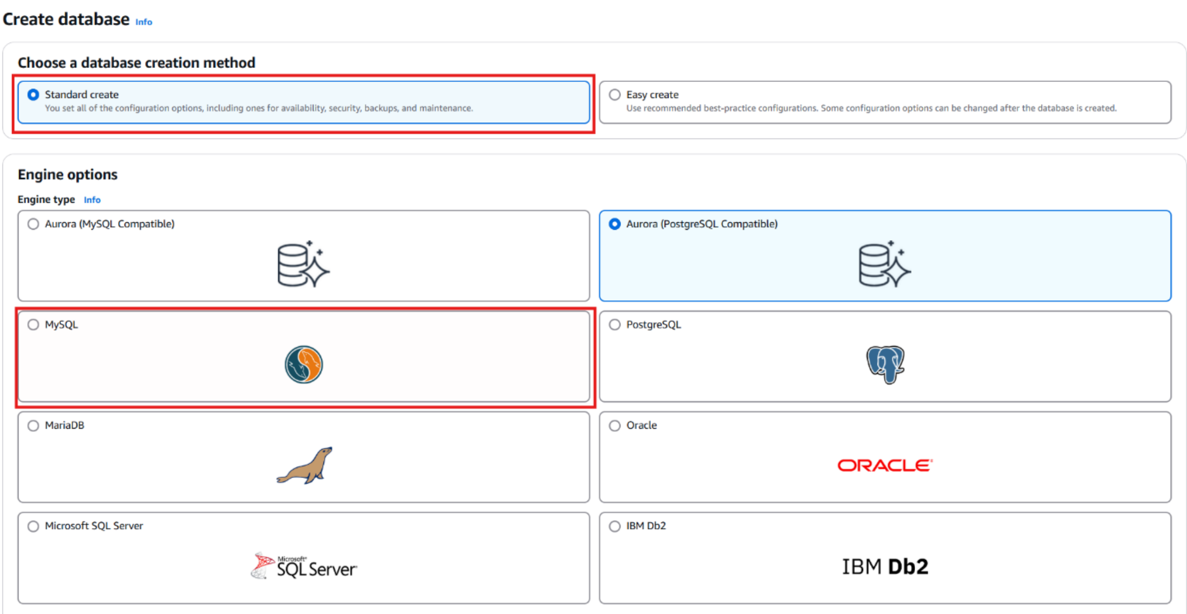
You should have something like this. Finally, click on **create**.

Create Database

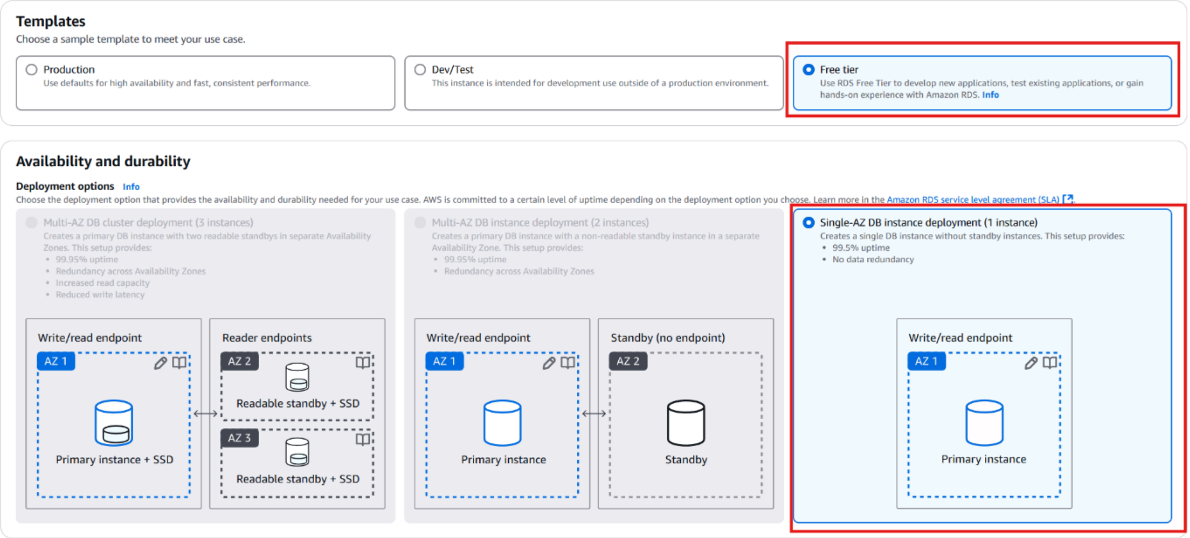




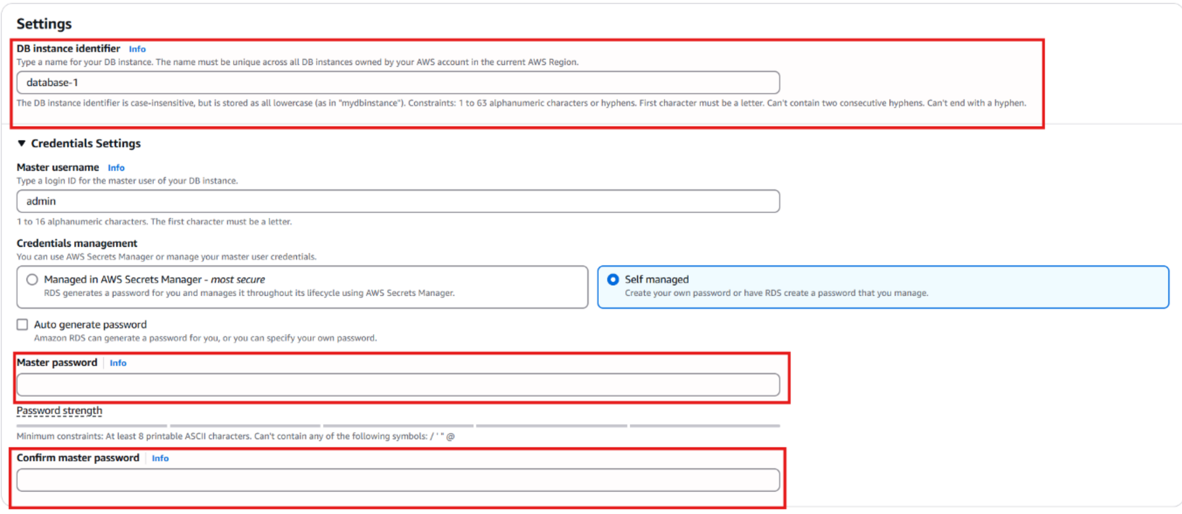
Choose from the side navigator Databases, then choose **Create Database**.



First, choose the **standard create** for the database creation method. Second, choose from the **Engine options: MySQL.**

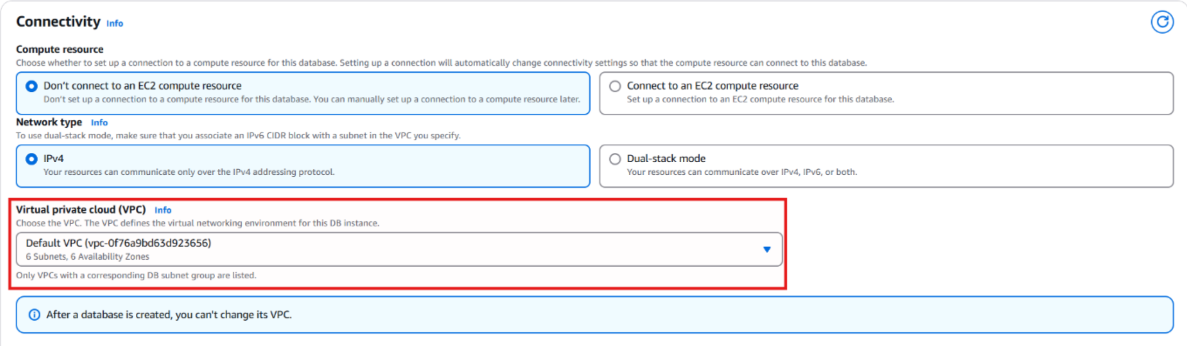


Scroll down to **Templates** and **choose Free tier,** and it will automatically choose for you "**Single-AZ DB instance deployment (1 instance)"** in the **Availability and durability.**

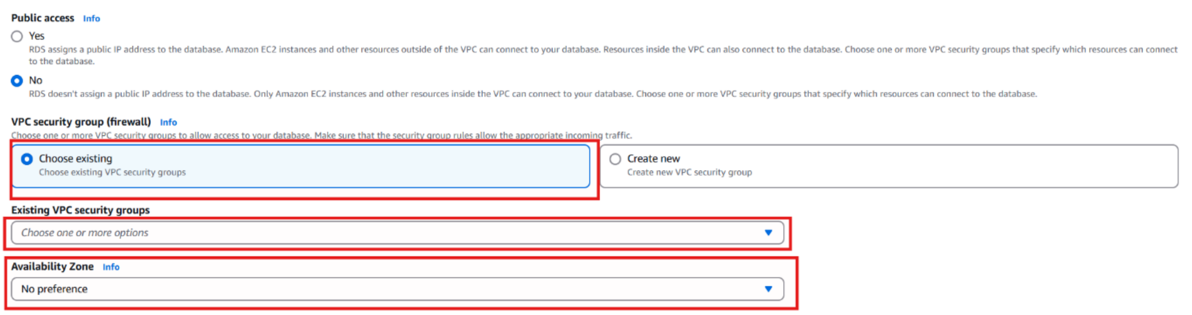


Scroll down to the settings and change the name for the **DB instance identifier.** Keep the default options and go to the **Master Password. Enter** *a strong password* and **remember it.**

Note: the password must be at least 8 characters.

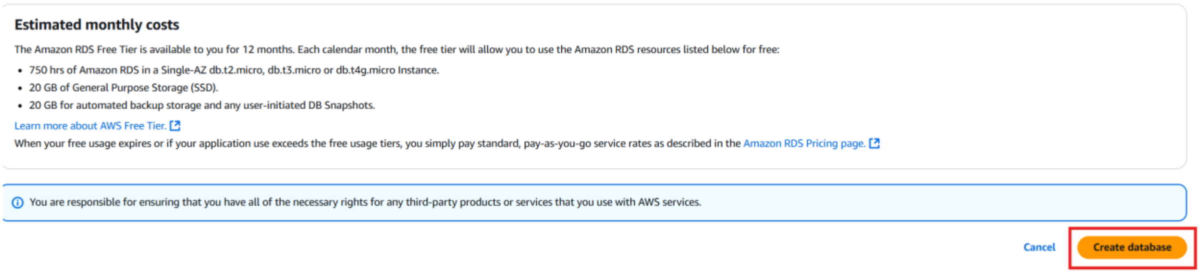


Scroll down to **Connectivity** and **choose your VPC** that you created before.



For the **VPC security group (firewall),** <u>choose existing</u>. Remove the default **VPC security groups** and add your **VPC security groups** that you have created before. For the **Availability Zone, choose it based on your private subnet's Availability Zone.**

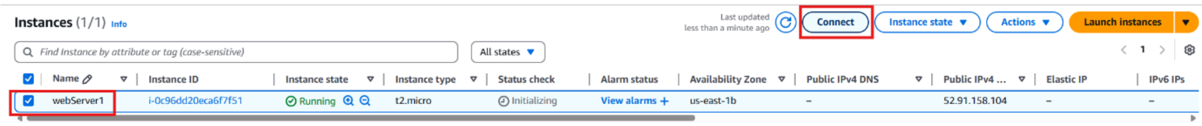
Note: You should **<u>Never select Yes</u> for Public Access** only if the question specifies it.



Click on **Create database.**

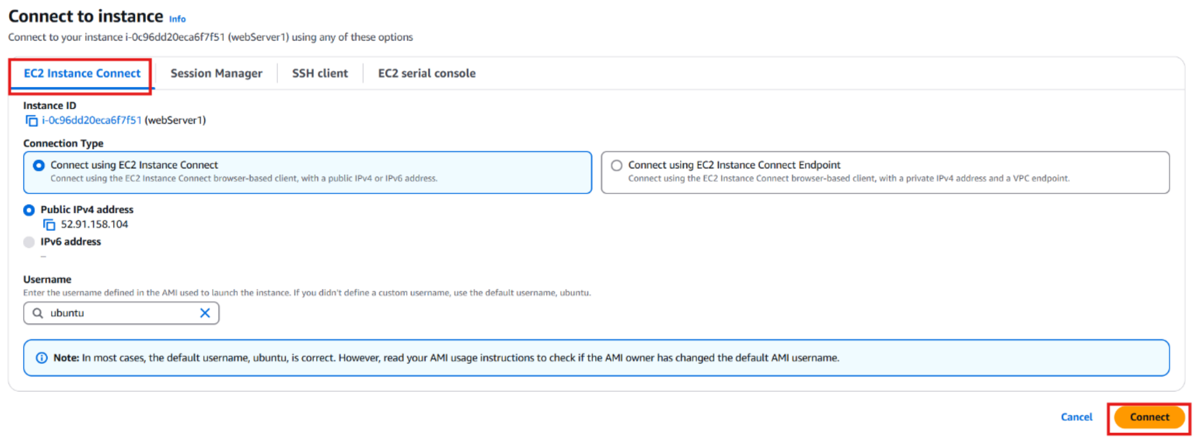
Note: it will take several minutes to fully create the database, wait for it for the status to change to **Available**.

Connect to your Database



Now go to the EC2 instance that you have created before:

To connect to the web server, **choose your instance** and click on **Connect**.

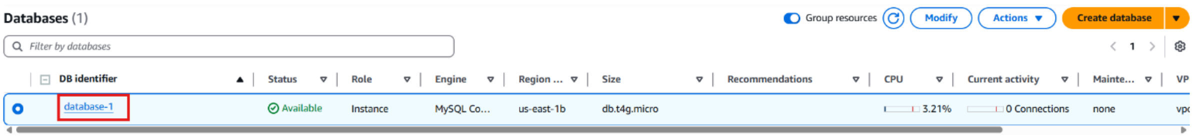


Choose **EC2 Instance Connect and click on Connect.**

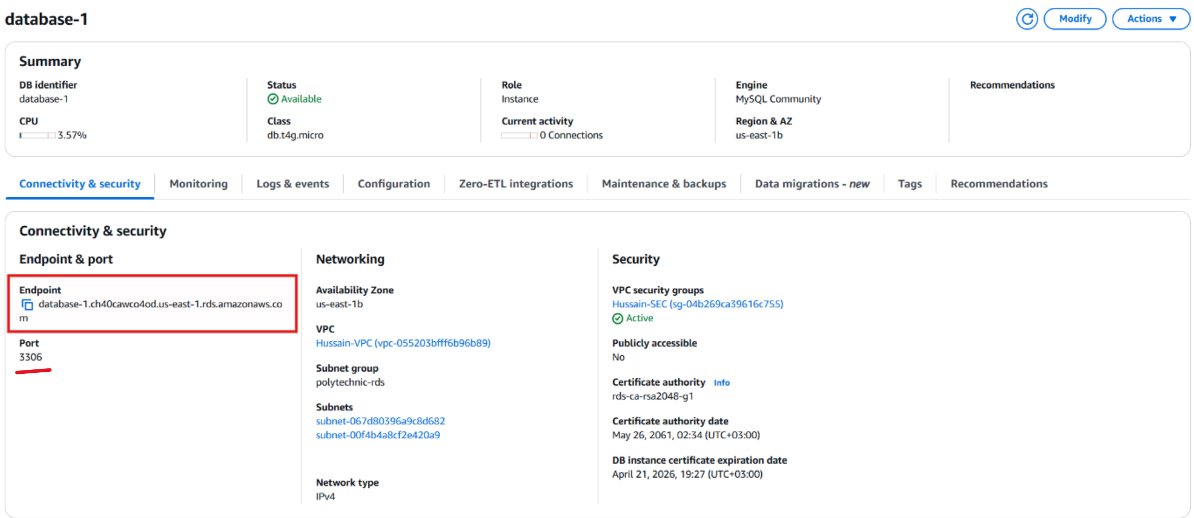
After you connect to the EC2 instance, issue the command "**Sudo apt update**". After finishing the update, run the following command "**sudo apt install mysql-client**" to install the **msql-client** in your EC2.

```
ubuntu@ip-192-168-0-19:~$ sudo apt install mysql-client
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  mysql-client-8.0 mysql-client-core-8.0 mysql-common
The following NEW packages will be installed:
  mysql-client mysql-client-8.0 mysql-client-core-8.0 mysql-common
0 upgraded, 4 newly installed, 0 to remove and 63 not upgraded.
Need to get 2765 kB of archives.
After this operation, 61.8 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

Enter “Y” to continue the installation.



To connect to the database that you have created before, go to **Aurora and RDS** and **click on the name of your database.**



Copy The **Endpoint** and remember the **port number**.

```
ubuntu@ip-192-168-0-19:~$ sudo su
root@ip-192-168-0-19:/home/ubuntu#
```

Enter in the terminal “**sudo su**” to go to the root.

```
root@ip-192-168-0-19:/home/ubuntu# mysql -h database-1.ch40cawco4od.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 31
Server version: 8.0.40 Source distribution

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Now you are at the root issue. The command “**mysql -h <Endpoint> -P <port number> -u admin -p**” will connect you to the MySQL database that you created. It will ask you to enter the password you chose when creating the database.

If it is shown mysql> in the end that means that you are in the database, you did correct.

Note: the endpoint and the port number are in the previous screenshot.

Now you can test how SQL work, try to enter this command for a test:

create database employees;

use employees;

create table poly(studentid int, studname varchar(20), marks int);

insert into poly values(1,'Ali', 30);

select * from poly;

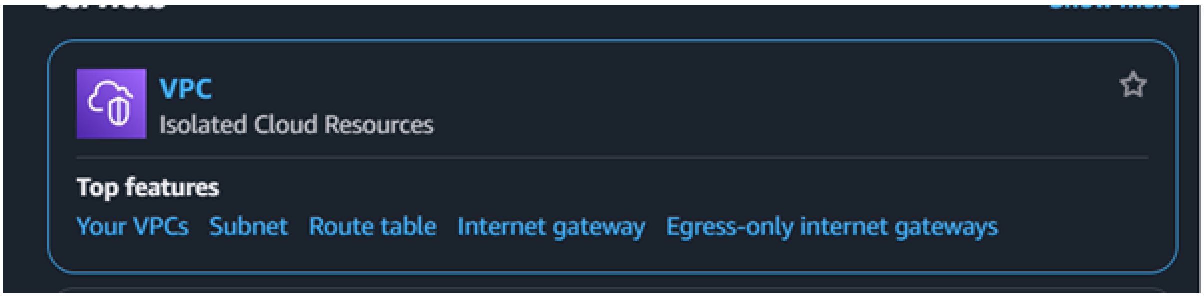
How to Create NAT Gateways

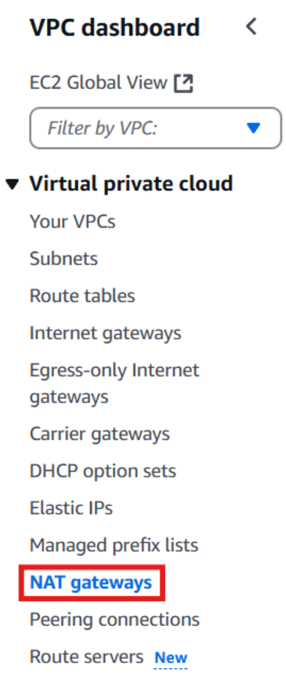
Note: you must create a VPC and two EC2 instances with Ubuntu on them (one with the public subnet and the other with the private subnet) then you can follow the steps.

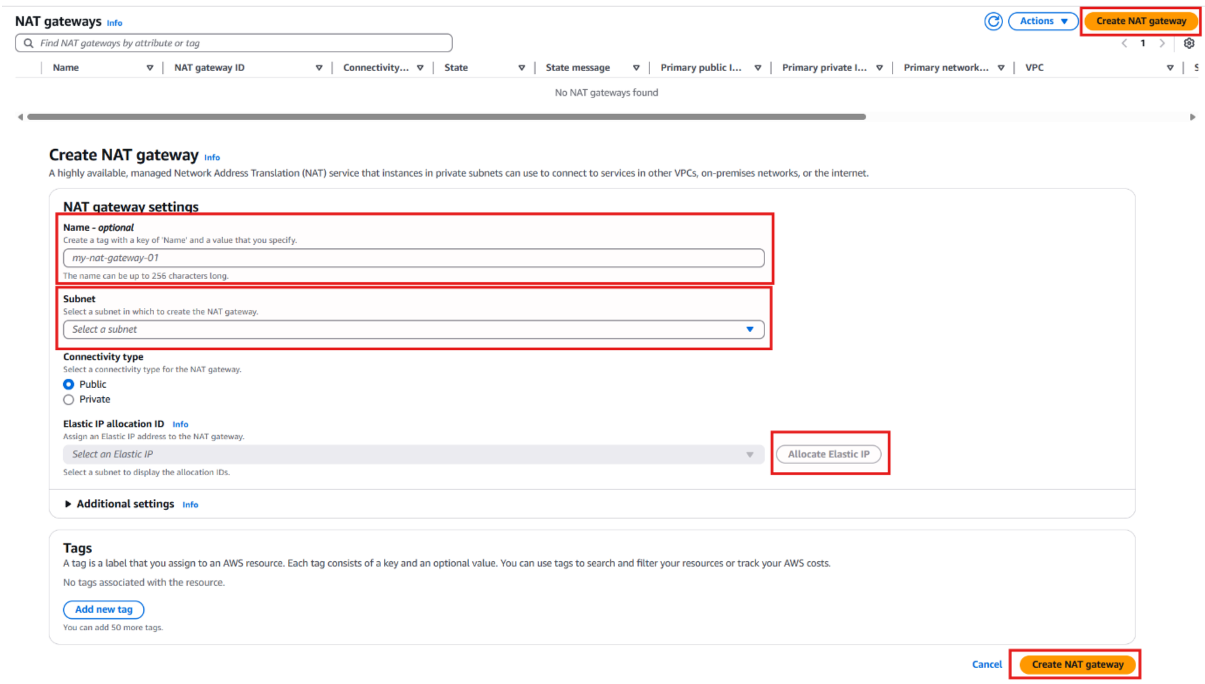
Remember:

- You must have at least **2 Subnets (Public, Private),** and each one must be in a different **Availability Zone.**
- The EC2 that you will create using the private subnet, you must **disable the Auto-assign public IP.**

Create NAT Gateways



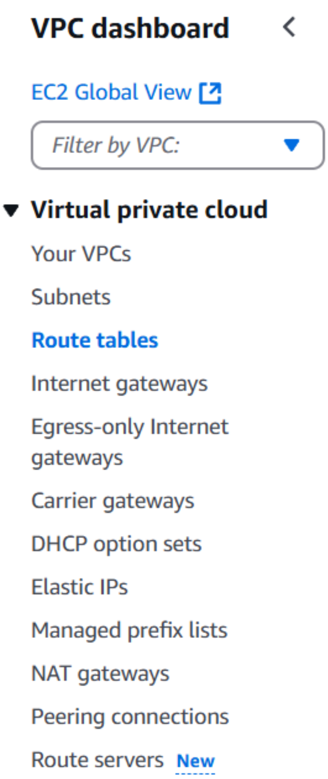


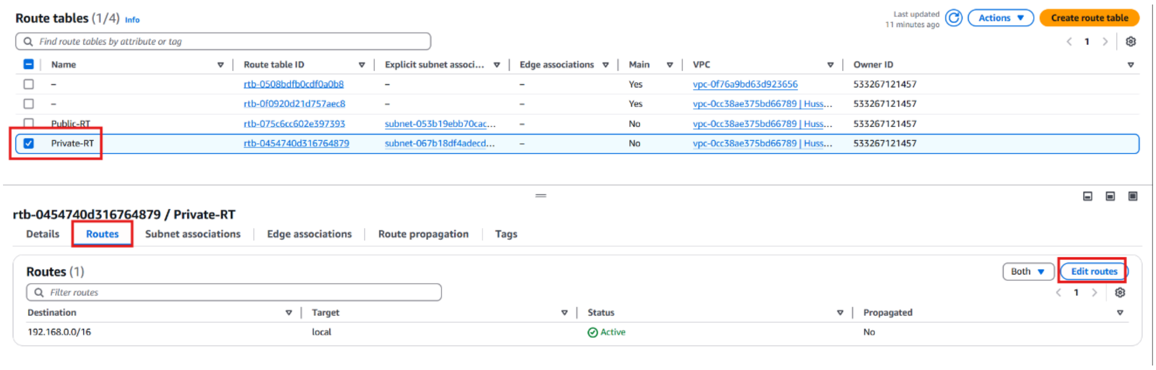


First, go to **VPC** from the left-side menu, choose **NAT Gateways,** and click **Create NAT Gateway.** Second, enter a name for the NAT gateway and choose the **public subnet.** Third, click on Allocate Elastic IP. Last, click on Create NAT gateway.

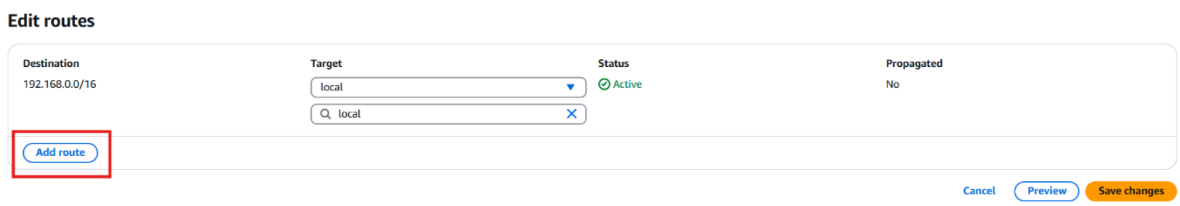
Note: the reason we create a NAT gateway is that the NAT gateway is a one-way direction that allows the service to access the internet, but doesn't allow the internet to access the service. It is a very good choice if you want to deploy an update or a batch for a database, for example.

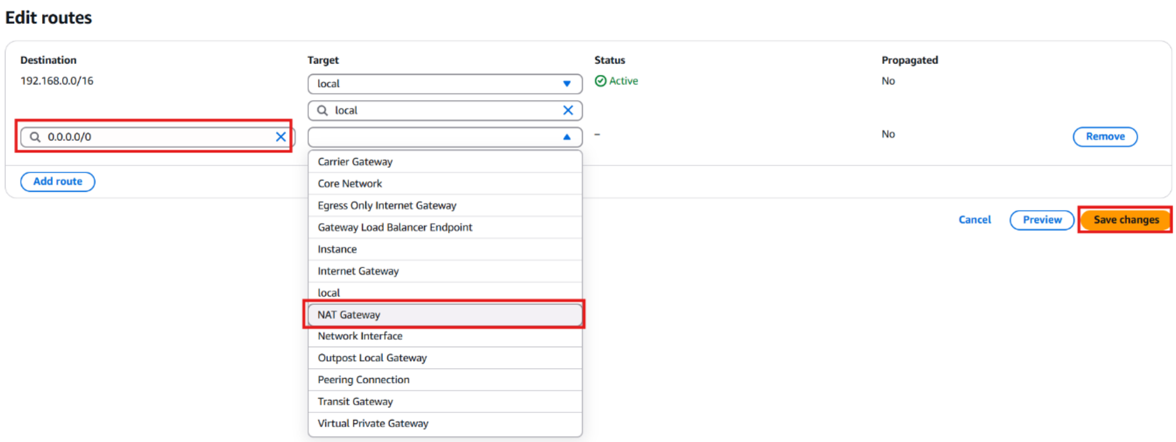
Link the NAT Gateway with a Private Subnet





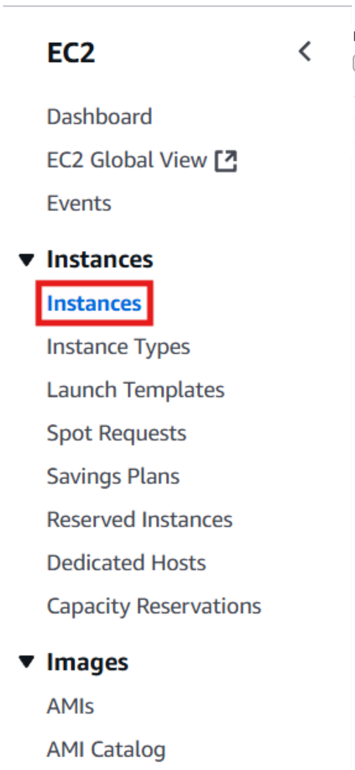
Go to **Routing tables** and choose your **private routing table**, and from the drop-down menu, choose **Routes** and click on **Edit routes**.

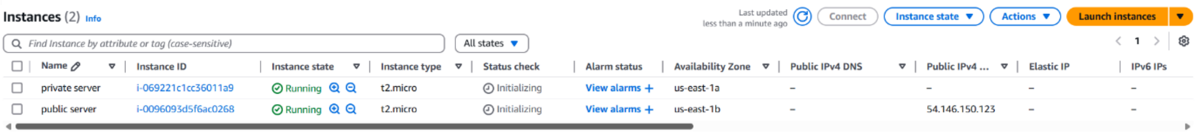




In Edit routes, click on **Add route** and choose **0.0.0.0/0** for the **destination,** and for the **Target, choose NAT Gateway** and **select your NAT gateway** you created before, and click on Save changes.

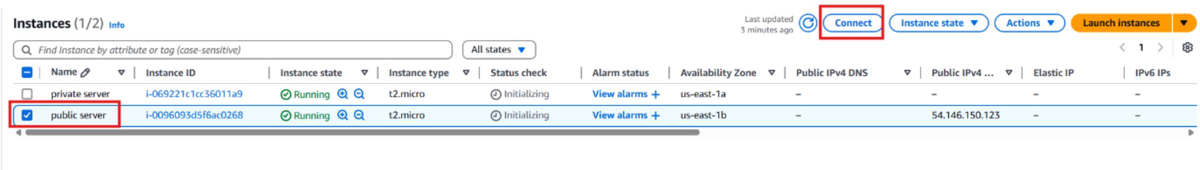
Checking the NAT gateway is working in the Private Server

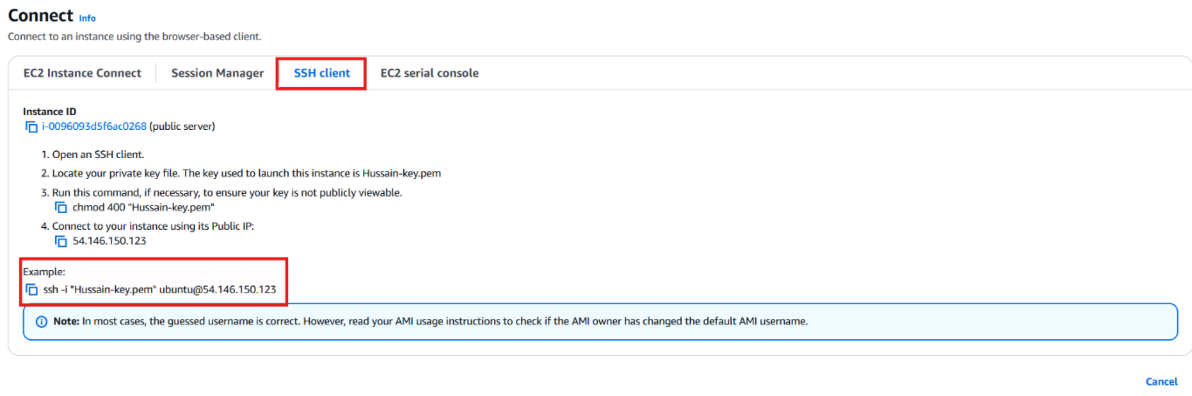




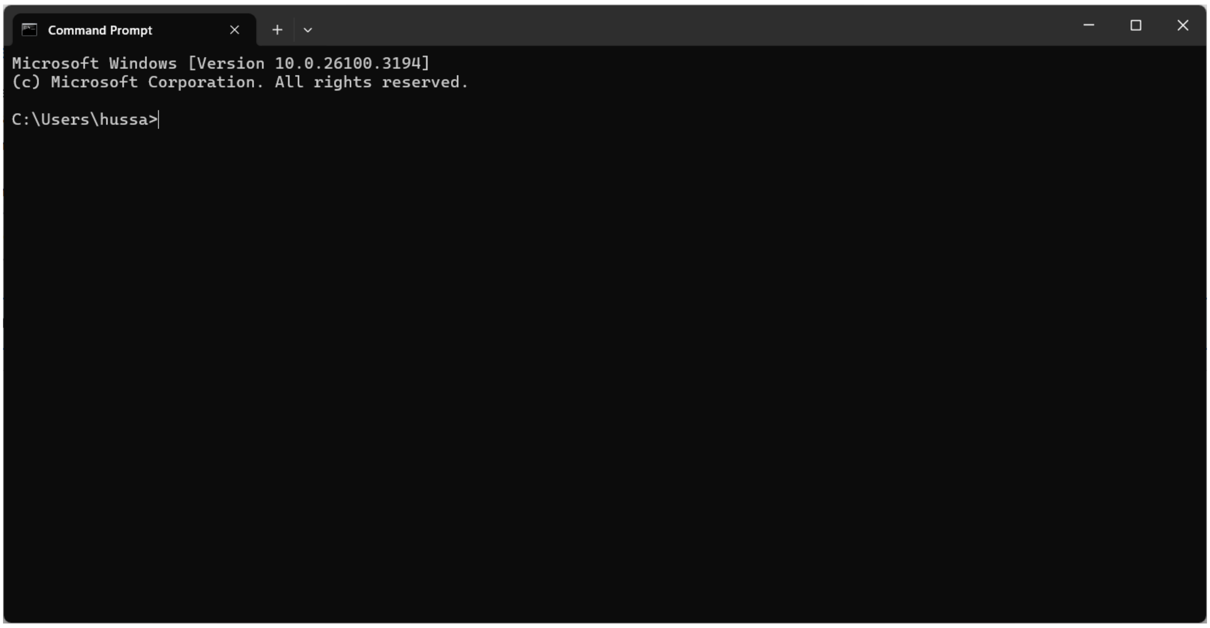
Go to EC2 and from the left-side menu choose **instances**. You should have **two instances running**, one with a public subnet and the other with a private subnet.

Note: You can use different keys and security groups for each instance, but note that **you must remember** what key you use for each instance.





Now select your instance that has the **public subnet** on it and click on **Connect.** Click on **SSH client** and **copy the Example**.



Open the **CMD** and go to where you saved your Key, and **paste the command you copied** from the AWS.

```
Command Prompt - ssh -i "H

Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hussa>cd Downloads

C:\Users\hussa\Downloads>ssh -i "Hussain-key.pem" ubuntu@54.146.150.123
The authenticity of host '54.146.150.123 (54.146.150.123)' can't be established.
ED25519 key fingerprint is SHA256:PzJFeHnXHfnbhsBDNWSVw7s6pq1WfCWjJ8k2J1jI+Pw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

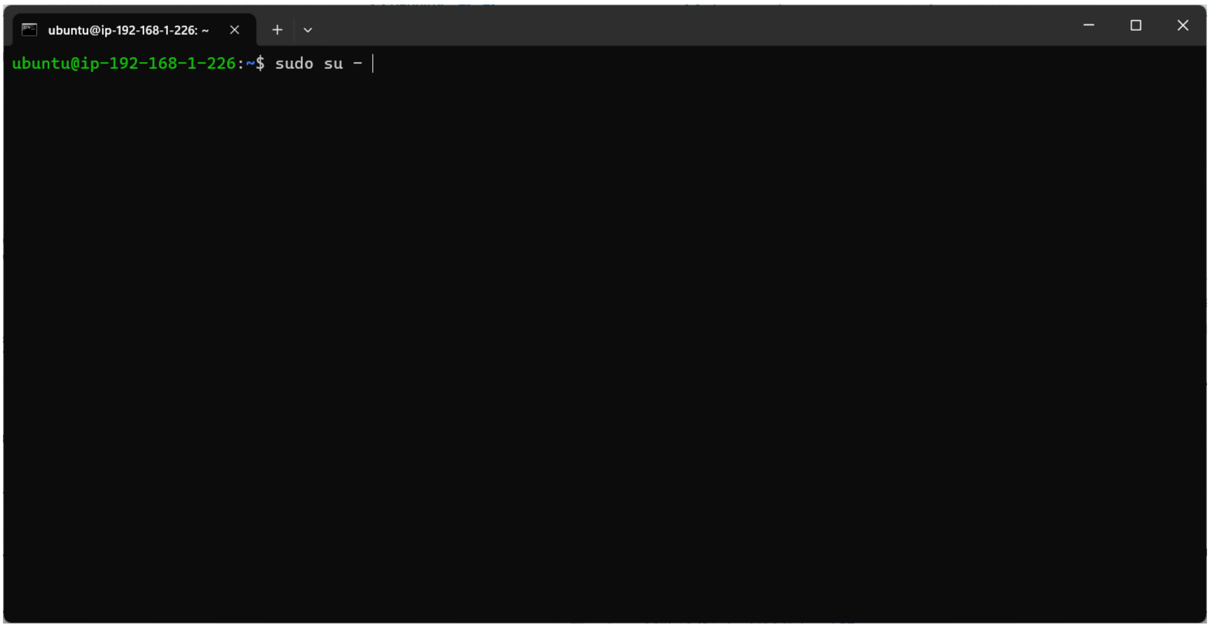
It will ask you to continue the connection by entering "YES".

```
ubuntu@ip-192-168-1-226: ~

 System information as of Sun Apr 27 15:02:01 UTC 2025

  System load:  0.0               Processes:             104
  Usage of /:   25.0% of 6.71GB   Users logged in:       0
  Memory usage: 20%               IPv4 address for enX0: 192.168.1.226
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-192-168-1-226:~$
```

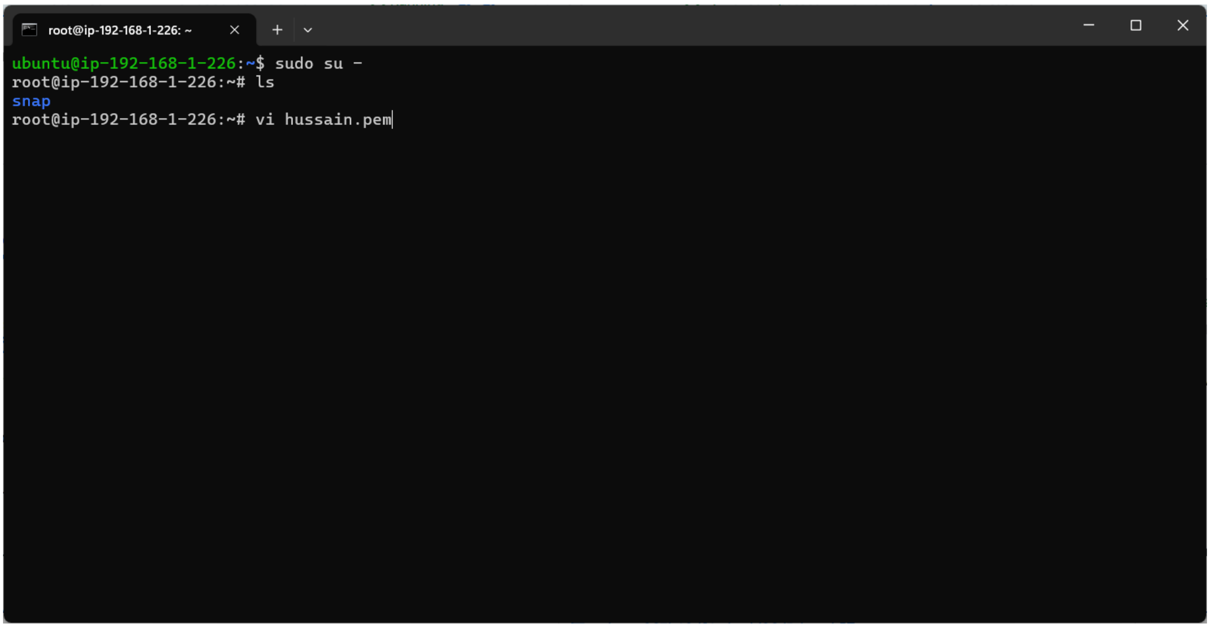
If you have done the EC2 instance correctly, it will open for you like this.

Now update your Ubuntu by typing the command "**sudo apt update**" and install Apache2 by running the command "sudo apt install apache2".

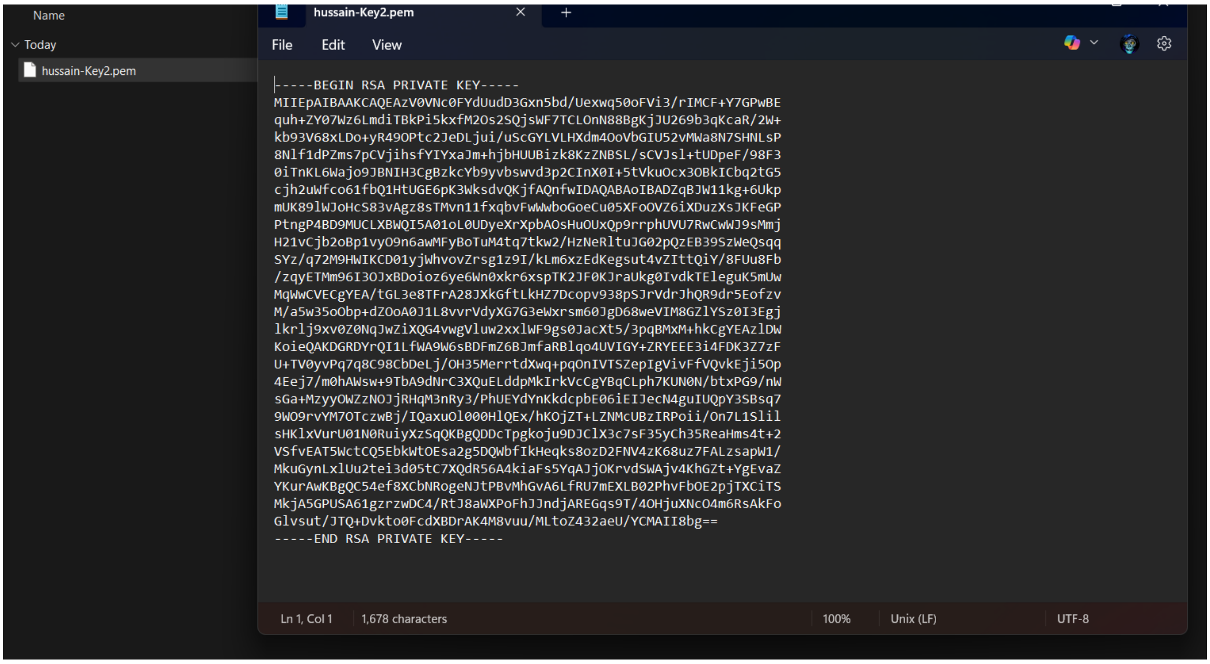
Connect to the Private Server using the Public Server.



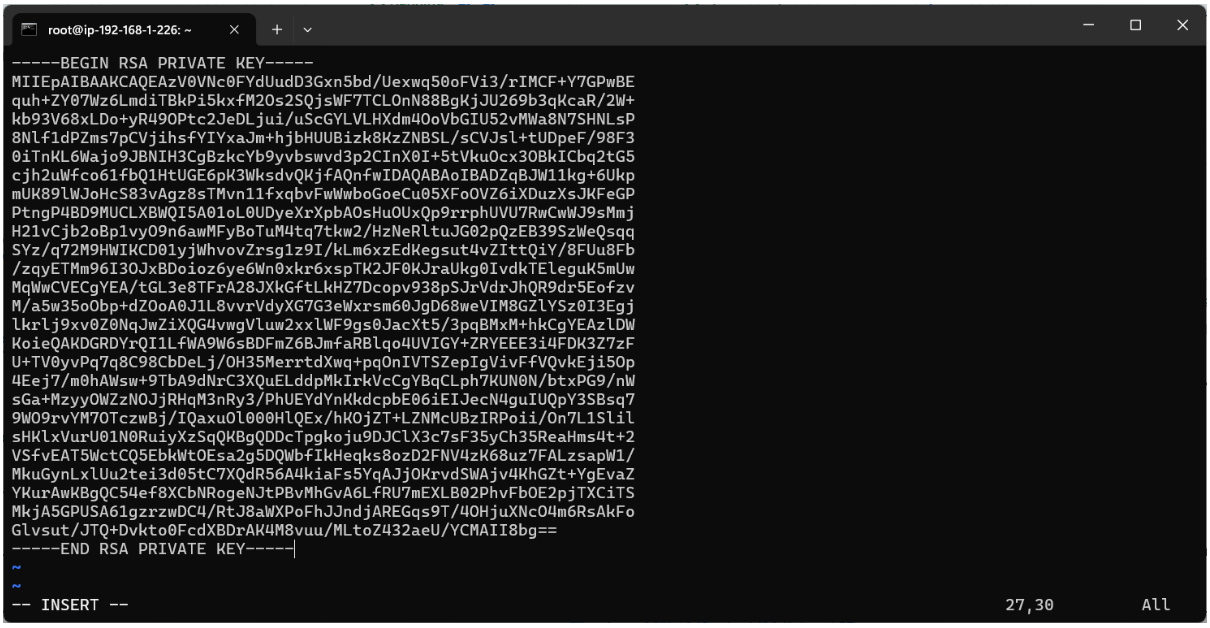
Now go to the Root by running the command “**sudo su -**”



In the root, issue the command “**Vi and any name. pem**” to create a file in which you will put the key for a private EC2 instance.



Go to your key, which is for the private instance, and open it with a notepad and copy the whole text.



Inside your file, you create, enter “**i**” to insert in the file, paste the key content in it, and click “**ESC: wq**” to close it and save what you entered.

```
-rw-r--r-- 1 root root 1679 Apr 27 15:23 hussain.pem
drwx------ 3 root root 4096 Apr 27 14:52 snap
root@ip-192-168-1-226:~#
```

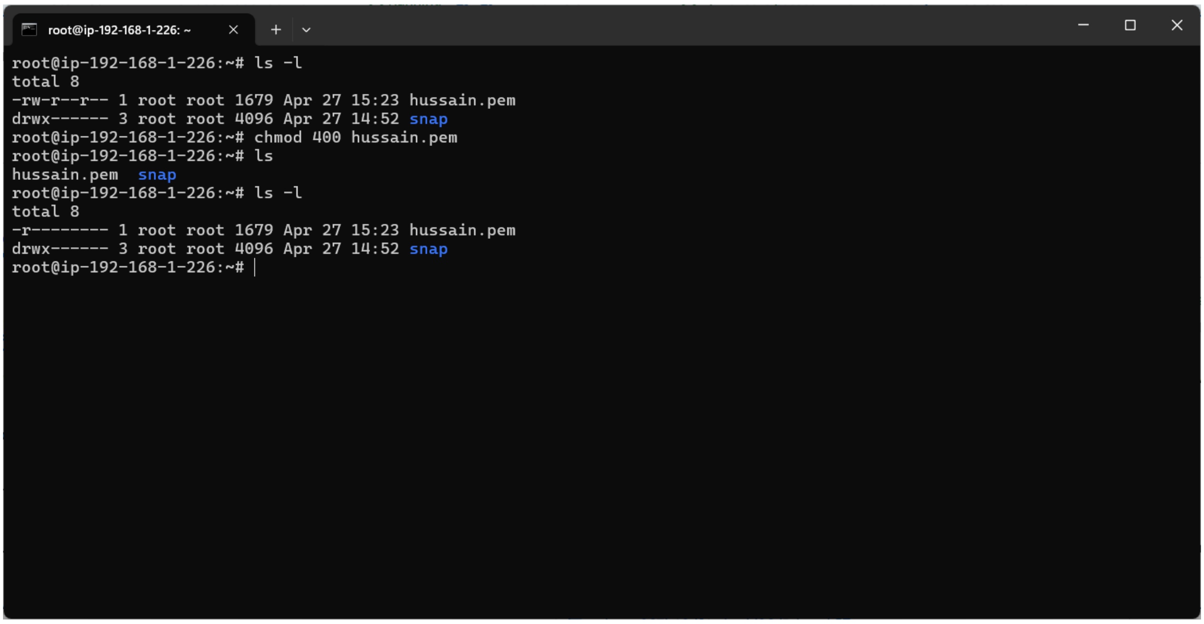
If you run the command "**ls -l**", you will see the key has read permissions for the group and others, and we do not want that.

Note: If we kept permissions like this, anyone can read and write in the key file, and we don't want that to happen.

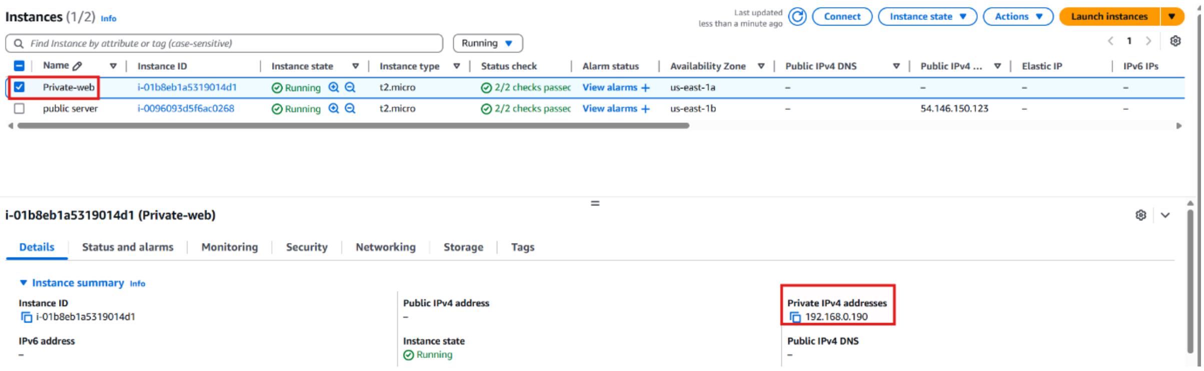
```
root@ip-192-168-1-226: ~
root@ip-192-168-1-226:~# ls -l
total 8
-rw-r--r-- 1 root root 1679 Apr 27 15:23 hussain.pem
drwx------ 3 root root 4096 Apr 27 14:52 snap
root@ip-192-168-1-226:~# chmod 400 hussain.pem
```

By running the command "**chmod 400 <name of the key>.pem**" it will change the permissions.

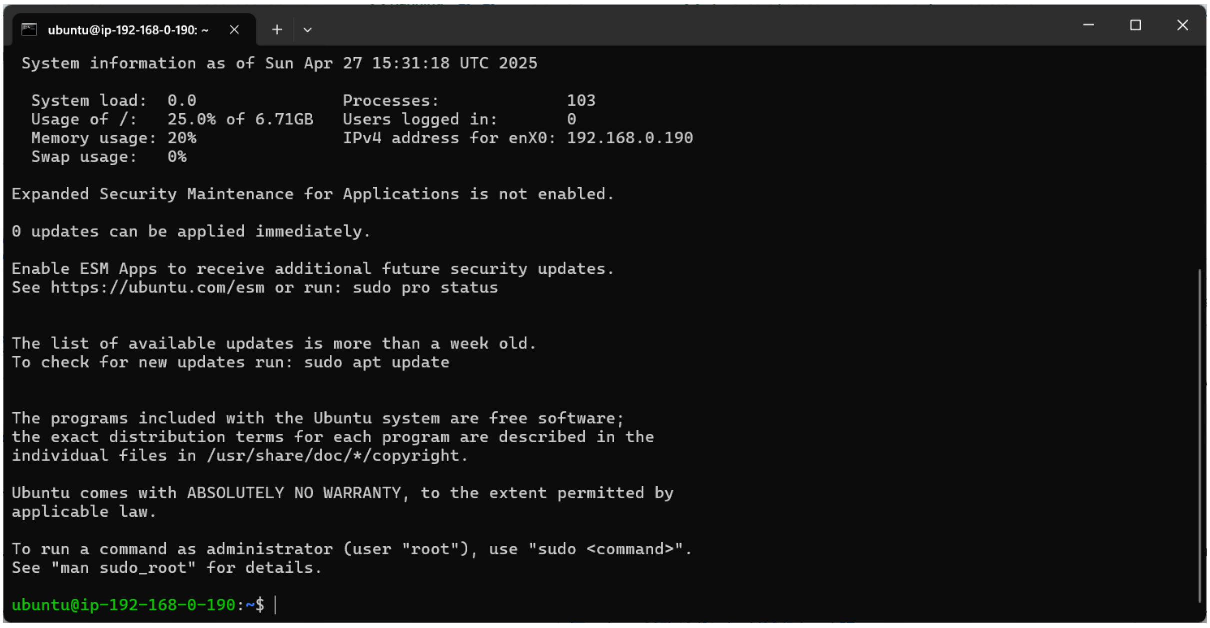
Note: After running the command, it will change the permissions of the key so that only the owner can read and modify the file.



As you can see, the **permission has been changed**.

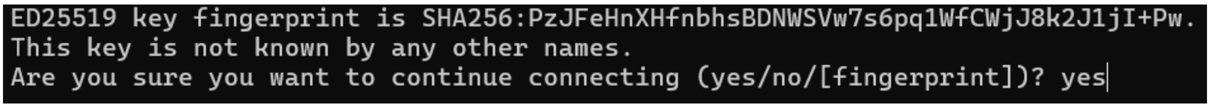


Go to your **private EC2** instance, select it, and **copy** from the drop-down menu the **private IP.**



```
ubuntu@ip-192-168-0-190: ~
System information as of Sun Apr 27 15:31:18 UTC 2025

  System load:  0.0               Processes:             103
  Usage of /:   25.0% of 6.71GB   Users logged in:       0
  Memory usage: 20%               IPv4 address for enX0: 192.168.0.190
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-192-168-0-190:~$
```

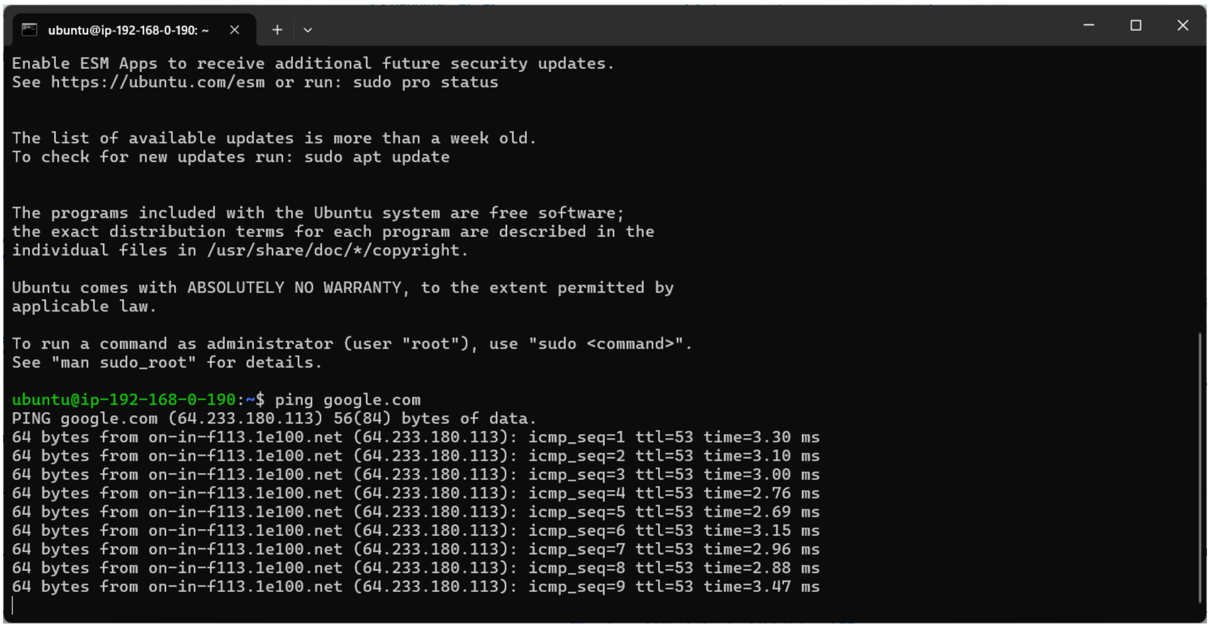
Run the command “**ssh -i <name of the key>.pem ubuntu@<private ip that you copy>**”



```
ED25519 key fingerprint is SHA256:PzJFeHnXHfnbhsBDNWSVw7s6pq1WfCWjJ8k2J1jI+Pw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

It will ask you to continue connecting, type: “YES”

Note: If you copy the key correctly and **change the permission,** and with the correct **private IP,** it will open like the picture.



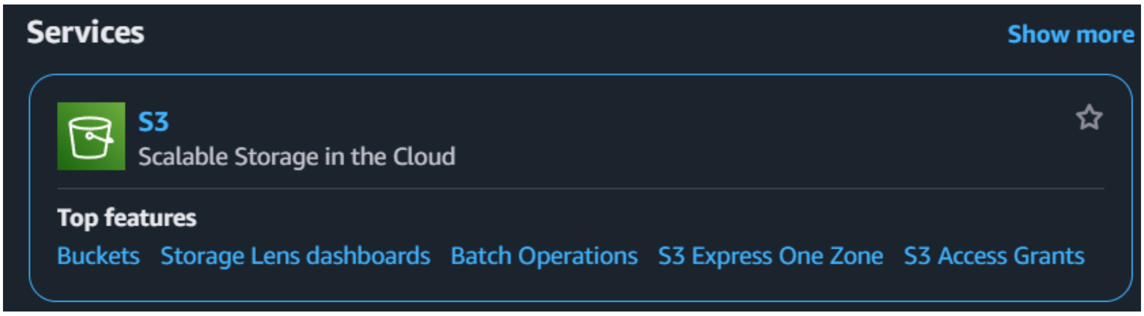
```
ubuntu@ip-192-168-0-190: ~
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-192-168-0-190:~$ ping google.com
PING google.com (64.233.180.113) 56(84) bytes of data.
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=1 ttl=53 time=3.30 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=2 ttl=53 time=3.10 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=3 ttl=53 time=3.00 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=4 ttl=53 time=2.76 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=5 ttl=53 time=2.69 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=6 ttl=53 time=3.15 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=7 ttl=53 time=2.96 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=8 ttl=53 time=2.88 ms
64 bytes from on-in-f113.1e100.net (64.233.180.113): icmp_seq=9 ttl=53 time=3.47 ms
```

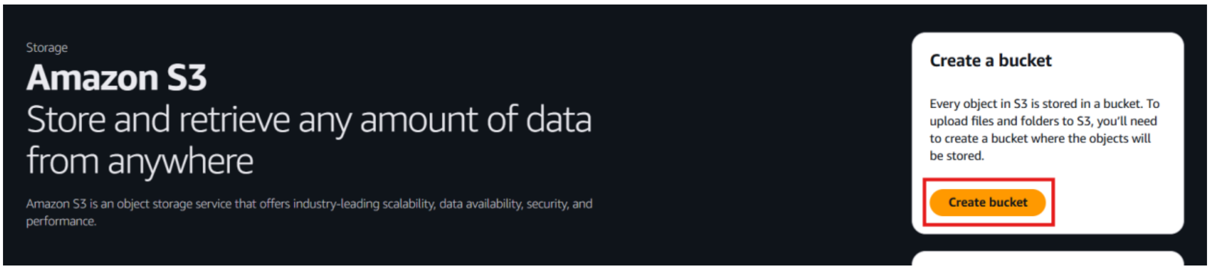
Run the command "**ping google.com**". If the ping is successful, that means you have done everything correctly and the **private EC2 instance has internet connection,** but no one can have access from the internet to your private server because of the NAT gateway.

How to Create an S3 Bucket

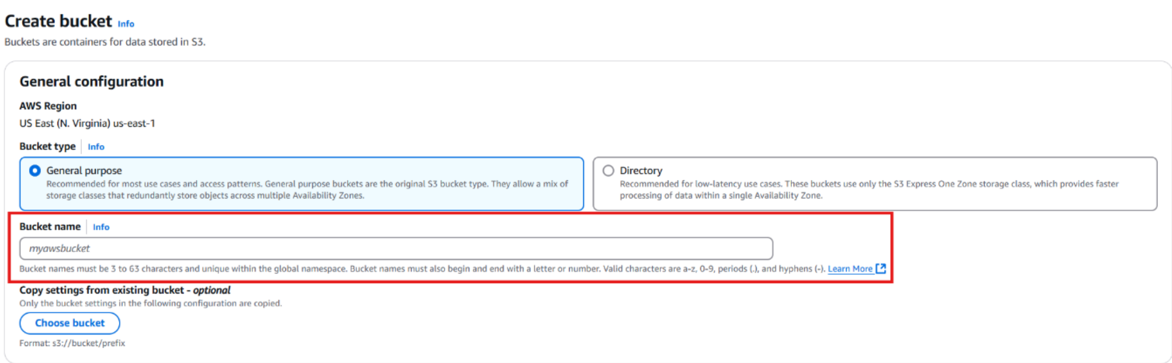
The S3 bucket is for storing any type of data, and what you store on it is called objects, and it can hold nearly unlimited data. There are many types of it, each one has its benefits, but for this demonstration, we will use the standard one.

Create an S3 bucket without ACL.

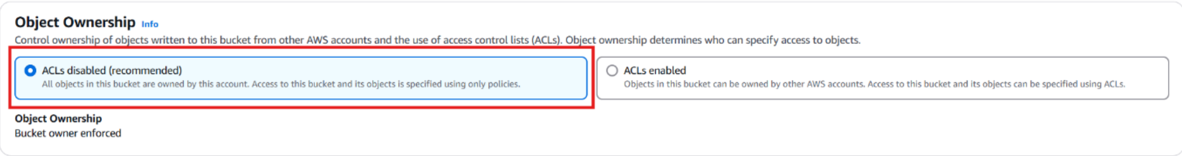




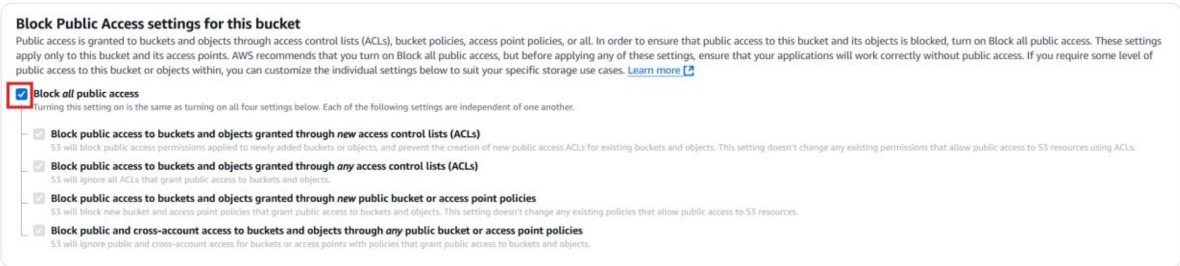
First, search for **S3** in the AWS console, in the home page on the S3 bucket, click on **Create bucket.**



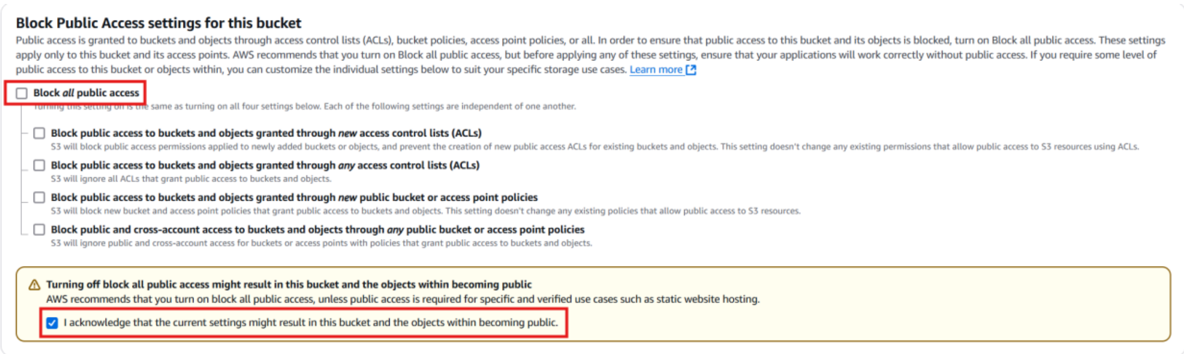
Leave the **bucket type as the default option, then choose a name for your bucket. It must be unique**.



For the **object ownership,** leave the **ACL disabled**.



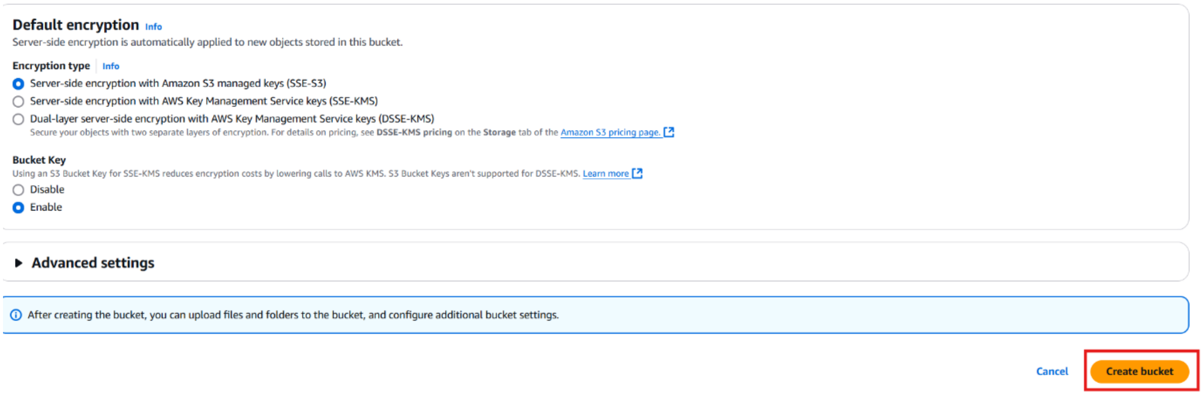
For the **Block Public Access settings for this bucket,** uncheck the **Block all public access " option**.



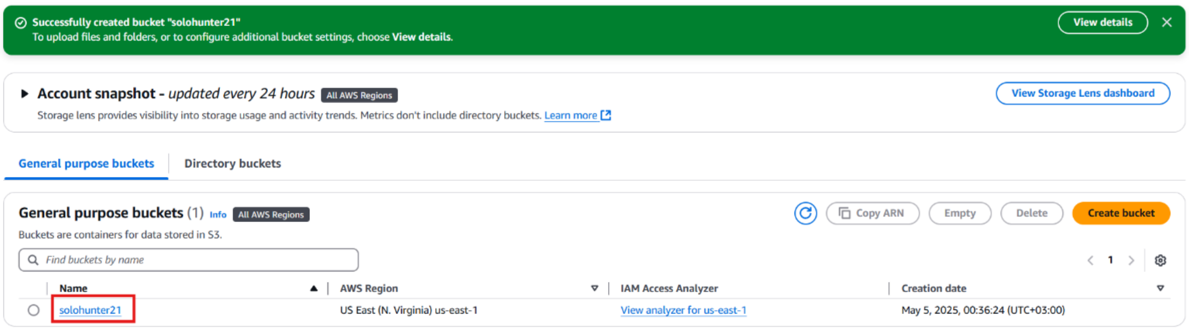
Select the **I acknowledge**

Note: by default, the S3 bucket is blocking public access so that you don't have your object in the public, but if you want to, you have to enable the public access.

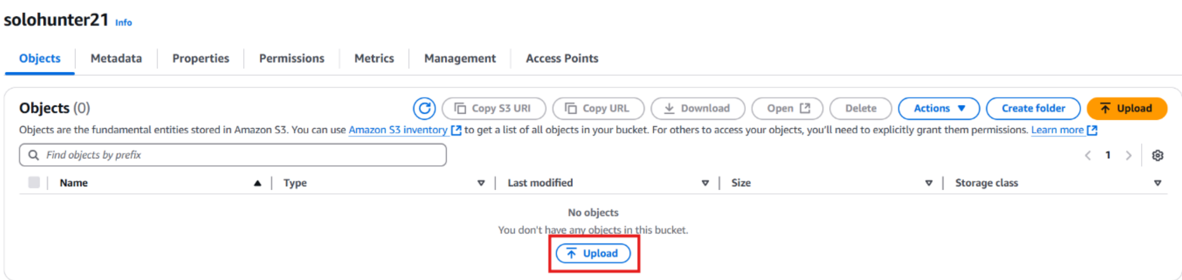
The meaning of **Versioning** is that every update you do for the bucket, AWS will keep a copy of your old version of the bucket, but for this demonstration, you will not use it.



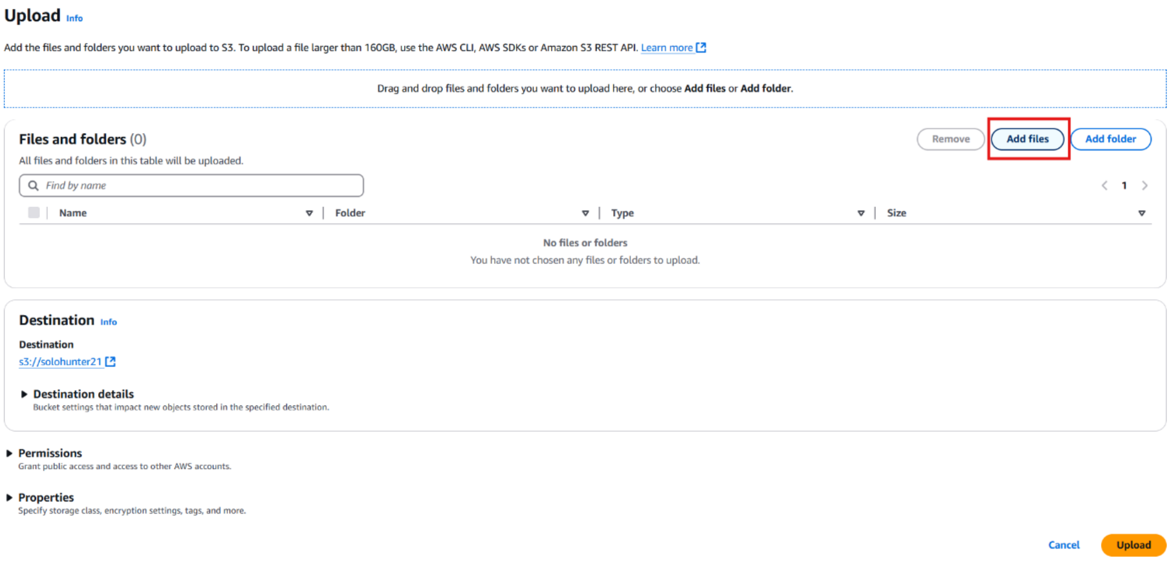
Click on **Create bucket**.



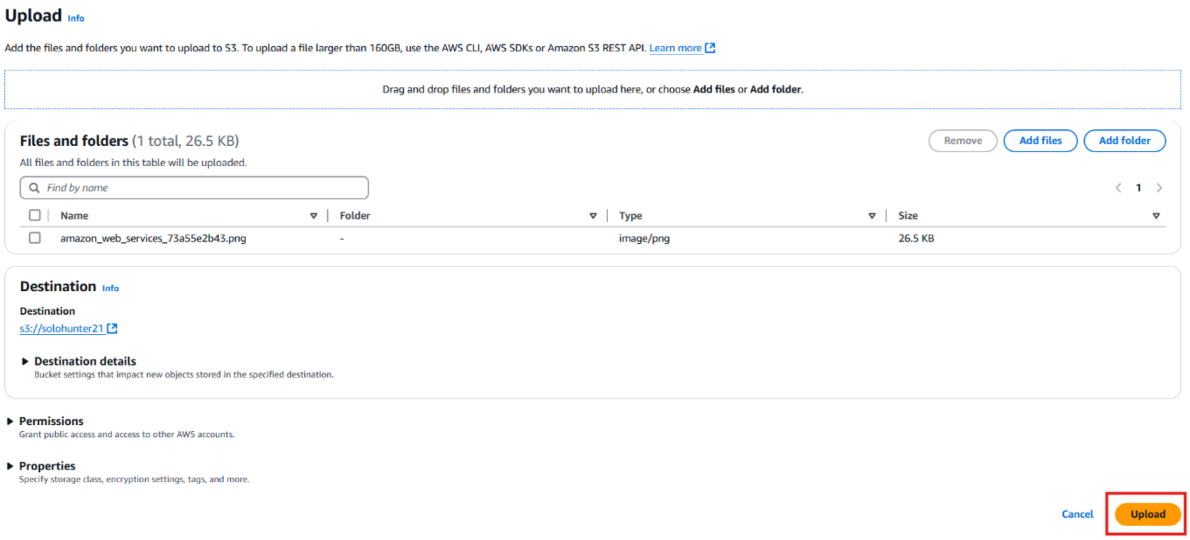
click on your bucket name.



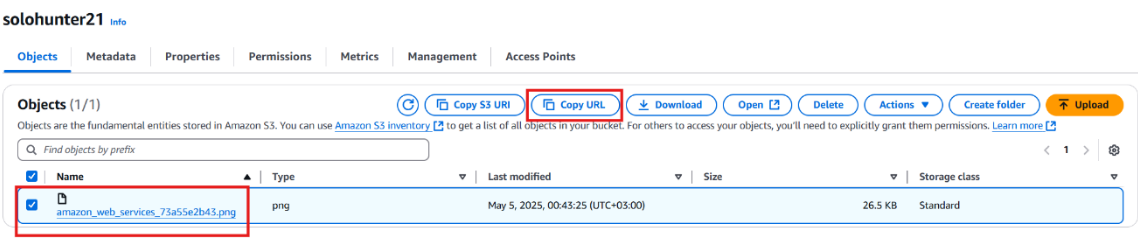
click on **upload** to upload your files or folders.



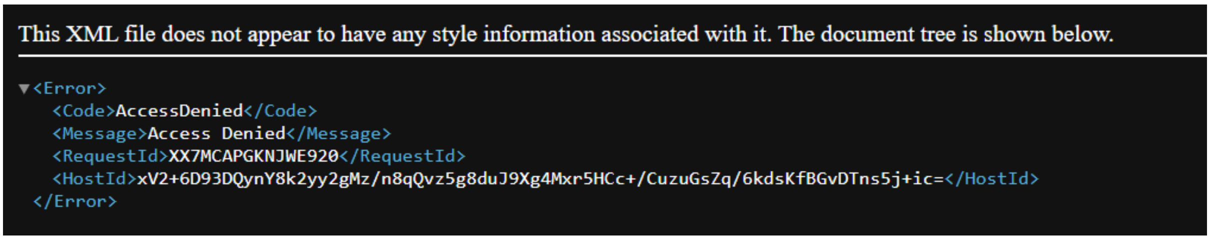
choose **Add files** or **folder** as you want.



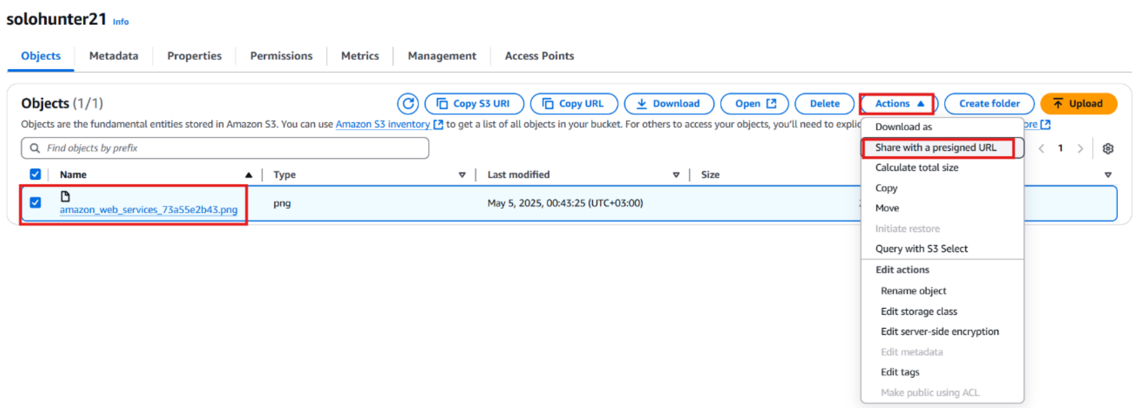
After uploading your file and folders click on **upload**.



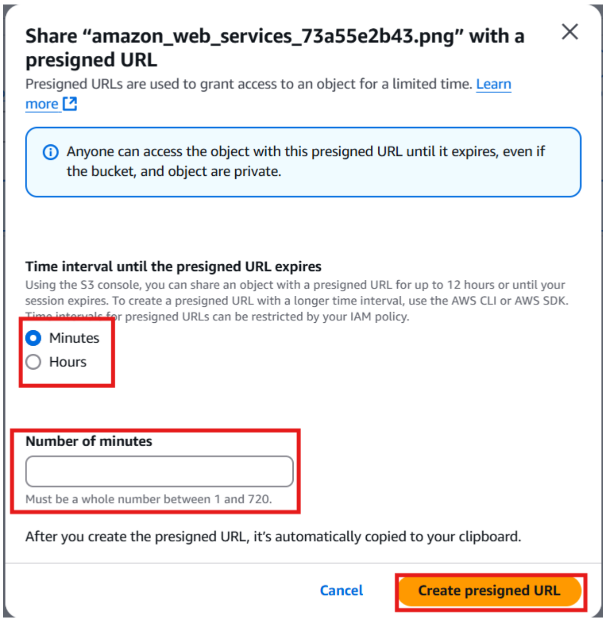
Return to your bucket and **select your file** that you upload and click on copy URL and open new tab and paste the URL and **see what happen**.



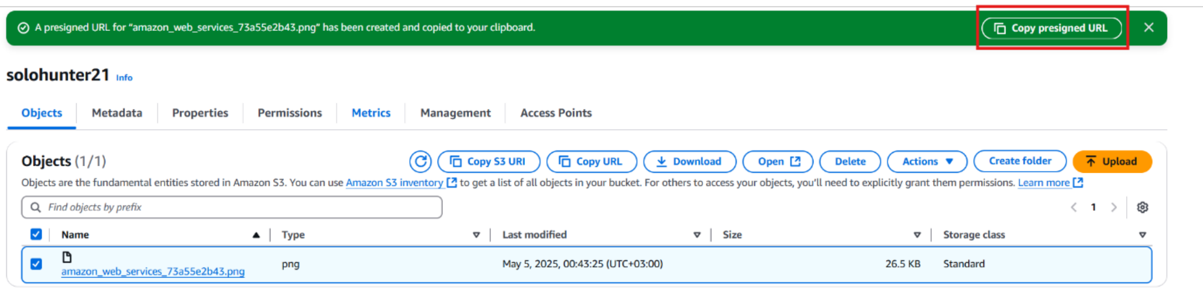
It will appear like this. The reason the photo won't open is that you gave public access to the bucket, not the file itself. If you want to access the files inside the bucket, you need to specify the exact file you want to access.



To give access for the photo first, **select the photo**. Second, click on **action,** then on **share with a presigned URL**.

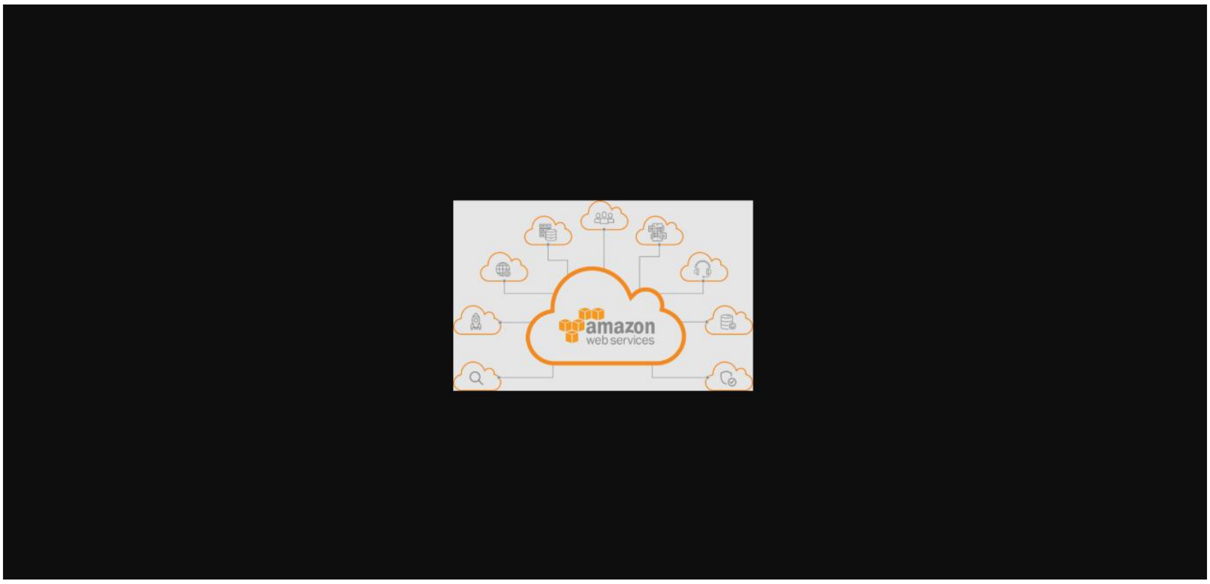


Choose how much time you want the photo to be accessible, and click on **Create presigned URL**.



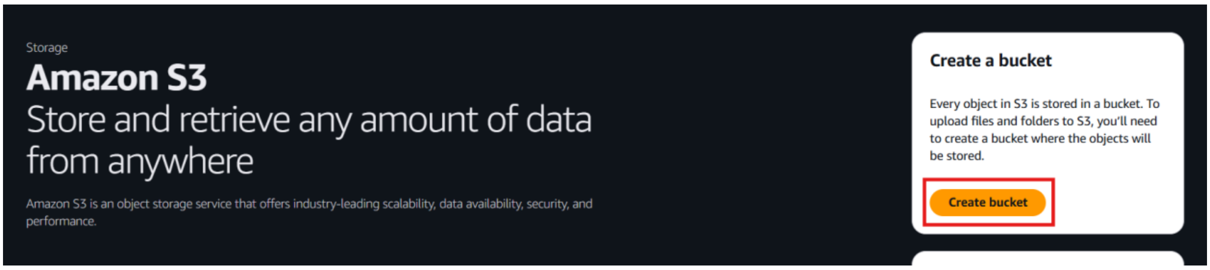
Click on **copy presigned URL** and paste it in a new tab.

If you don't have this, you can click on **open**, and it will open the photo in a new tab.

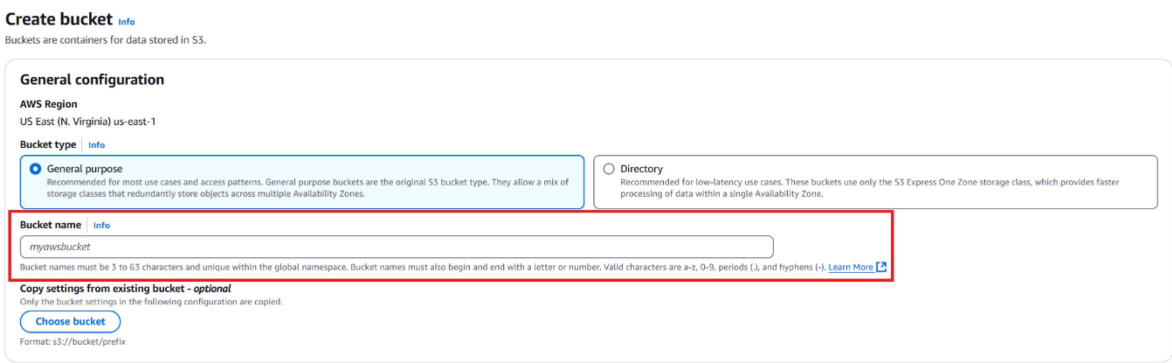


As you can see, the photo can now be accessed.

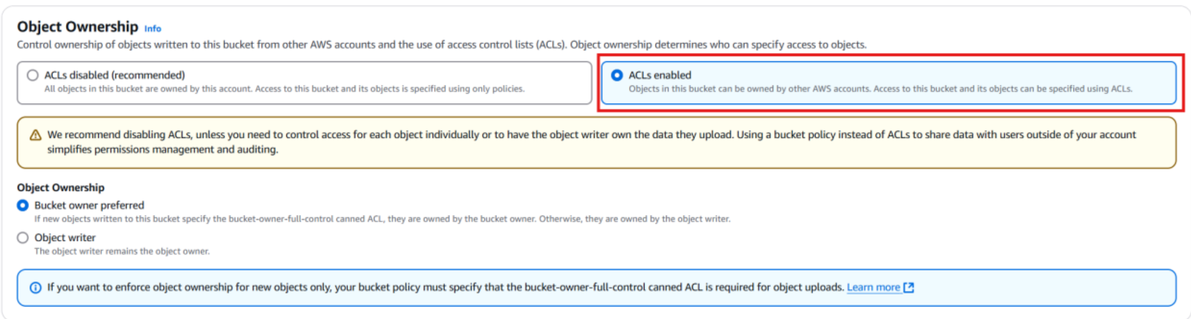
Create an S3 bucket with ACL



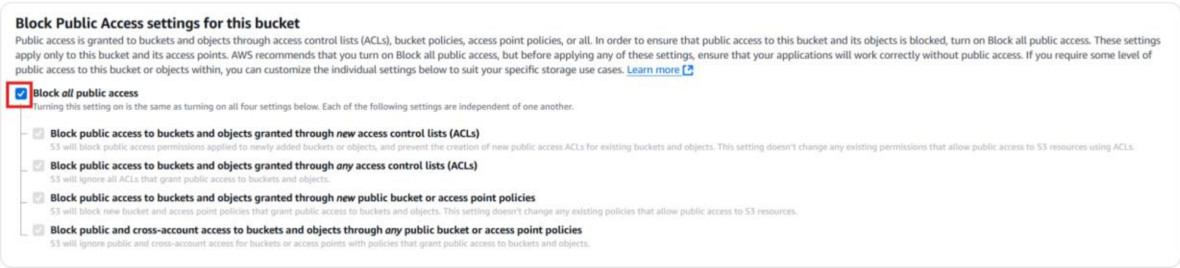
First, search for the S3 bucket, then in the home page of the S3 bucket, click on **create bucket.**



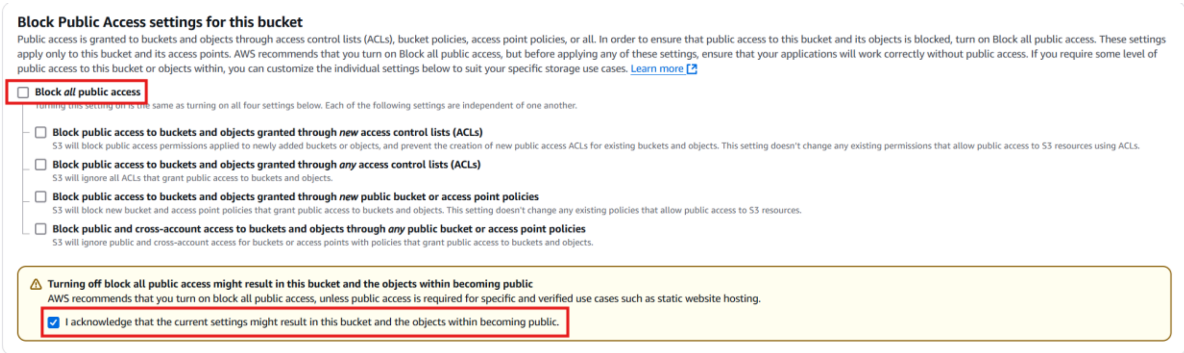
Leave the **bucket type as default choice**, then choose a name for your bucket**. It must be unique**.



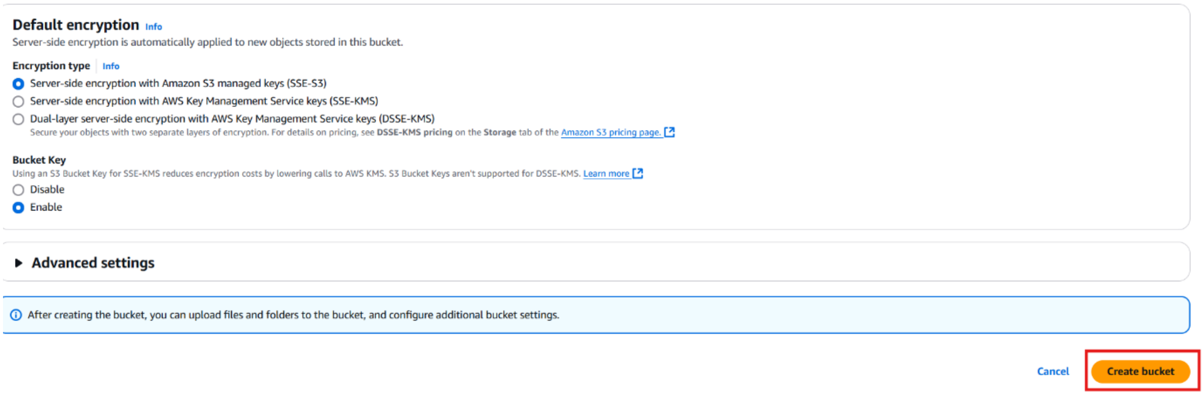
Now you will choose **ACL enable** and leave the **object Ownership** as the **default choice**.



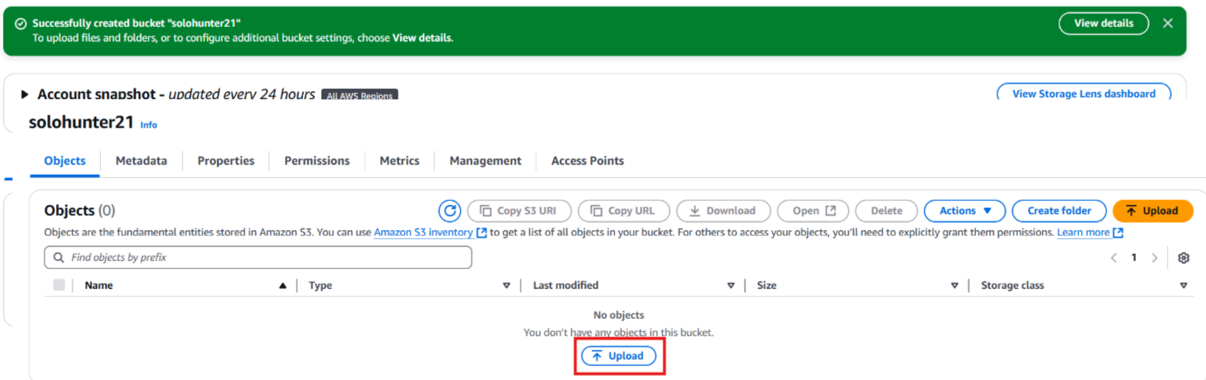
For the **Block Public Access settings for this bucket,** uncheck the Block all public access.



Select the **I acknowledge.**



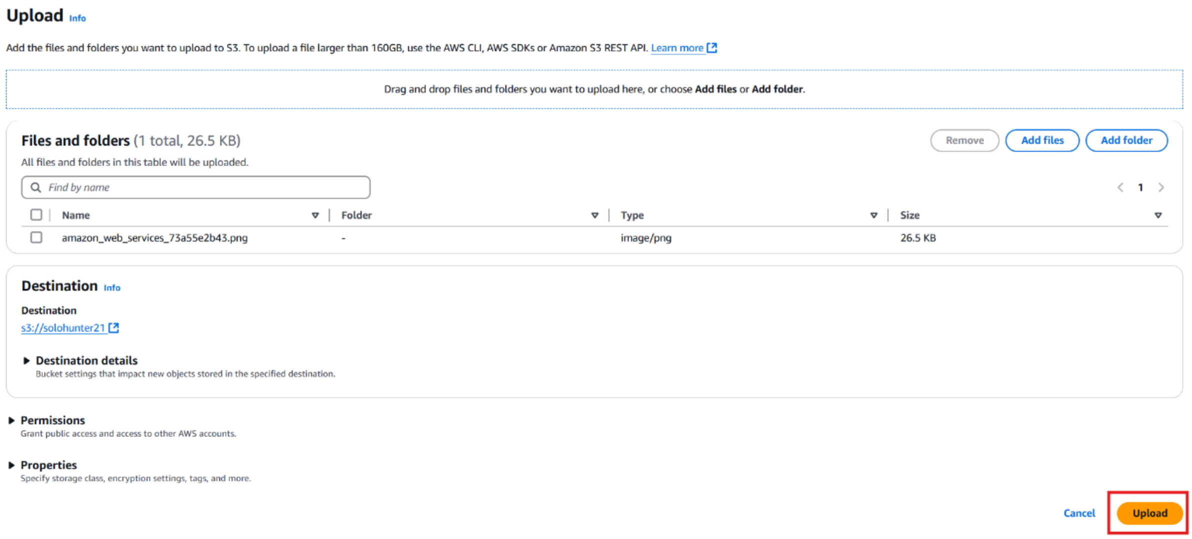
Leave the rest as the default option and click on Create bucket.

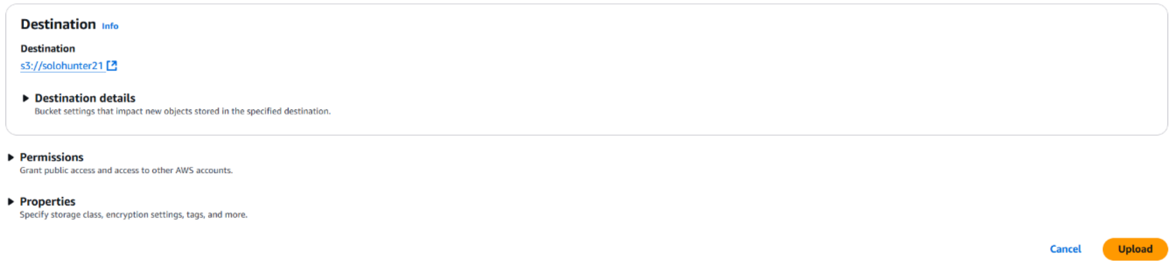


click on your bucket name.

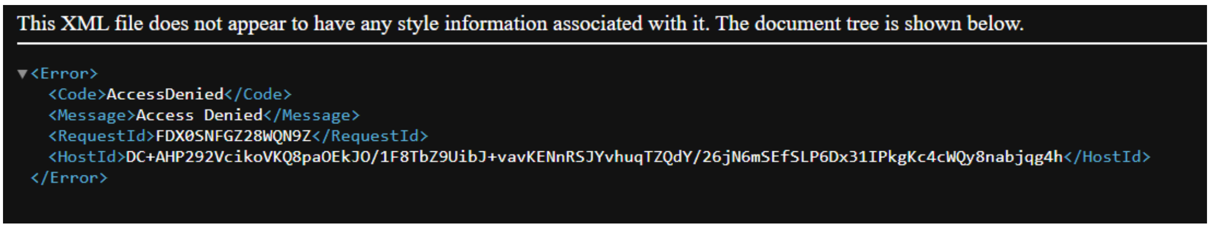
Click on **upload** to upload your files or folders.

choose Add files or folder as you want.

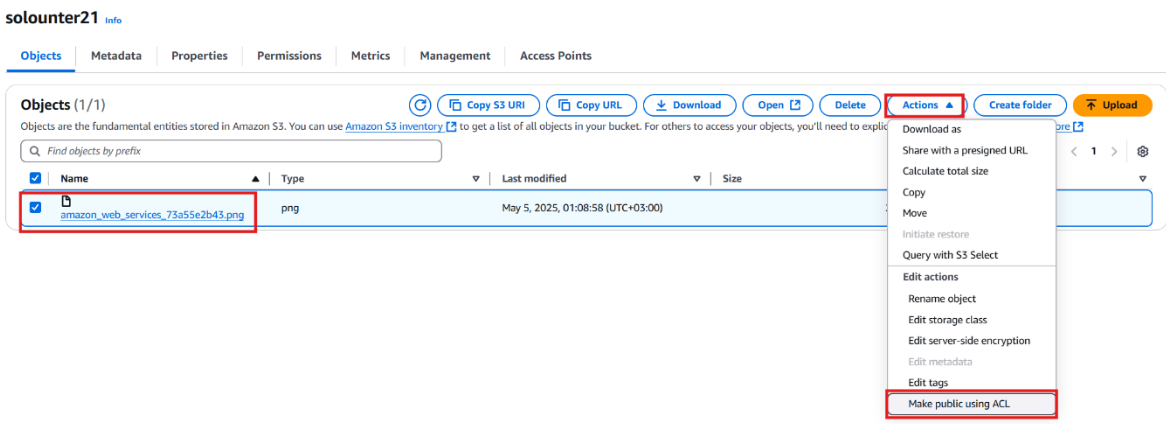




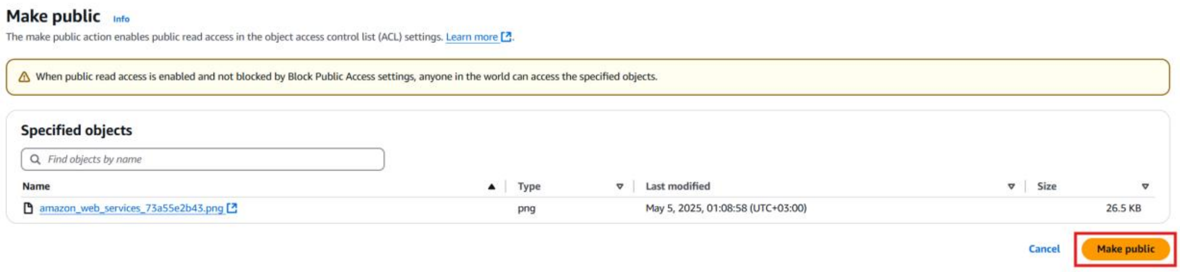
After uploading your file and folders click on **upload**.



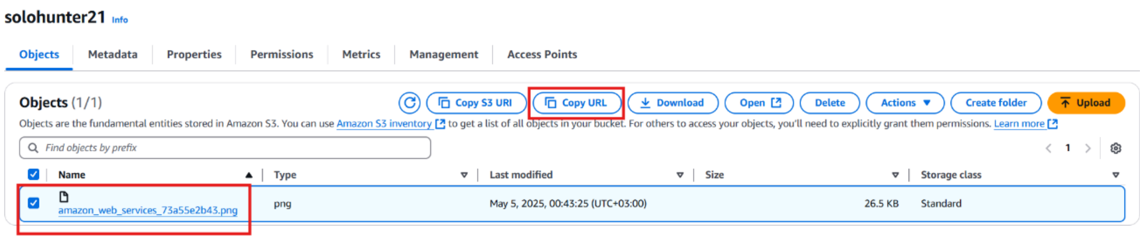
If you try to access the photo that you upload it will return **Access Denied because of the ACL rules**.



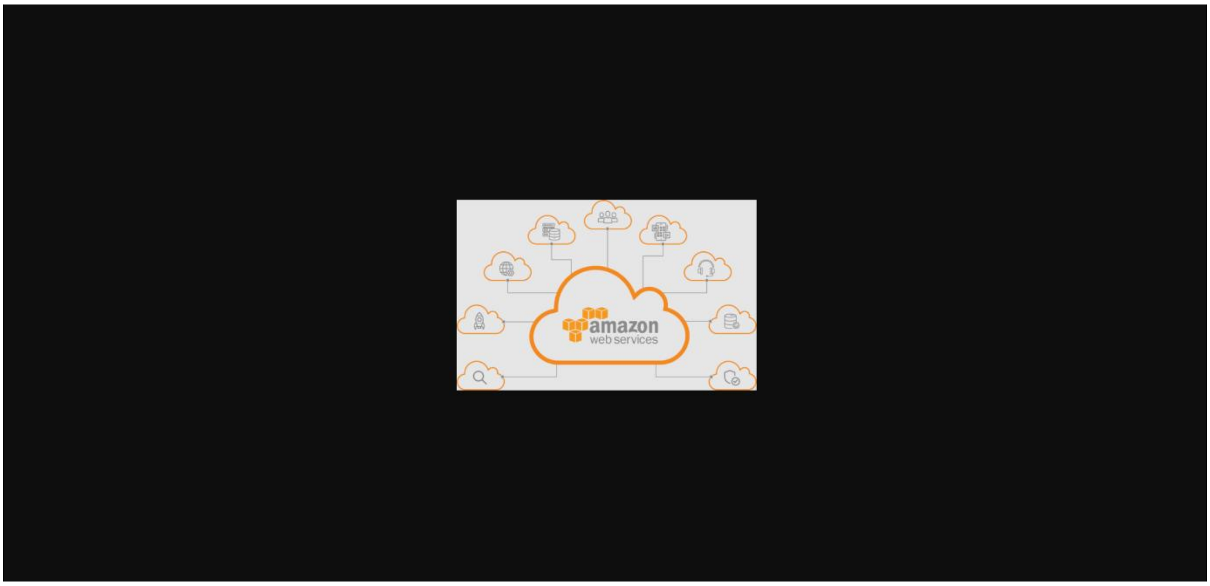
To give the photo permission to be accessed by the public, you select **Actions**, then **Make public using ACL**.



Click on **Make public**.



Select your photo the choose **copy URL** and passed it on **new tab**.

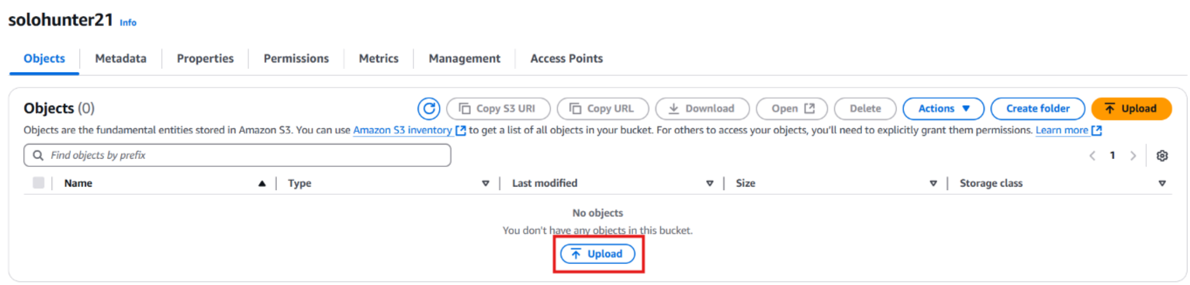


As you can see, the public can now access the photo **until you disable it**.

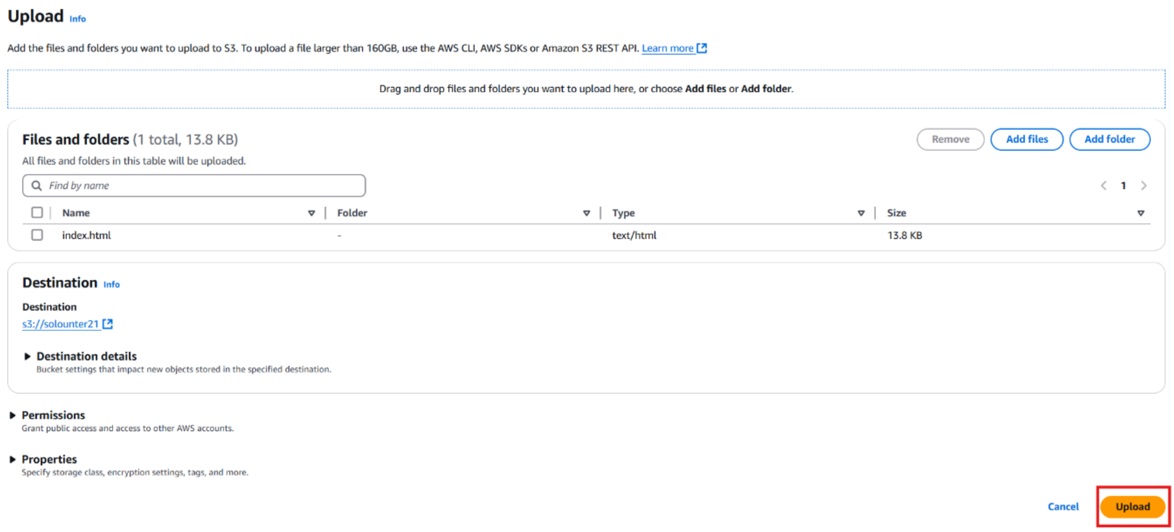
Create a Static Website in S3 Bucket

First, create a bucket with ACL enabled on it, same as in the above demonstration.

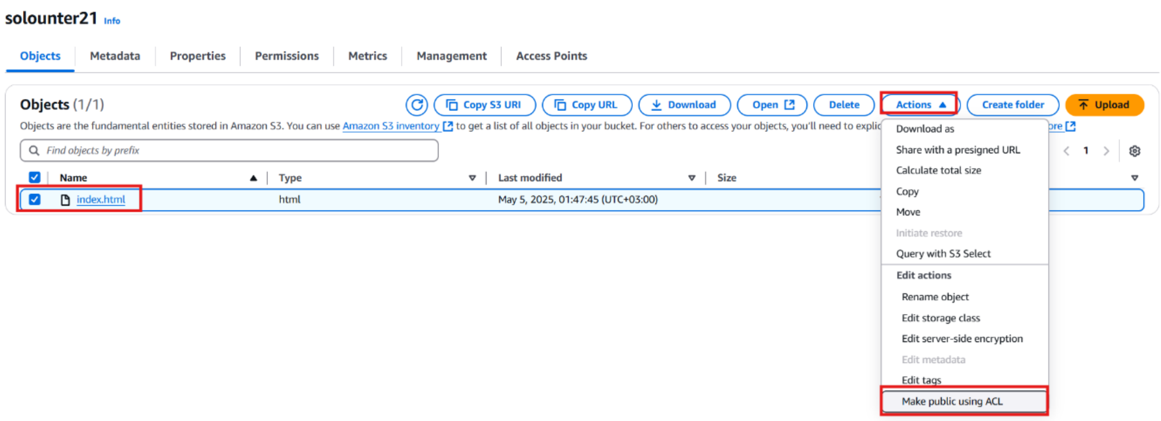
Second, you must have an **index.html** file ready.



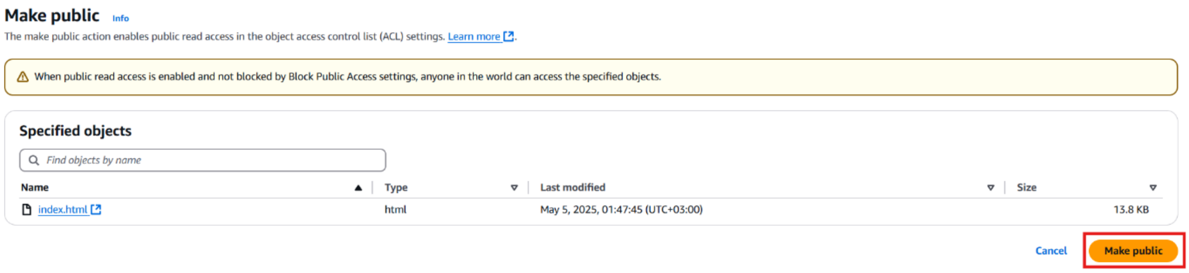
Click on **upload.**



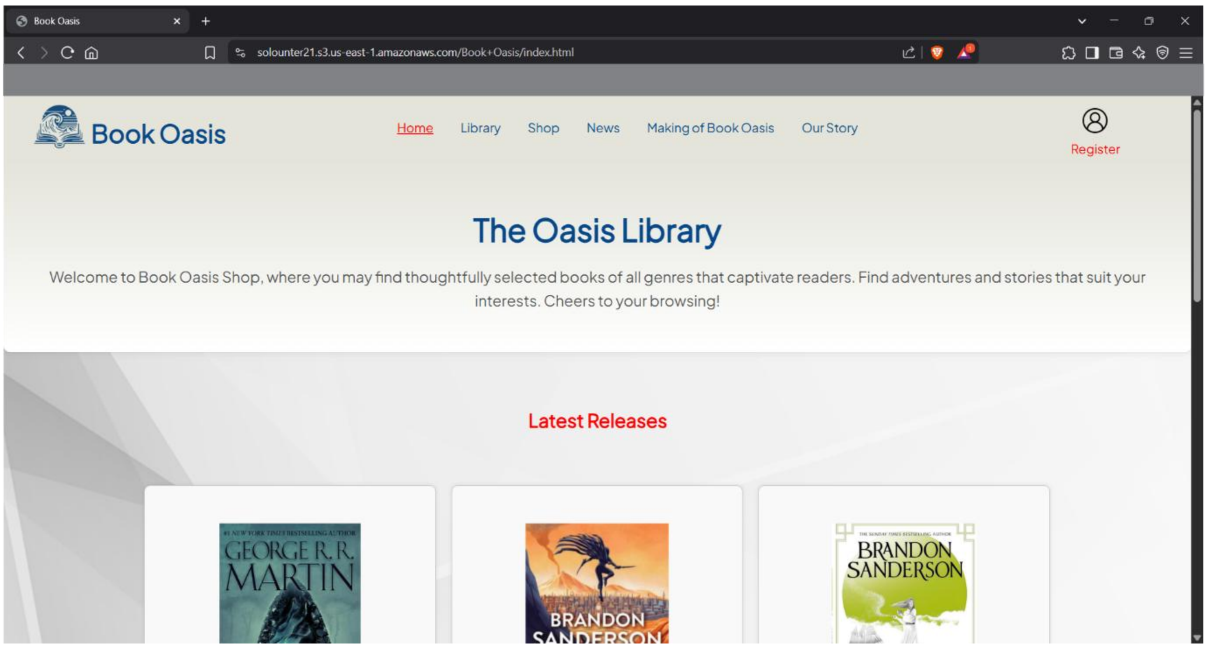
Select Add file and add your index.html file, then click on **upload**. Then return to your bucket.



Select your index file, click on Action, and make it public using ACL.



Choose make Public. Then go back to it and copy the URL. And paste in a new tab.



Now you can see the web page index.html is now accessible using s3 bucket.

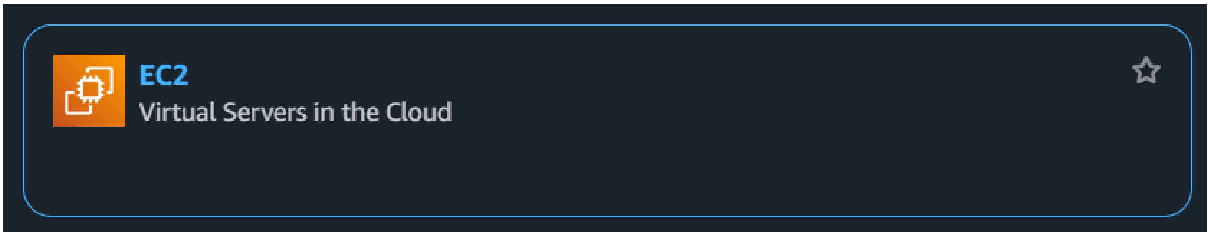
How to Create EBS Storage

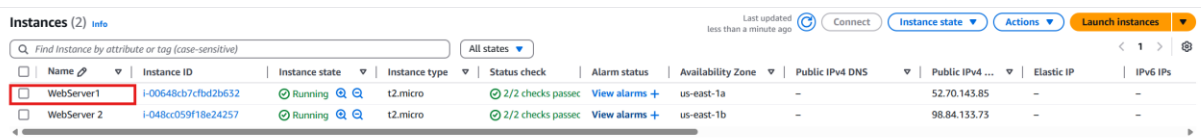
Requirements:

1. 2 EC2 instances that run Ubuntu in different availability zones.
2. 2 public subnets in different availability zones.

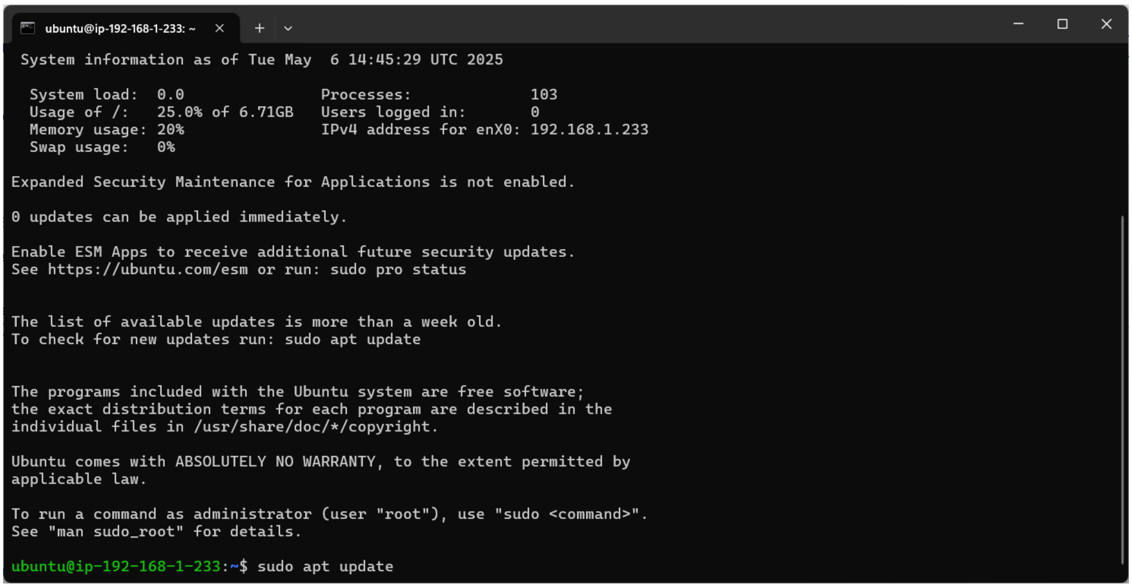
How to create and mount EBS storage in EC2 instance

First, EBS storage is suitable for high-performance workloads that require low latency. However, it must be mounted to a single EC2 instance and must exist in the same availability zone. Additionally, EBS volumes can only be expanded, not shrunk, and are supported only on **Linux**. A method to enable EBS functionality across different availability zones will be shown and explained below.

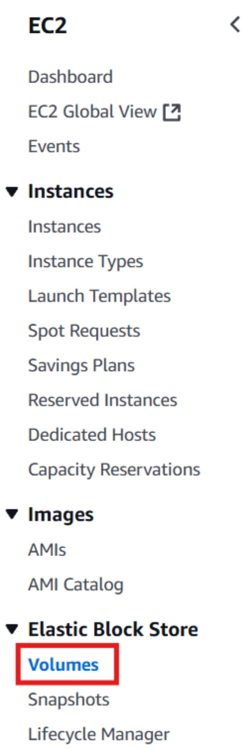


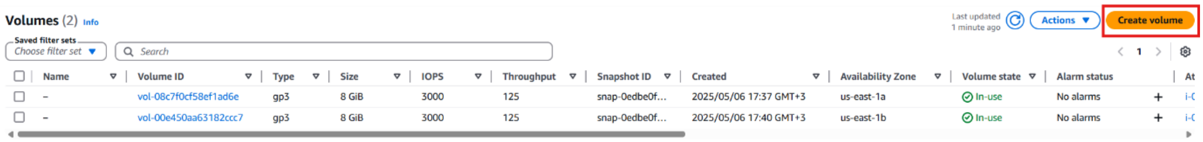


In EC2 instances, connect to one of your EC2 instances. In this demonstration, we will start with webserver 1.

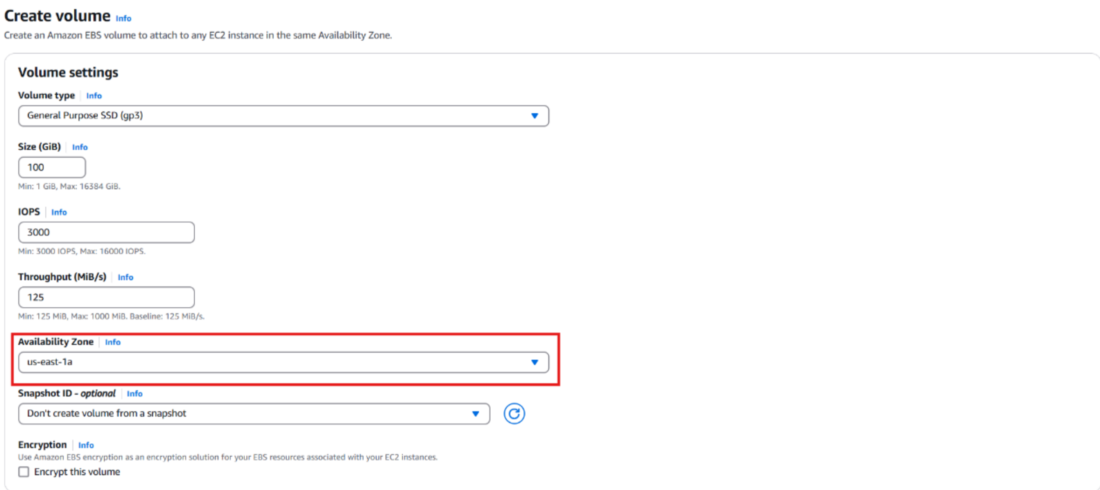


The first thing to do is to **update ubuntu**.

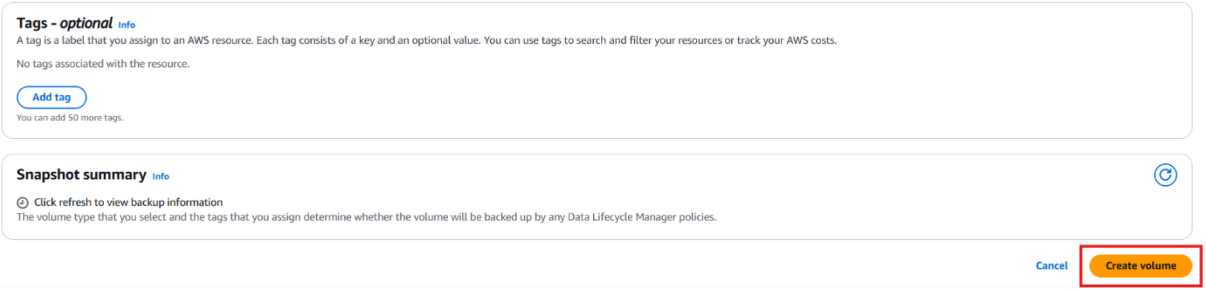




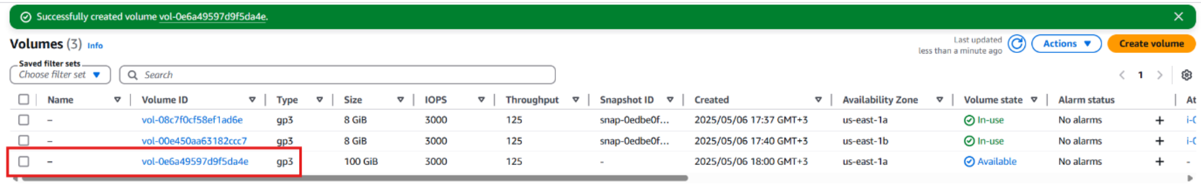
In the left-side menu, choose **Volumes**. You will notice that there are two volumes listed, which are associated with the EC2 instance you have created. Click on **Create Volume**.



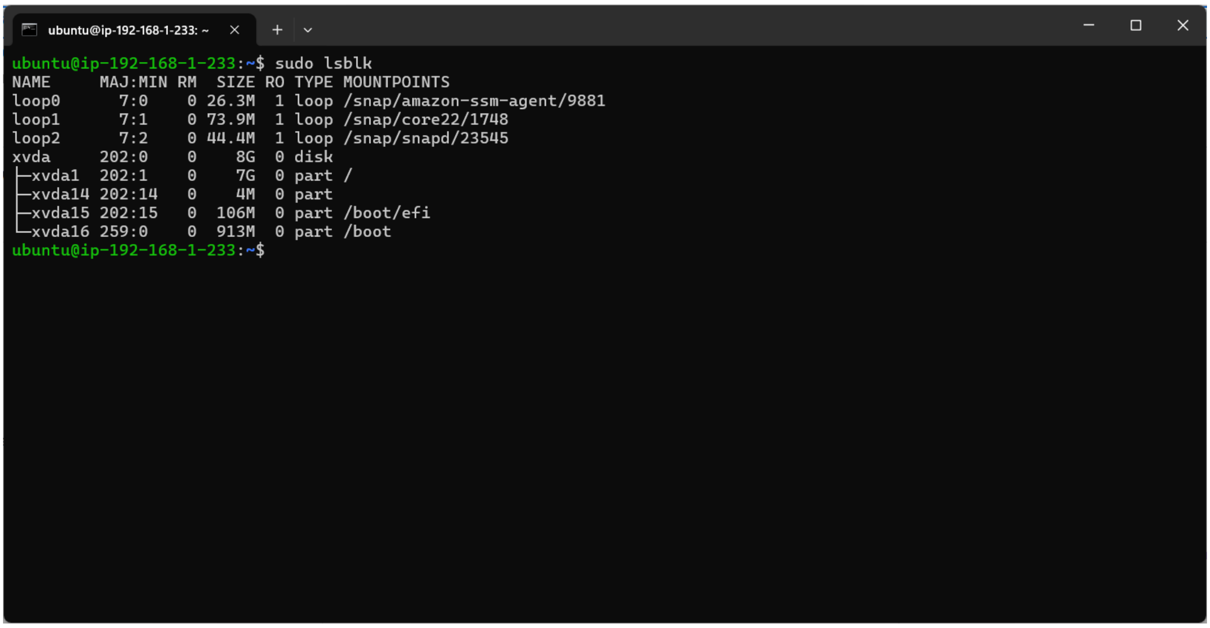
Make sure you choose the **Availability zone the same as your EC2 instance**.



Leave the rest as the default options, and click on **Create Volume**.

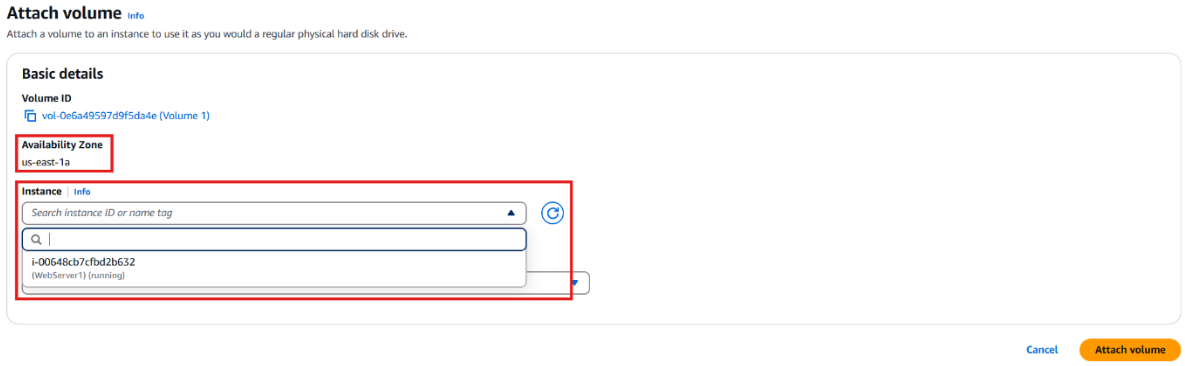


Refresh the page; you will see that the volume has been created in volume state. You will notice that it says **Available,** which means it is **not attached to anything for the meantime,** and **give it a name.**



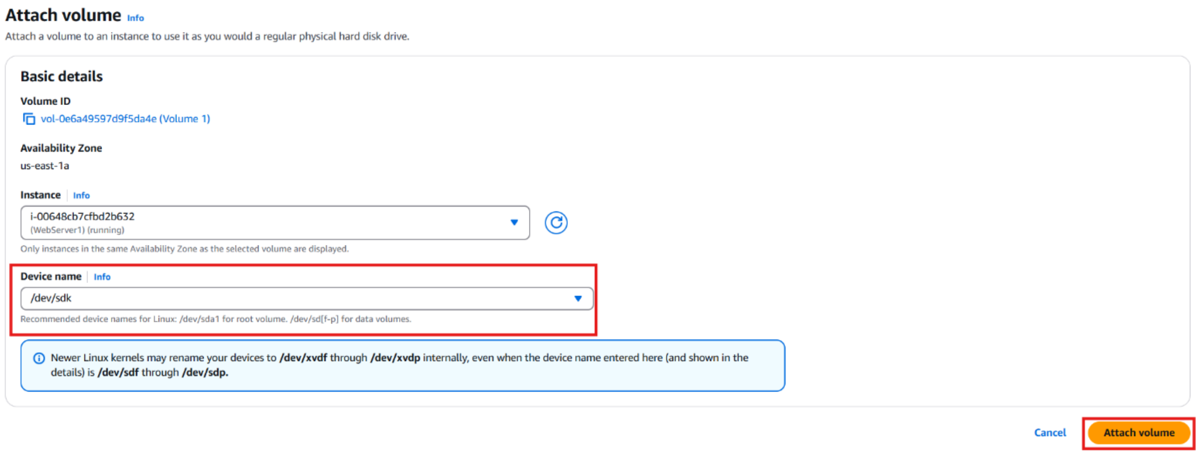
Return to your EC2 and run the command "**sudo lsblk**" this command will list devices, partitions, mount points, and sizes.

Note: you can see all the devices that are connected to your Ubuntu because we did not mount the owner volume that we created, so it will not show.

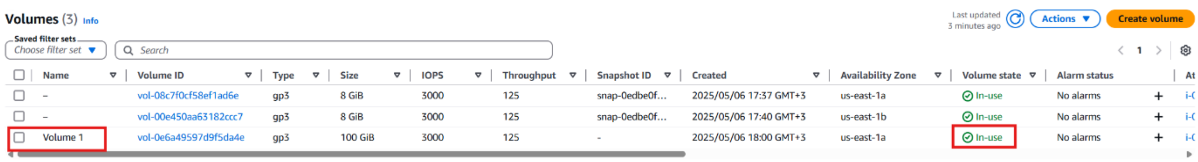


Go back to your AWS console and in volume, select your **volume** and click on **Action,** choose **Attach Volume**.

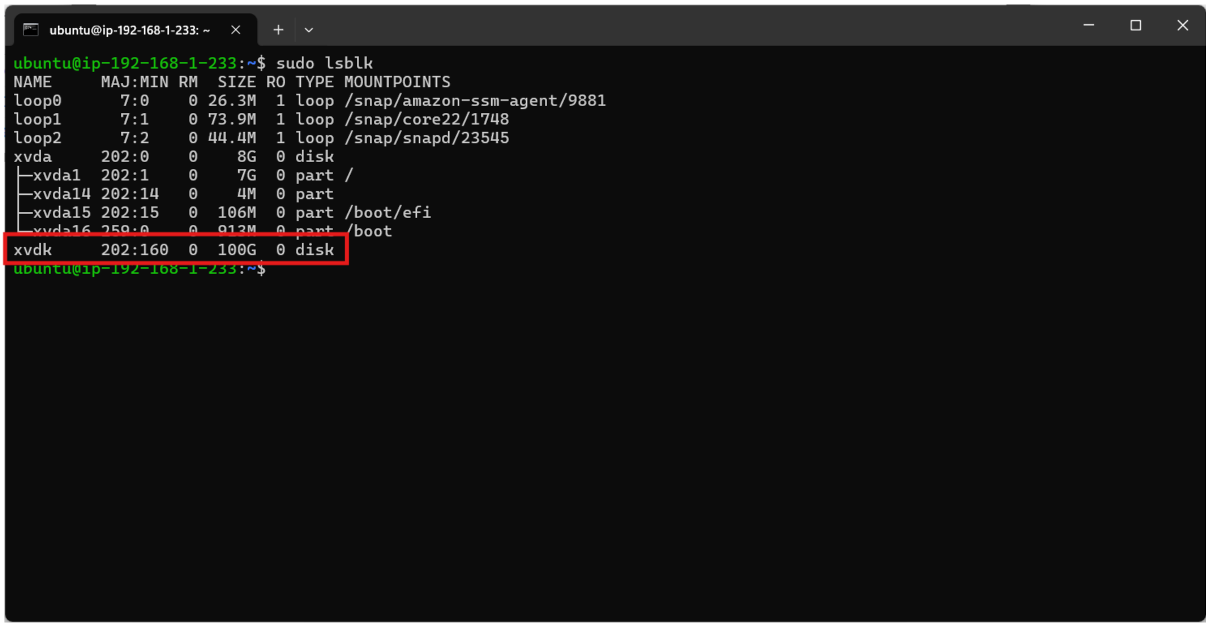
In the **Attach volume under the instance**, you will see only your instances that are in the same Availability zone. Choose your instances.



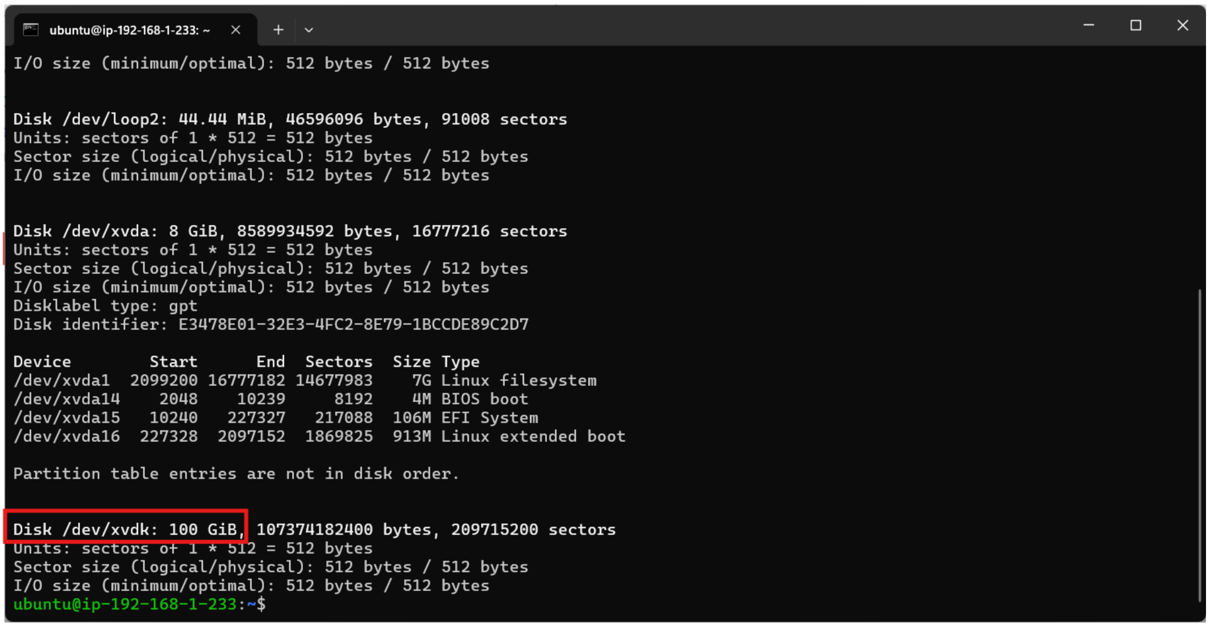
For the **Device Name,** you can choose any name, then click on **Attach Volume**.



You will notice that the volume state changed to in-use and if you go again to your EC2



instance and run the command again "**sudo lsblk**" you will notice it will show you a new volume.



In order to mount the drive to your Linux machine you has to have the name of the disk to get it run the command "**sudo fdisk -l**" this command will list all the disk in your Linux machine scroll down you will see the 100 gib disk that you attached early copy the name "/dev/xvdk" it could be different for you copy what is shown to you.

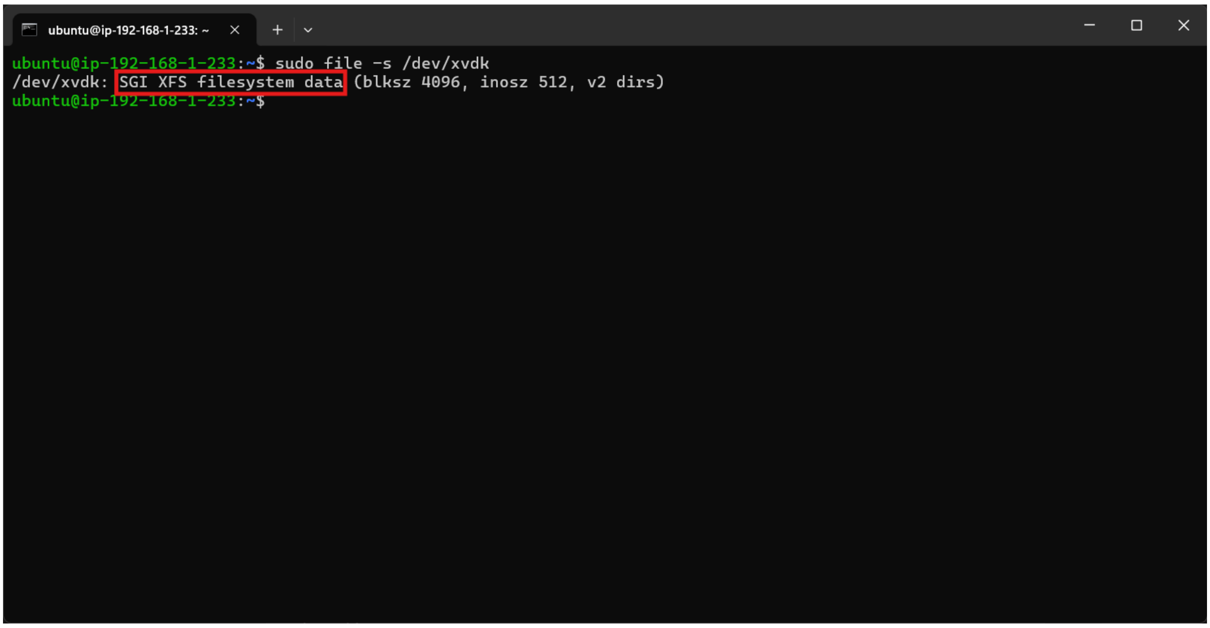
```
ubuntu@ip-192-168-1-233:~$ sudo file -s /dev/xvdk
/dev/xvdk: data
ubuntu@ip-192-168-1-233:~$
```

If you run the command “**sudo file -s <your drive-name>**”, it will show you that it is a date that means it is raw, not formatted, and used by the Linux system.

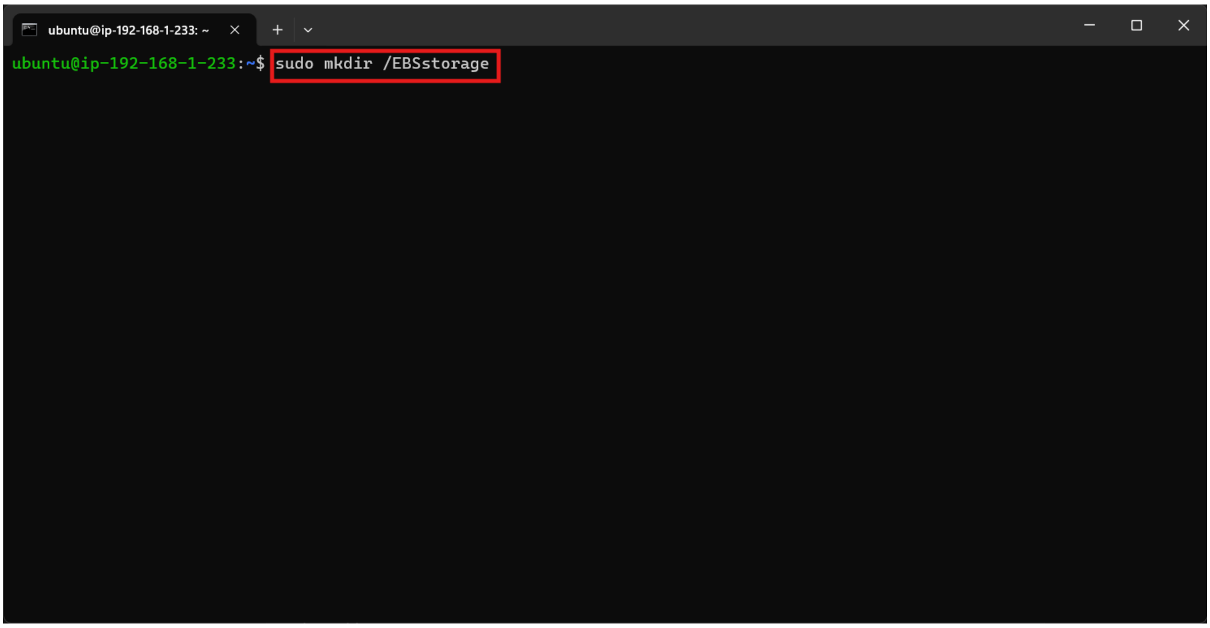
```
ubuntu@ip-192-168-1-233:~$ sudo mkfs -t xfs /dev/xvdk
meta-data=/dev/xvdk              isize=512    agcount=4, agsize=6553600 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=1
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=26214400, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
ubuntu@ip-192-168-1-233:~$
```

To use the drive, you must make a file system on it by running the command “**sudo mkfs -t <file system> <name of the drive>**”

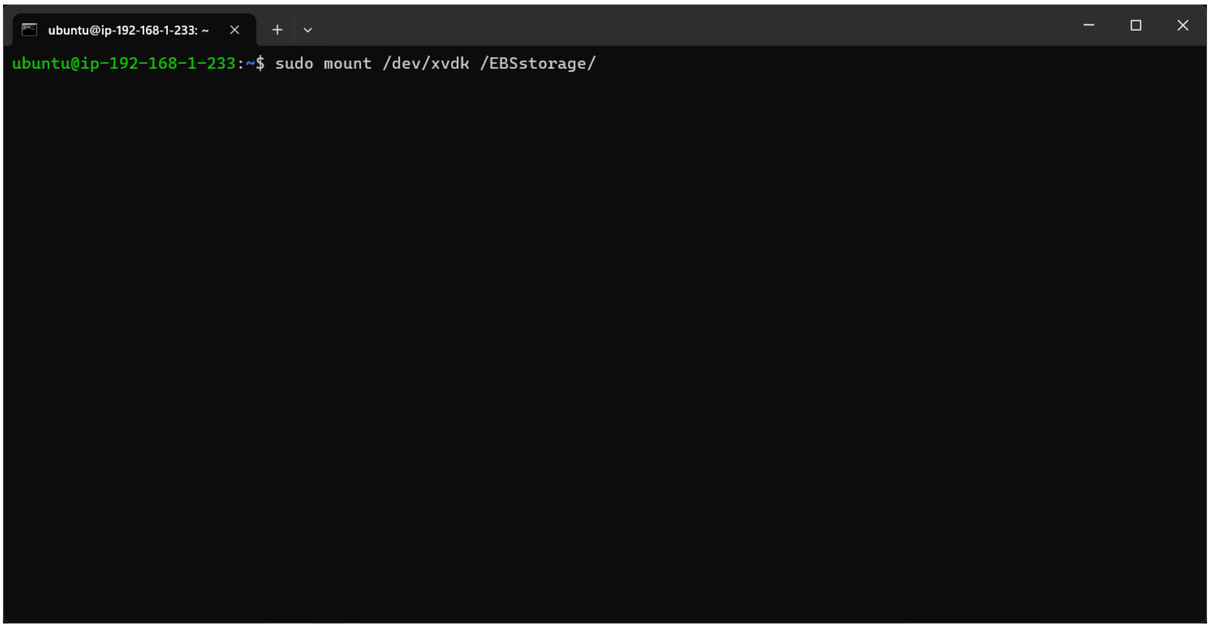
This command will create a file system on the drive so you can use it.



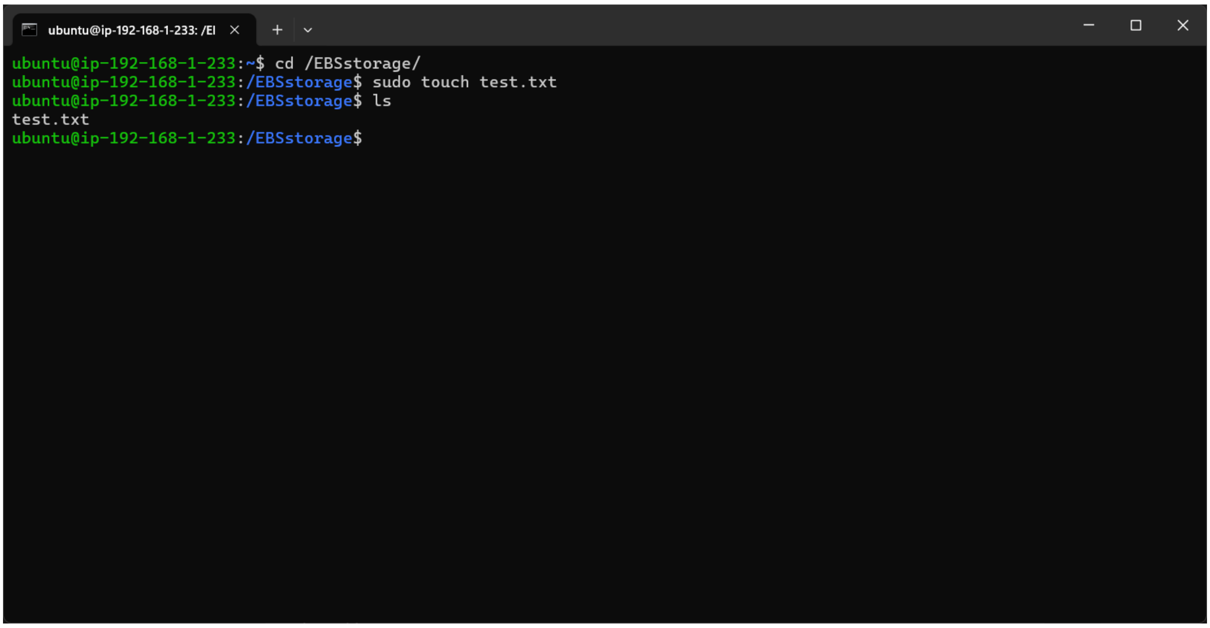
To check if the drive creates the file system by running the command "**sudo file -s <drive name>**", as you can see, the file system has been created on the drive.



To mount the drive, you must have a folder to mount it on. First, create a Folder by running the command "**sudo mkdir <directory name>**"

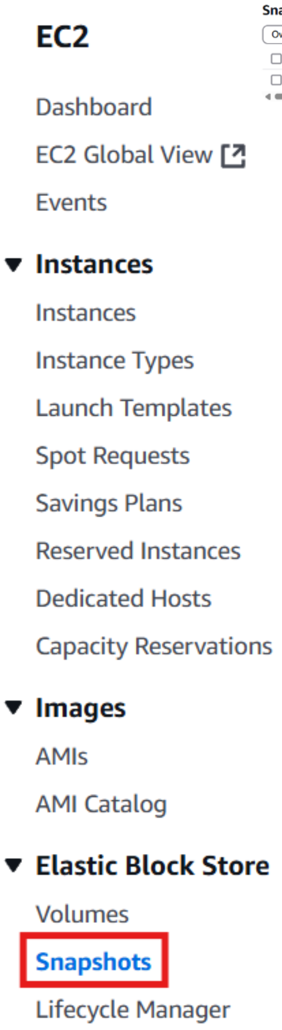


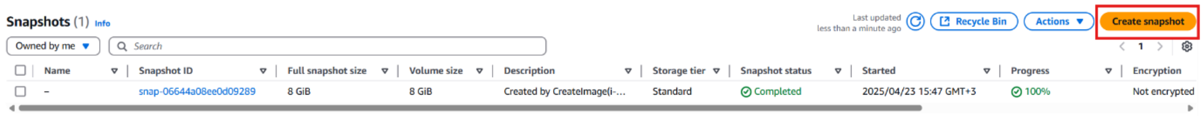
After creating the directory, you must mount the drive on the directory by issuing the command “**sudo mount <drive name> <directory location and name>**”.



After mounting the drive, go to the directory and **create a file in it**.

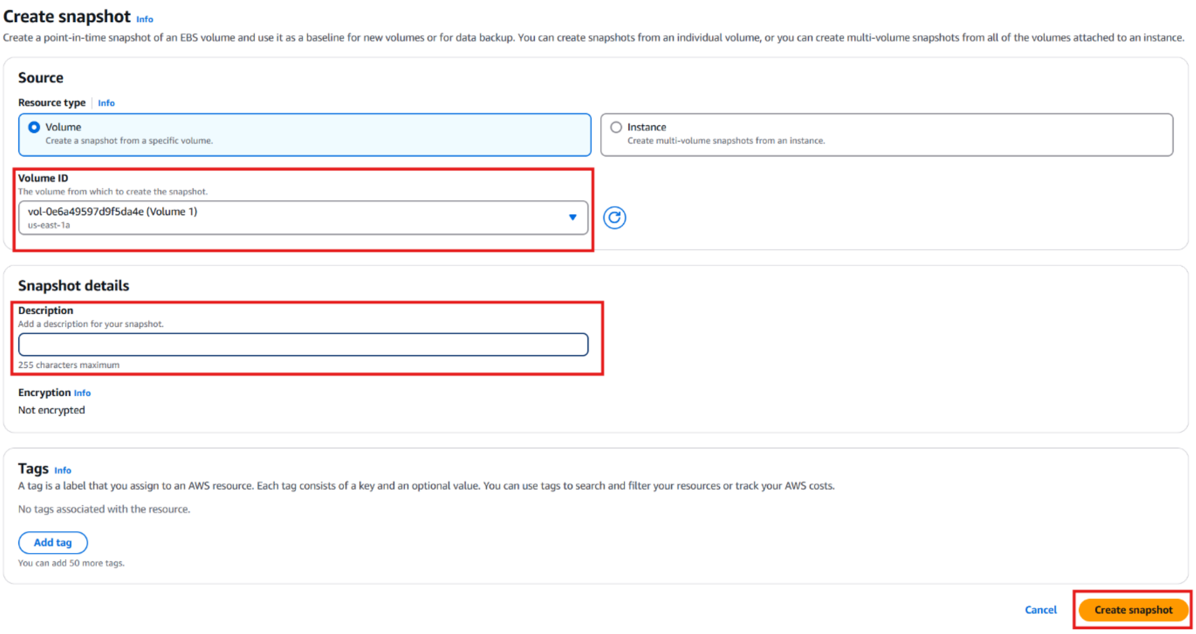
Create Snapshot from EBS



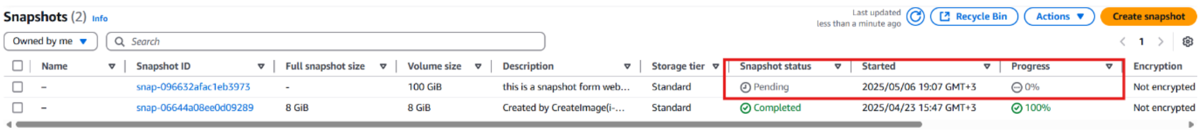


Return to your AWS console and in the left-side menu in your **EC2** select **snapshot**, then click on the **snapshot**.

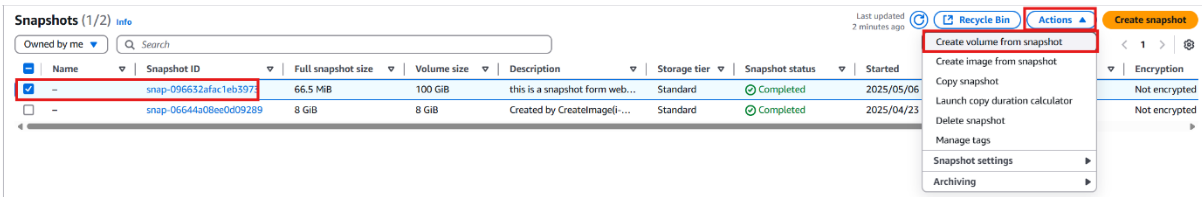
Note: because of the limitation of the EBS, you cannot mount EBS in two different EC2 instances in different Availability zones, but you can take a copy of the drive by using a snapshot.



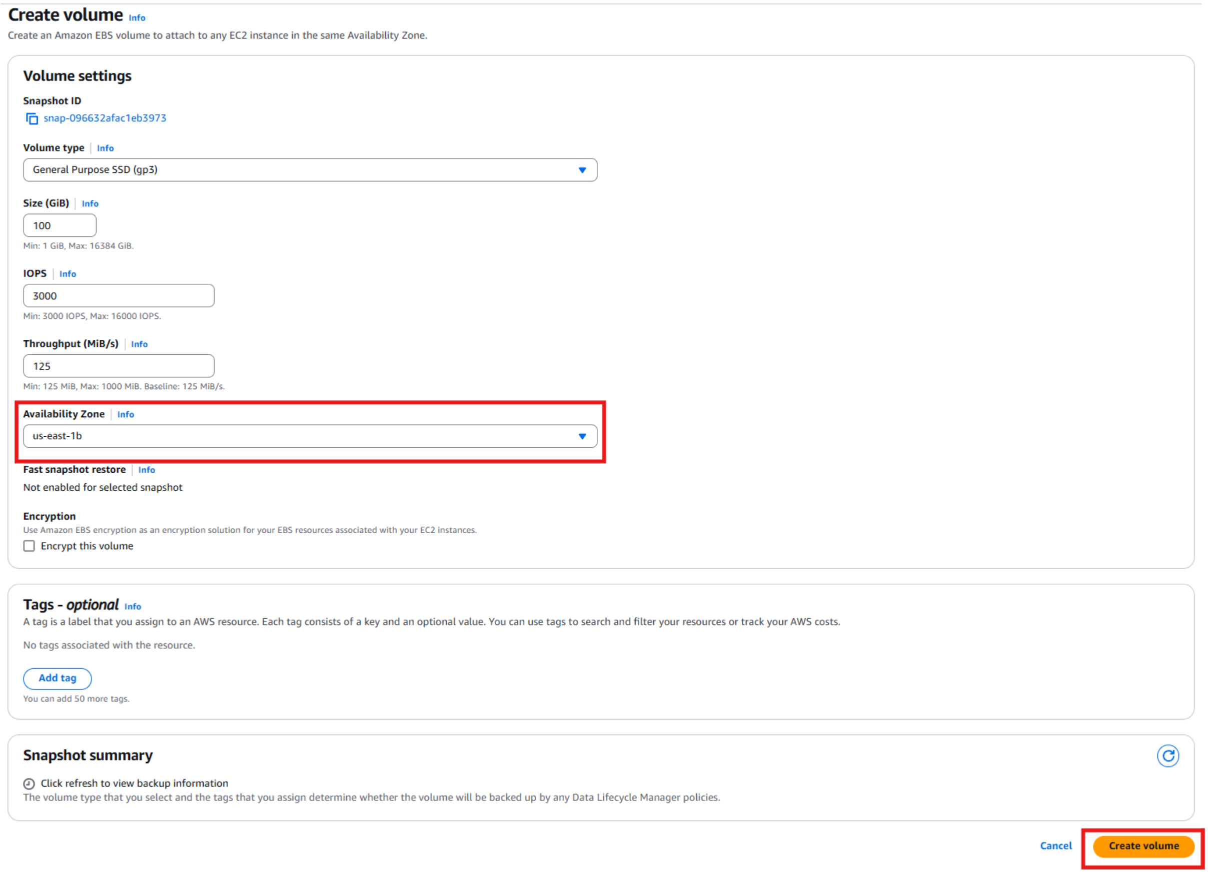
First, make sure you select the Volume for the Resource type. Second, select your Volume. Third, write a meaningful description. Last click on **create snapshot**.



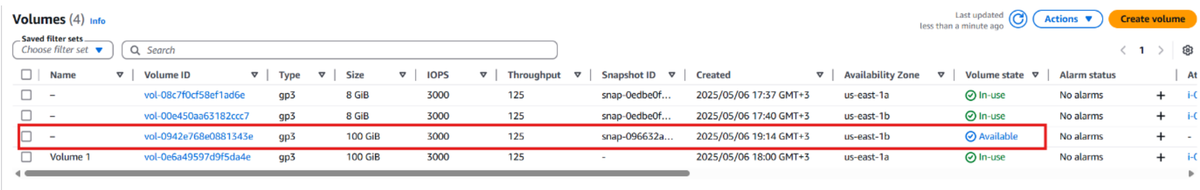
After creating the snapshot, wait for some time for the snapshot state to say **complete**.



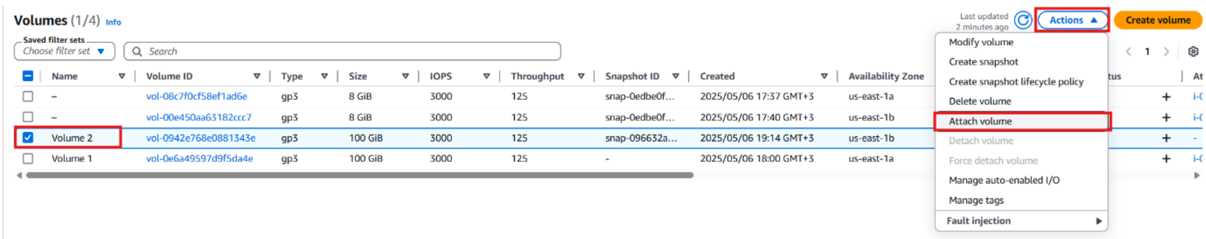
After the snapshot is completed, select the ID and click on **Action,** choose **Create Volume from snapshot.**



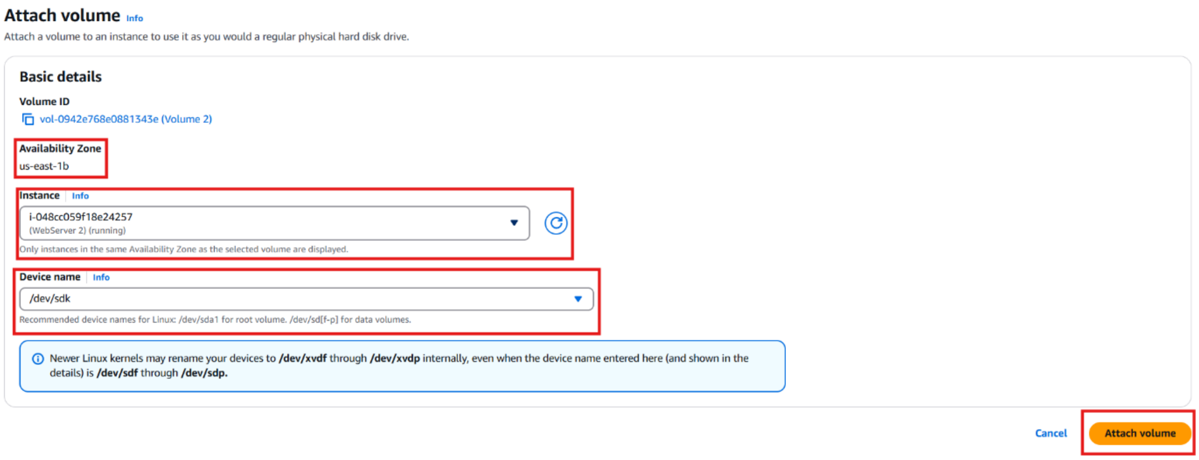
Select your **Availability Zone** that matches your **second EC2 instance**. Leave the rest as a default and click on Create Volume.



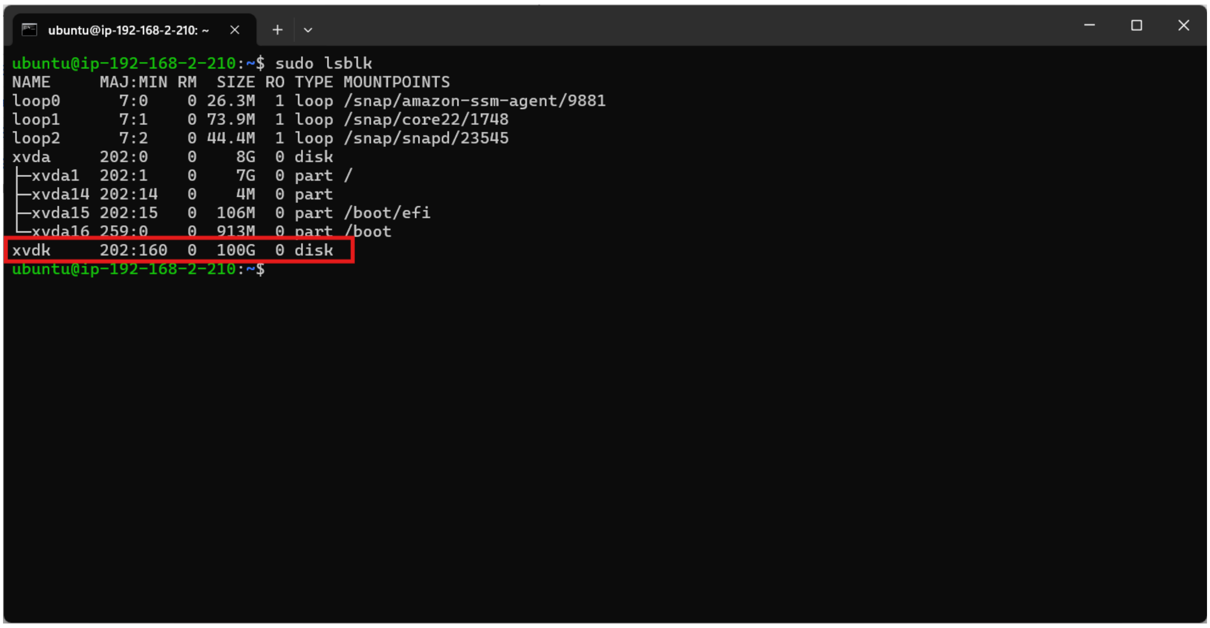
From the left-side menu, **select volume**. You will find the new volume **that you created from the snapshot. Give it a name.**



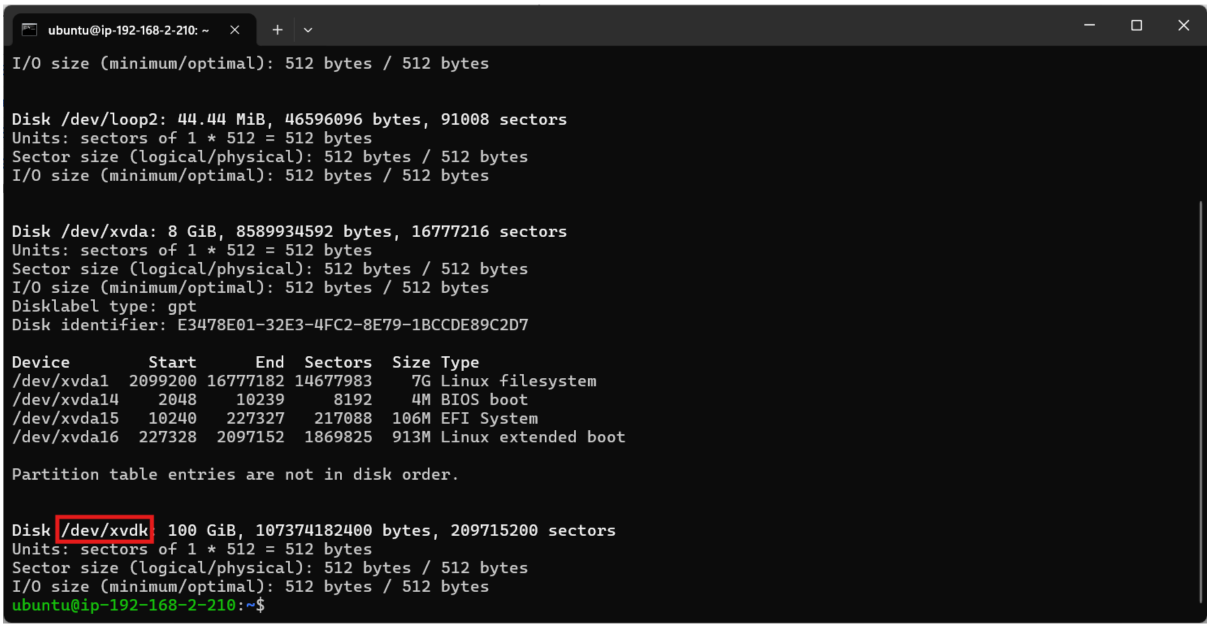
Now, you must **attach the Volume to your second EC2 instance**. Select the volume you want to attach, click on **Action,** and choose from the drop-down menu **Attach volume**.



Select your **second EC2 instance** and choose a Device name, then click on **Attach volume**.

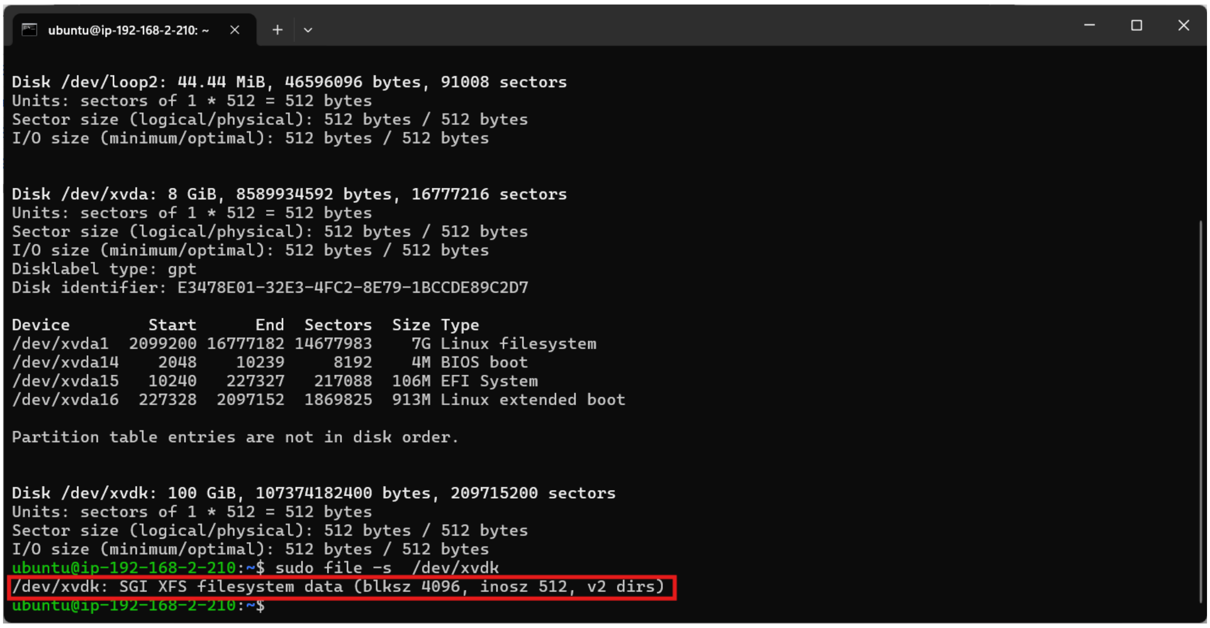


Connect to your second EC2 instance and update Ubuntu, then check if the disk is attached, run the command "**sudo lsblk**"; as you can see, the disk is attached.



```
ubuntu@ip-192-168-2-210: ~
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/loop2: 44.44 MiB, 46596096 bytes, 91008 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: E3478E01-32E3-4FC2-8E79-1BCCDE89C2D7

Device        Start      End  Sectors  Size Type
/dev/xvda1  2099200 16777182 14677983    7G Linux filesystem
/dev/xvda14    2048    10239     8192    4M BIOS boot
/dev/xvda15   10240   227327   217088  106M EFI System
/dev/xvda16  227328  2097152  1869825  913M Linux extended boot

Partition table entries are not in disk order.


Disk /dev/xvdk: 100 GiB, 107374182400 bytes, 209715200 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
ubuntu@ip-192-168-2-210:~$
```

To check the file system of the drive. First, you must know the name of the drive by running the command “**sudo fdisk -l**” to list all the drives, and scroll down to see your disk name. Copy it.



```
ubuntu@ip-192-168-2-210: ~

Disk /dev/loop2: 44.44 MiB, 46596096 bytes, 91008 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: E3478E01-32E3-4FC2-8E79-1BCCDE89C2D7

Device        Start      End  Sectors  Size Type
/dev/xvda1  2099200 16777182 14677983    7G Linux filesystem
/dev/xvda14    2048    10239     8192    4M BIOS boot
/dev/xvda15   10240   227327   217088  106M EFI System
/dev/xvda16  227328  2097152  1869825  913M Linux extended boot

Partition table entries are not in disk order.


Disk /dev/xvdk: 100 GiB, 107374182400 bytes, 209715200 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
ubuntu@ip-192-168-2-210:~$ sudo file -s  /dev/xvdk
/dev/xvdk: SGI XFS filesystem data (blksz 4096, inosz 512, v2 dirs)
ubuntu@ip-192-168-2-210:~$
```

After obtaining the name of the drive, you can check the file system of it by issuing the command “**sudo file -s <drive name>**”. As you can see, the drive is using the XFS file system.

```
ubuntu@ip-192-168-2-210: /E

Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: E3478E01-32E3-4FC2-8E79-1BCCDE89C2D7

Device         Start      End  Sectors  Size Type
/dev/xvda1   2099200 16777182 14677983    7G Linux filesystem
/dev/xvda14     2048    10239     8192    4M BIOS boot
/dev/xvda15    10240   227327   217088  106M EFI System
/dev/xvda16   227328  2097152  1869825  913M Linux extended boot

Partition table entries are not in disk order.


Disk /dev/xvdk: 100 GiB, 107374182400 bytes, 209715200 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
ubuntu@ip-192-168-2-210:~$ sudo file -s  /dev/xvdk
/dev/xvdk: SGI XFS filesystem data (blksz 4096, inosz 512, v2 dirs)
ubuntu@ip-192-168-2-210:~$ sudo mkdir /EBS-storage
ubuntu@ip-192-168-2-210:~$ sudo mount  /dev/xvdk /EBS-storage/
ubuntu@ip-192-168-2-210:~$ cd /EBS-storage/
ubuntu@ip-192-168-2-210:/EBS-storage$ ls
test.txt
ubuntu@ip-192-168-2-210:/EBS-storage$
```

Now you need to mount the drive to a directory. First make a directory by using the command “**sudo mkdir <directory name>**” Second, mount the drive by running the command “**sudo mount <drive name> <directory location and name>**” Third, to check If the file you created in the first instance is there or not issue the command “**ls**” will list everything in the folder and that is how to create a copy of your EBS drive and mounted in different EC2 instance in different availability zone.

If you have done everything, the correct file will be there.

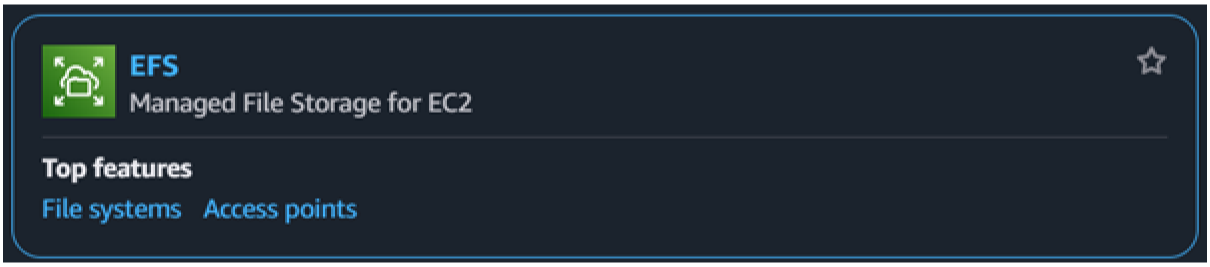
How to Create EFS Storage

Requirements:

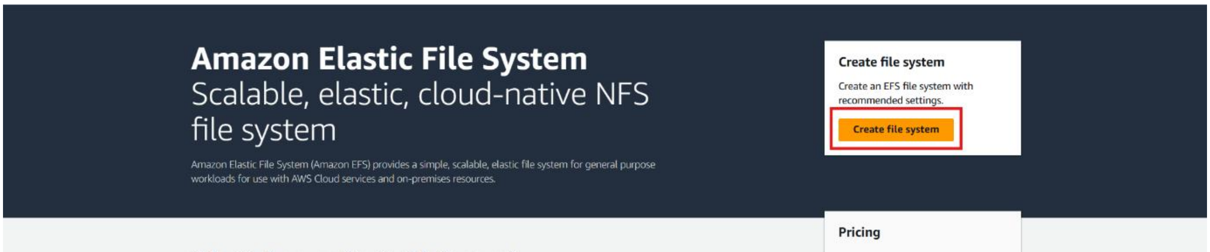
3. [2 EC2] instances that run Ubuntu in different availability zones.
4. [2 public] subnets in different availability zones.

How to create and mount EFS Storage in EC2 instance

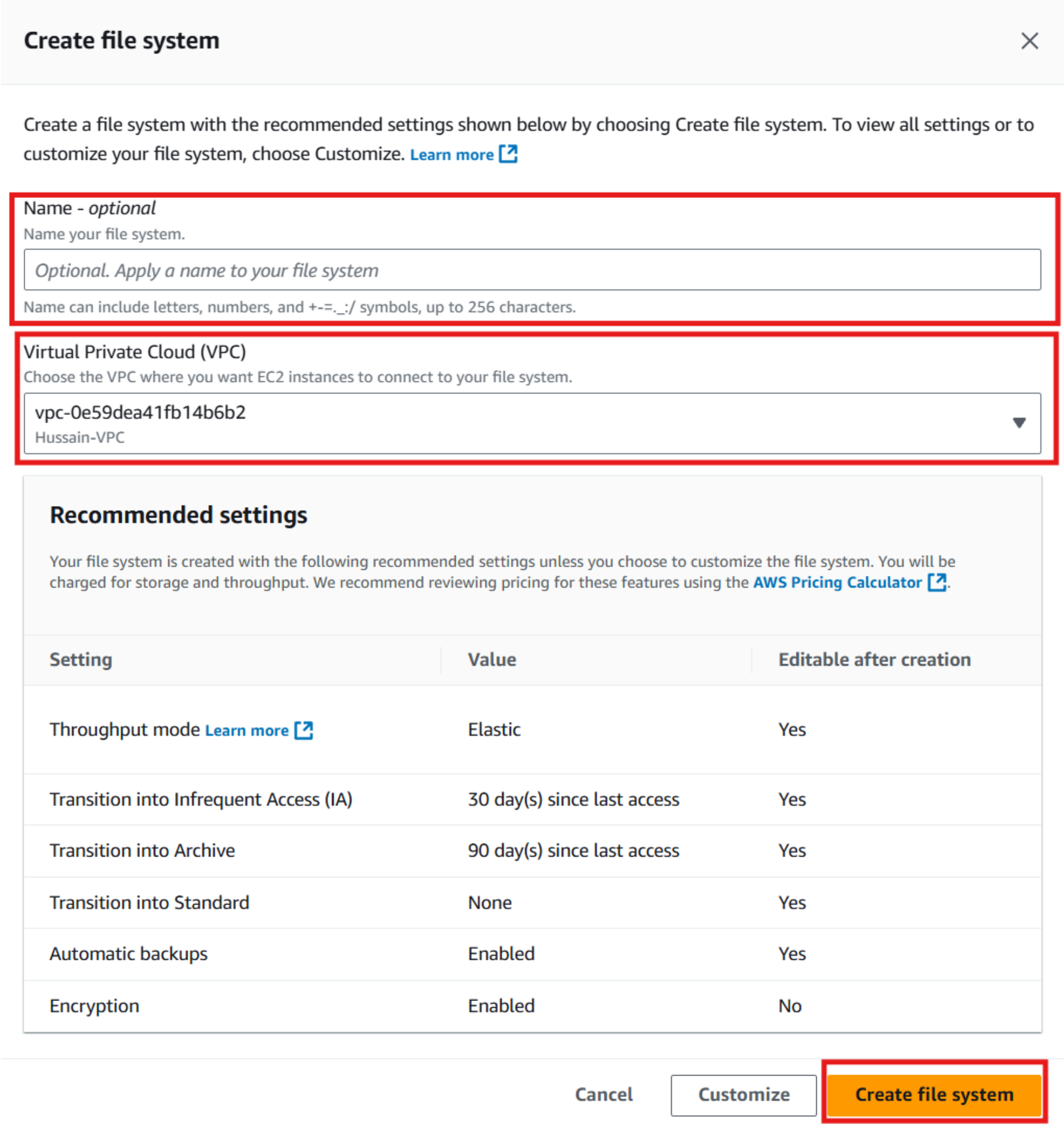
EFS Storage can be mounted in multiple EC2 instances, and at the same time, you can shrink or expand your drive because it is elastic. It is best if you need shared storage across multiple instances. In this demonstration, we will use a single EFS volume to be connected to 2 EC2 instances in different Availability zones.



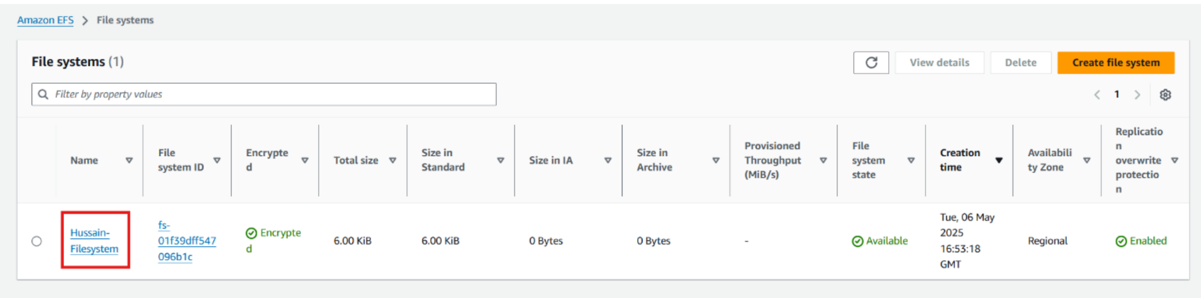
Create EFS File System



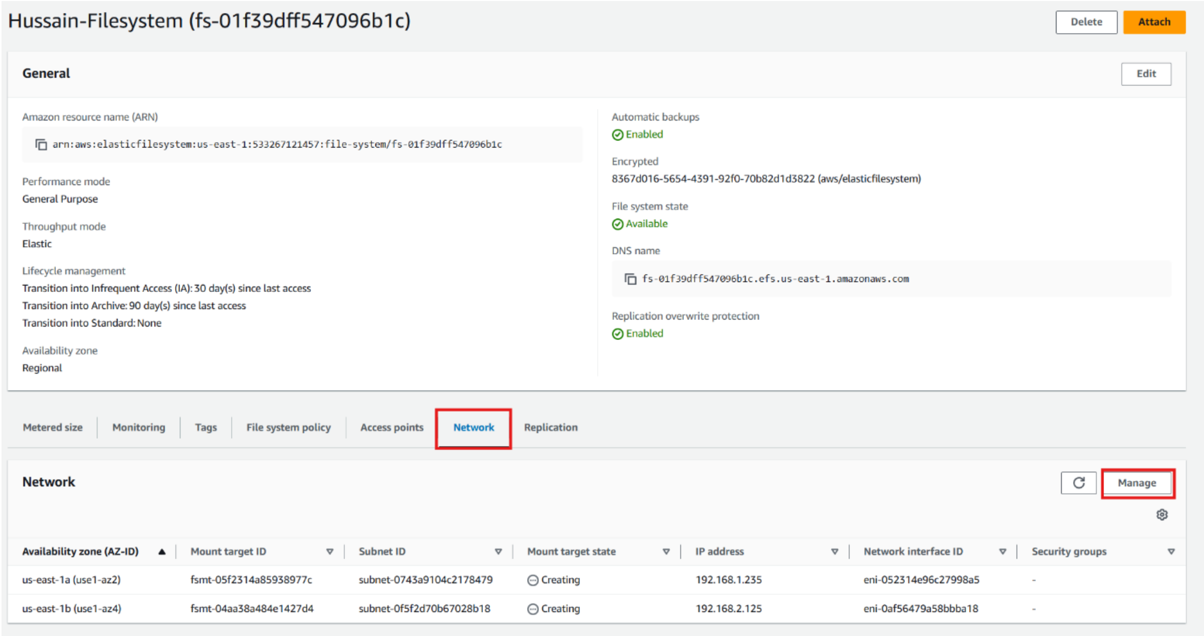
Go to **EFS** and click on **Create file system**.



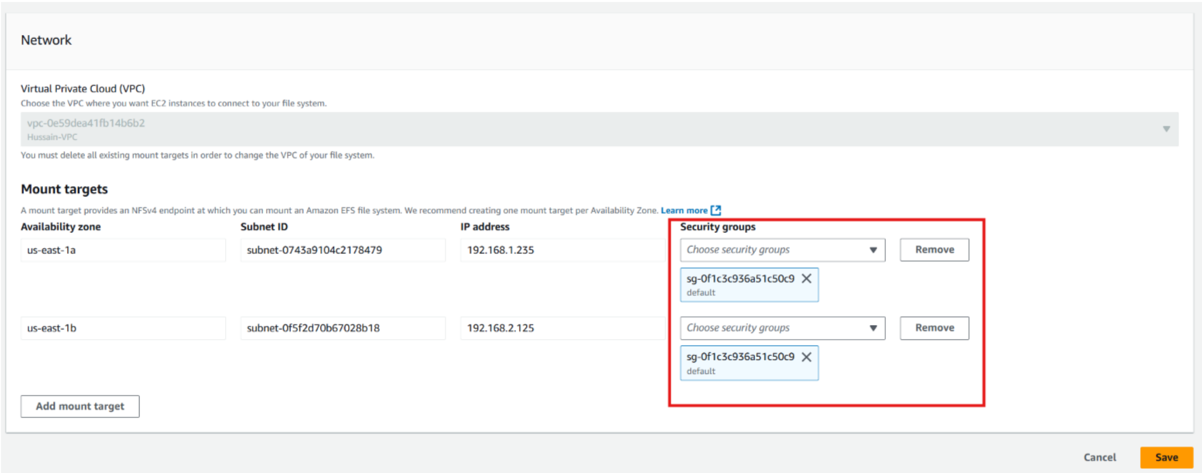
Choose a **name** for your file system and **choose your VPC,** then click on create file system.



After successfully creating the file system, **click on your file system**.



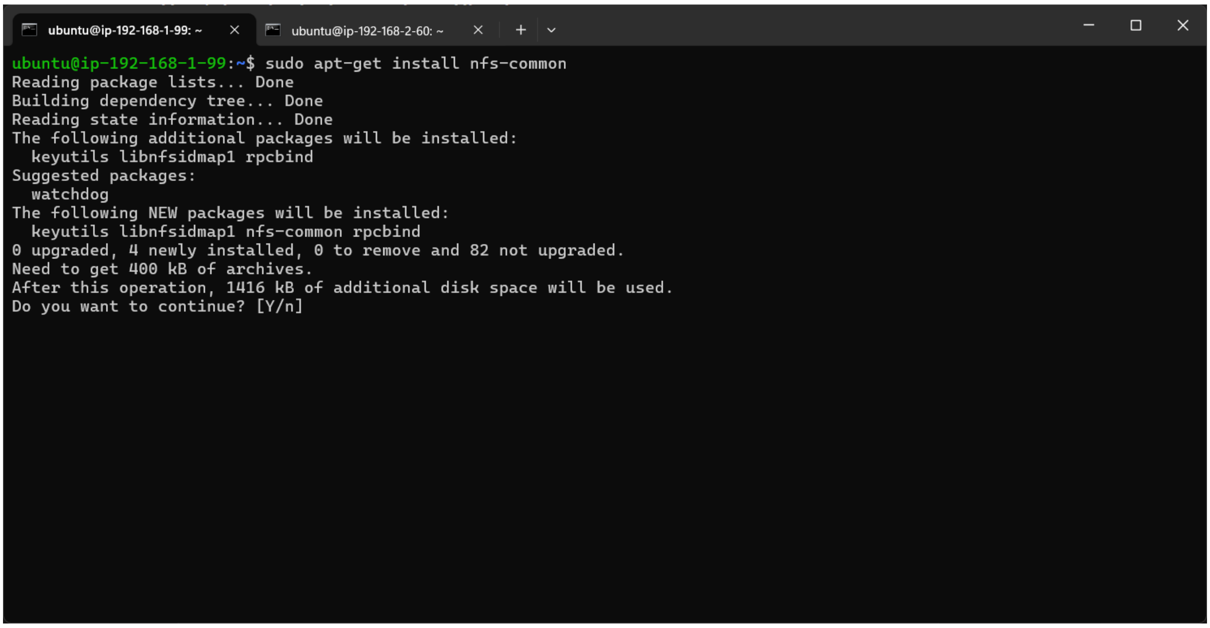
Scroll down and choose from the drop-down menu, **Network,** you will notice it is creating for you in every Availability zone your VPC has. After the mount target becomes Available, click on **Manage**.



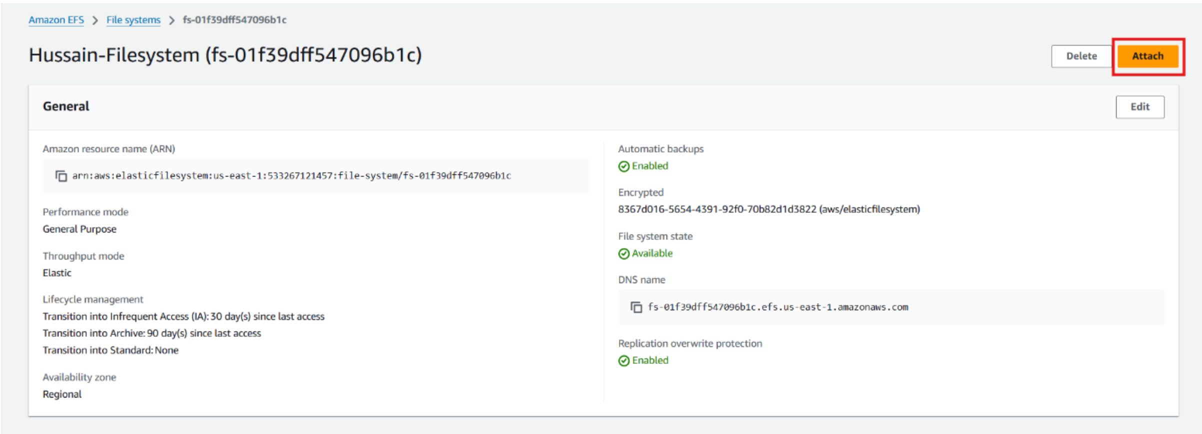
In this tab, you can remove the filesystem that is in the Availability zone you don't need, and remove the default option of the security group from your security group that you created before, then click save.

Note: In some cases, if you click on save, it will give you an error message saying failed. All you have to do is click back on manage, make sure that you did everything correctly, and click on save again, and it will *hopefully* give you a success message.

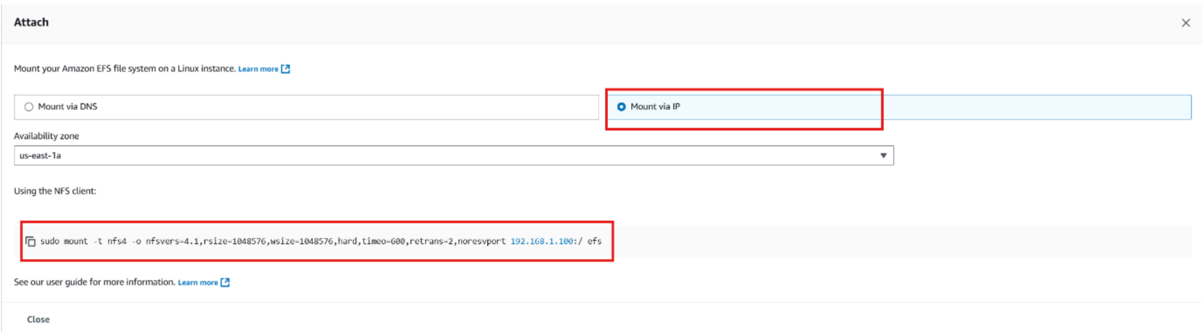
Mounting the EFS drive to EC2 instance



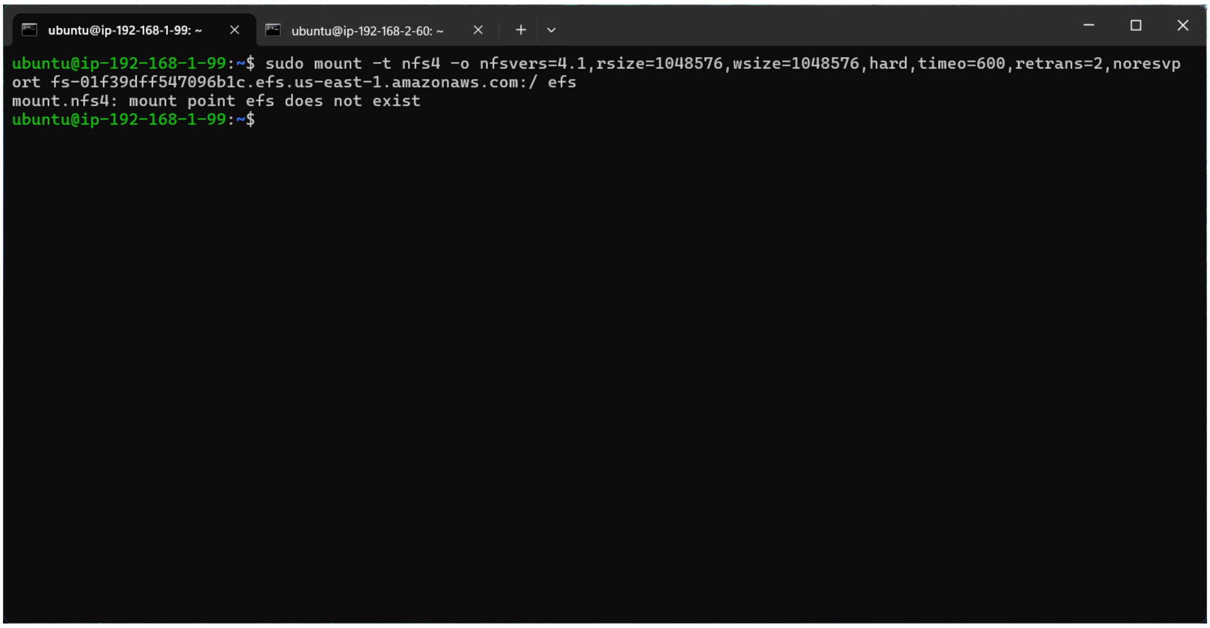
Connect to your EC2 instance and update Ubuntu, then install the nfs client to mount the EFS volume by running the command “**sudo apt-get install nfs-common**”. You will be asked to continue, enter “**Y**”.



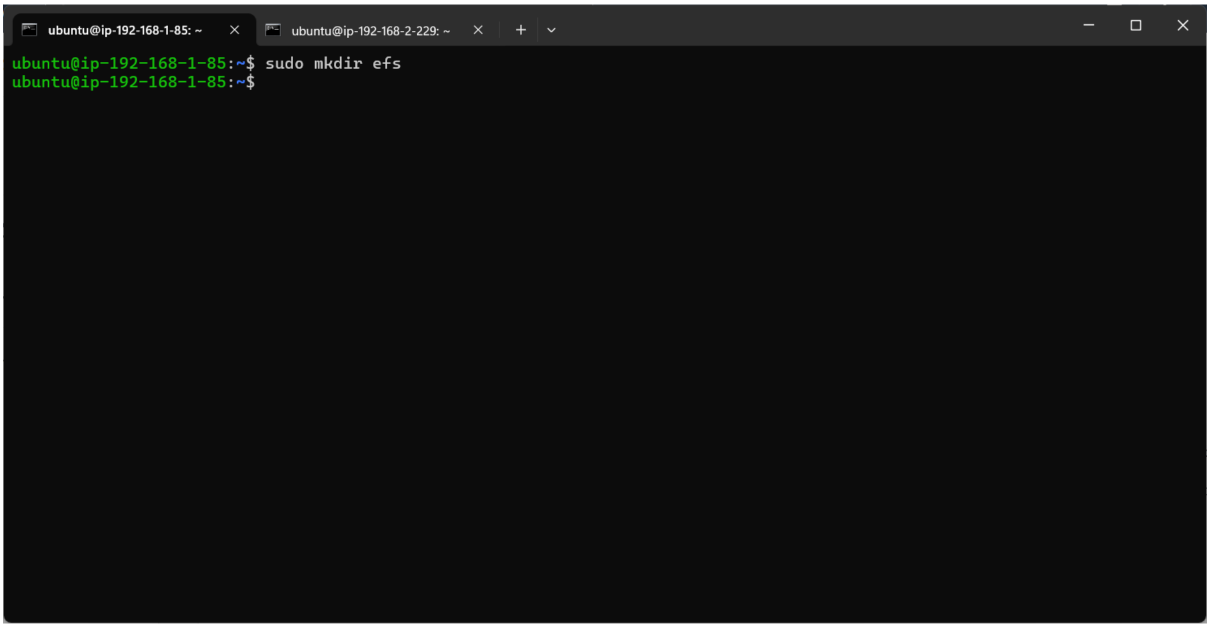
Go back to your file system and click on Attach.



Choose **mount via IP** and **select your availability zone for your first EC2 instance,** then copy the command.

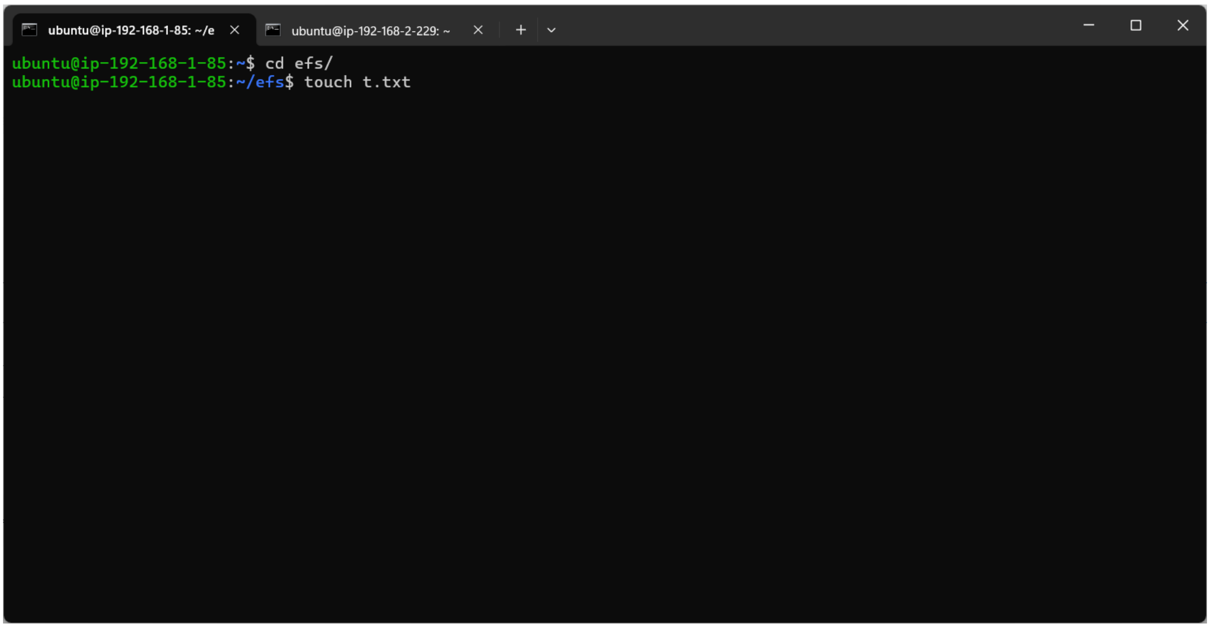


If you try to run the command, you will get an error message that says **mount point efs does not exist**. The reason for the error message is that there is **no efs directory** to mount the drive on it. As you know, you can't mount a drive without specifying the directory in Linux.

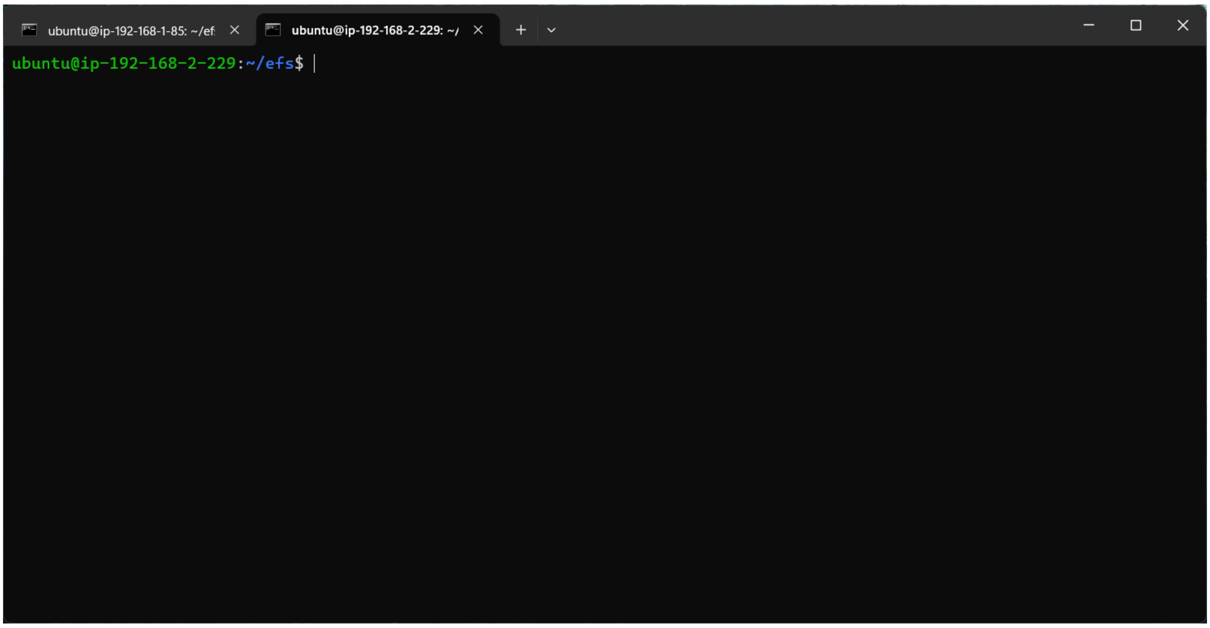


You must create a directory named "**efs**" and then paste the command you copied.

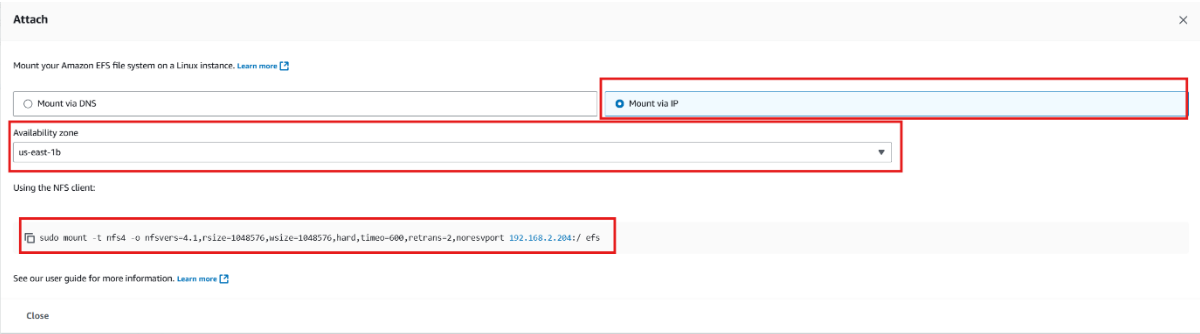
Mounting EFS drive in Different Availability zone



Now, go into your EFS directory and create a file to test if the drive is working in multiple Availability zones.

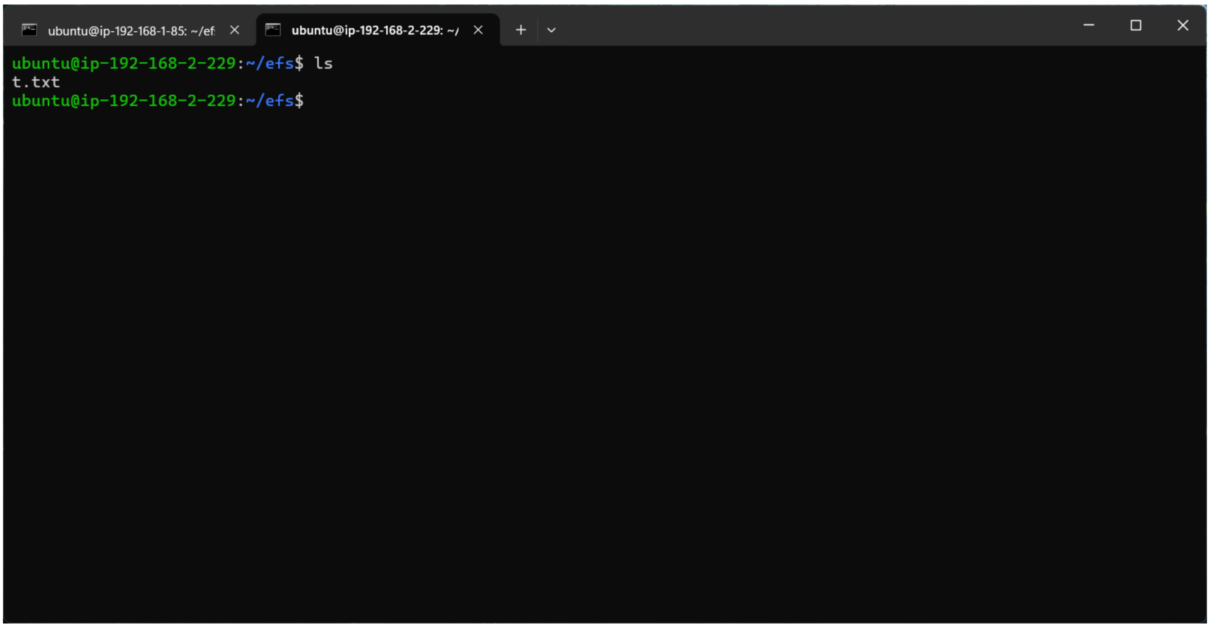


Now **connect to your second EC2 instance** and **update Ubuntu** and **install the nfs client** by running the command “**sudo apt-get install nfs-common**”.



Go back to your **EFS file system** and click on **Attach,** choose **mount via IP,** and choose your **second availability zone where your second EC2 instance** is, **copy the command** and **paste in your EC2 instance after you create a directory named efs** by running the command "**sudo mkdir efs**"

Verifying Cross-Instance File Availability Between Two EC2 Instances



Go inside your EFS directory and run "**ls**" to list the contents of the file. You will see the file you created in the first EC2 instance.

Note: the benefit of EFS is that it can be mounted in different Availability zones and have the same data without the fuss of creating a snapshot and mount the snapshot in EBS

GOOD LUCK